

CENTRO DE INVESTIGACIÓN Y DE ESTUDIOS AVANZADOS
DEL INSTITUTO POLITÉCNICO NACIONAL

Unidad Cinvestav Tamaulipas

**Método adaptativo-reactivo de
replicación para sistemas de
almacenamiento de alta
disponibilidad**

Tesis que presenta:

Jesús Ignacio Castillo Barrios

Para obtener el grado de:

**Maestro en Ciencias
en Ingeniería y Tecnologías
Computacionales**

Dr. José Luis González Compeán, Co-Director

Dr. Iván López Arévalo, Co-Director

Cd. Victoria, Tamaulipas, México.

Agosto, 2022

La tesis presentada por Jesús Ignacio Castillo Barrios fue aprobada por:

Dr. Edwyn Javier Aldana Bobadilla

Dr. Miguel Morales Sandoval

Dr. José Luis González Campeán, Co-Director

Dr. Iván López Arévalo, Co-Director

Cd. Victoria, Tamaulipas, México., 17 de Agosto de 2022

Agradecimientos

- A mi familia por su apoyo incondicional.
- A mis revisores, Dr. Miguel Morales Sandoval y Dr. Edwyn Javier Aldana Bobadilla, por su esfuerzo y disponibilidad para revisar y sugerir modificaciones para mejorar el documento de tesis.
- A mis directores, Dr. Iván López Arévalo y Y Dr. José Luis González Campeán, por acompañarme en esta etapa de mi vida, por guiarme y aconsejarme. Aprecio mucho todo el esfuerzo y tiempo invertido en aclarar cualquier duda.
- A Alfredo Barrón por su disponibilidad para explicarme y aclarar cualquier duda sobre almacenamiento.
- Al Consejo Nacional de Ciencia y Tecnología (CONACYT) por la beca otorgada para la realización de mis estudios de maestría.
- Al Centro de Investigación y de Estudios Avanzados del Instituto Politécnico Nacional por brindarme una formación profesional durante mis estudios de maestría.
- Al Proyecto 41756 “Plataforma tecnológica para la gestión, aseguramiento, intercambio y preservación de grandes volúmenes de datos en salud y construcción de un repositorio nacional de servicios de análisis de datos de salud” del FORDECYT-PRONACES-CONACYT.

Índice General

Índice General	I
Índice de Figuras	V
Índice de Tablas	VII
Índice de Algoritmos	IX
Resumen	XI
Abstract	XIII
Nomenclatura	XV
1. Introducción	1
1.1. Contexto	1
1.2. Sistemas de almacenamiento de alta demanda	2
1.3. Métodos de replicación	7
1.4. Planteamiento del problema	9
1.5. Hipótesis de investigación	11
1.6. Objetivos	12
1.7. Representación conceptual del prototipo	12
1.8. Metodología de trabajo	14
1.9. Organización de la tesis	16
2. Marco teórico	17
2.1. Virtualización	17
2.2. Conceptos básicos de tecnologías en la nube	18
2.3. Almacenamiento	20
2.3.1. Almacenamiento tradicional	20
2.3.2. Almacenamiento por red	21
2.3.3. Almacenamiento en la nube	21
2.3.4. Almacenamiento definido por software	22
2.4. Teoría de manejo de colas y tiempos de espera	22
2.5. Replicación	31
2.5.1. Factores de la replicación de datos	31
2.5.2. Métodos de replicación	32
2.6. Balanceo de carga	33

3. Estado del arte	35
3.1. Replicación de datos	35
3.1.1. Replicación estática	35
3.1.2. Replicación dinámica	37
3.2. Balanceo de carga	39
3.2.1. Balanceo de carga estático	39
3.2.2. Balanceo de carga dinámico	40
3.3. Consistencia de datos	42
3.3.1. Consistencia eventual	42
3.3.2. Consistencia de datos fuerte	43
3.4. Almacenamiento en la nube	44
3.5. Infraestructura definida por código	44
3.6. Almacenamiento definido por software	46
3.7. Resumen del estado del arte	48
4. Método adaptativo-reactivo de replicación para sistemas de almacenamiento de alta disponibilidad	51
4.1. Glosario	51
4.2. Descripción general	54
4.3. La replicación en un sistema de almacenamiento	56
4.4. Etapas	60
4.4.1. Etapa 1. Selección de nodos con encolamiento	60
4.4.2. Etapa 2. Selección de esferas más accedidas	63
4.4.3. Etapa 3. Crear nodos de almacenamiento	64
4.4.4. Etapa 4. Balanceo y distribución de carga	66
4.4.4.1. Balanceo de carga: Esferas	66
4.4.4.2. Balanceo de carga: Nodos	67
4.4.5. Etapa 5. Replicación	67
4.4.5.1. Replicación activa	68
4.4.5.2. Replicación pasiva	73
4.4.5.3. Replicación como Código	76
4.5. Implementación	80
4.5.1. Arquitectura del prototipo	80
4.5.1.1. Componentes	80
4.5.1.2. Módulos	81
4.5.2. Construcción del sistema de replicación	83
4.5.2.1. Modelo contextual de definición	83
4.5.2.2. Construcción	89
4.5.2.3. Despliegue	90
4.5.2.4. Ejecución	91

5. Evaluación experimental y resultados	93
5.1. Entorno de evaluación	93
5.1.1. Infraestructura empleada	94
5.1.2. Lanzador de cargas	94
5.1.3. Generador de cargas de trabajo sintéticas	95
5.1.4. Métricas	96
5.1.5. Configuraciones definidas	97
5.2. Evaluación	98
5.2.1. Evaluación de los tiempos de respuesta de las configuraciones definidas . . .	99
5.2.2. Análisis de patrones de producción y consumo	106
5.2.3. Análisis de la dinámica de accesos por nodo	113
6. Conclusiones y trabajo futuro	123
6.1. Resumen	123
6.2. Limitaciones	127
6.3. Aportaciones	128
6.4. Trabajo futuro	128
6.5. Datos de contacto	129

Índice de Figuras

1.1. Esquemas de almacenamiento	3
1.2. Representación conceptual de los componentes que forman la solución propuesta. . .	13
1.3. Metodología de trabajo.	15
2.1. Representación conceptual de una línea de espera en el contexto de los sistemas de almacenamiento.	23
2.2. Representación conceptual de una línea de espera y los tiempos asociados a este tipo de eventos.	25
2.3. Representación de una línea de espera en conjunto con el componente de balanceo de carga.	27
2.4. Representación conceptual de los tiempos asociados a una línea de espera con balanceo de carga.	28
2.5. Representación conceptual de un sistema de colas con gestor de réplicas.	29
2.6. Representación conceptual de líneas de espera con carga subyacente resultado de la replicación.	30
2.7. Diagrama conceptual de la metáfora <i>balls-into-bins</i>	30
2.8. Métodos de replicación	33
2.9. Balanceo de carga	34
3.1. Etapas de construcción de <i>AWS CloudFormation</i>	46
4.1. Etapas del método de replicación propuesto.	55
4.2. Representación conceptual de la replicación sistémica y replicación de datos.	65
4.3. Esquema de replicación activa: interacción entre cliente, gestor de réplicas y nodos de almacenamiento.	70
4.4. Esquema de replicación pasiva: interacción entre cliente y nodos de almacenamiento(<i>sn</i>).	74
4.5. Módulos del gestor de réplicas.	81
4.6. Diagrama de los pasos para la construcción del sistema de replicación con una configuración declarada.	83
4.7. Diagrama de clases de los nodos.	84
4.8. Atributos de una piscina de almacenamiento.	86
4.9. Diagrama de clases que representa la asignación de recursos, colocación y políticas de reinicio.	89
4.10. Clase que representa un <i>Compose File</i>	91
5.1. Tiempos de respuesta para las cinco configuraciones con tres clientes concurrentes. .	101
5.2. Tiempos de respuesta para las cinco configuraciones con seis clientes concurrentes. .	102
5.3. Tiempos de respuesta para las cinco configuraciones con doce clientes concurrentes. .	102

5.4. Porcentajes de ganancia con respecto a la configuración sin replicación incrementando el nivel de concurrencia.	104
5.5. Porcentaje de ganancia con respecto a la replicación activa.	105
5.6. Rendimiento de las cinco configuraciones procesando un lote de 1000 archivos con tamaño promedio de 100MB con nivel de concurrencia 3.	107
5.7. Rendimiento de las cinco configuraciones procesando un lote de 1000 archivos con tamaño promedio de 100MB con nivel de concurrencia 6.	108
5.8. Rendimiento de las cinco configuraciones procesando un lote de 1000 archivos con tamaño promedio de 100MB con nivel de concurrencia 12.	108
5.9. Nivel de concurrencia 3 y archivos de tamaño promedio de 1MB	109
5.10. Nivel de concurrencia 6 y archivos de tamaño promedio de 1MB	109
5.11. Nivel de concurrencia 12 y archivos de tamaño promedio de 1MB	110
5.12. Nivel de concurrencia 3 y archivos de tamaño promedio de 10MB	110
5.13. Nivel de concurrencia 6 y archivos de tamaño promedio de 10MB	111
5.14. Nivel de concurrencia 12 y archivos de tamaño promedio de 10MB	111
5.15. Nivel de concurrencia 3 y archivos de tamaño promedio de 100MB	112
5.16. Nivel de concurrencia 6 y archivos de tamaño promedio de 100MB	112
5.17. Nivel de concurrencia 12 y archivos de tamaño promedio de 100MB	113
5.18. Comparación entre activa y pasiva.	118
5.19. Comparación entre técnicas activa y activa elástica.	119
5.20. Comparación entre pasiva y pasiva elástica.	120
5.21. Comparación entre activa elástica y pasiva elástica.	121

Índice de Tablas

3.1.	Descripción de las abreviaciones utilizadas para los parámetros de comparación. . . .	49
3.2.	Comparativa de diferentes sistemas de replicación de la literatura.	50
4.1.	Variables involucradas en la definición de un sistema de replicación.	57
5.1.	Infraestructura empleada para desplegar el sistema de almacenamiento.	94
5.2.	Atributos de una traza sintética.	96
5.3.	Descripción de abreviaciones de las configuraciones definidas.	99
5.4.	Descripción de las combinaciones definidas	100

Índice de Algoritmos

Método adaptativo-reactivo de replicación para sistemas de almacenamiento de alta disponibilidad

por

Jesús Ignacio Castillo Barrios

Unidad Cinvestav Tamaulipas

Centro de Investigación y de Estudios Avanzados del Instituto Politécnico Nacional, 2022

Dr. Iván López Arévalo, Co-Director

Dr. José Luis González Campeán, Co-Director

Los servicios de almacenamiento en la nube ofrecen a los usuarios sistemas de alta disponibilidad de contenidos, basados en técnicas de replicación. Estos sistemas de replicación mejoran la disponibilidad de los recursos (recursos disponibles, RD) al reducir los tiempos de espera observados por los usuarios, mientras que la replicación de datos mejora los efectos producidos por los accesos simultáneos (conurrencia). Las técnicas tradicionales de replicación de datos producen continuamente varias copias utilizando un factor de replicación estático (FR), mientras que las técnicas dinámicas pueden cambiar el valor de FR en función de un criterio determinado, siempre que $FR < RD$. La eficiencia de las técnicas de replicación viene determinada por el número de accesos en comparación con FR , que viene definido por RD . Cuando los accesos son mayores que FR , la probabilidad de disponibilidad es baja produciendo un efecto de encolamiento, en caso contrario, esta probabilidad mejora, pero aumentará el consumo de almacenamiento, lo que incrementa la subutilización de los sistemas de almacenamiento al albergar réplicas no utilizadas. La clave, por tanto, para medir el éxito de un sistema de replicación es conseguir una alta probabilidad de disponibilidad y reducir los costos asociados al factor de replicación (FR) cambiando el RD y no sólo el FR como realizan las técnicas de replicación tradicionales. Sin embargo, los métodos de replicación estática generalmente no consiguen este equilibrio, mientras que la replicación dinámica sólo lo consigue a largo plazo. En esta tesis se propone un método de replicación adaptativa-reactiva para sistemas de almacenamiento

de alta disponibilidad. El método incluye un método de construcción para cambiar dinámicamente el factor RD mediante una estrategia de replicación sistémica (elasticidad del parámetro RD). Esta estrategia, en combinación con una replicación de datos adaptativa, basada en las interacciones de carga y descarga de los usuarios (definición dinámica de FR), reacciona a las cargas de trabajo para mejorar la probabilidad de disponibilidad de los contenidos almacenados. Mediante un modelo contextual de definición los diseñadores pueden crear estrategias de replicación elástica definiendo la infraestructura como código. Se realizó una amplia evaluación experimental, la cual reveló que el método propuesto mejora la probabilidad de disponibilidad y el rendimiento en comparación con las técnicas estáticas. También se demostró que las técnicas de replicación elástica producen mejoras de rendimiento, incluso en comparación con los métodos tradicionales de replicación dinámica.

Adaptive-reactive replication method for high availability storage systems

by

Jesús Ignacio Castillo Barrios

Unidad Cinvestav Tamaulipas

Research Center for Advanced Study from the National Polytechnic Institute, 2022

Dr. Iván López Arévalo, Co-advisor

Dr. José Luis González Compeán, Co-advisor

Cloud storage services provide end-users with high content availability systems based on replication techniques. These replication systems improve the availability of resources (available resources, RD) by reducing the waiting times observed by users, whereas data replication improves the effects produced by simultaneous accesses (concurrency). Traditional data replication techniques continuously produce several copies by using a static replication factor (FR), whereas dynamic techniques can change the value of FR depending on a given criterion, whenever $FR < RD$. The efficiency of replication techniques is determined by the number of accesses compared to FR , which is defined by RD . When the accesses are greater than FR , the probability of availability is low producing a queuing effect, otherwise, this probability improves but it will increase the storage consumption, which would increase the under-utilization of the storage systems when hosting unused replicas. The key, therefore, to measure the success of a replication system is to achieve a high probability of availability and to reduce the costs associated with the replication factor (FR) by changing the RD , not only the FR as performed by traditional replication techniques. However, static replication methods generally fail to achieve this balance, whereas dynamic replication only achieves this balance in the long-term. In this thesis, an adaptive-reactive replication method for high availability storage systems is proposed. The method includes a construction method to dynamically change the RD factor by using a strategy of systemic replication (elasticity of the RD parameter). This strategy, in combination with an adaptive data replication based on user upload and download interactions

(dynamic definition of FR), reacts to the workloads to improve the availability probability of stored contents. By means of a context-definition model, designers can create elastic replication strategies defining the infrastructure as code. An extensive experimental evaluation revealed that the proposed method improves the availability probability and performance compared to static techniques. It is also shown that the elastic replication techniques yield performance improvements even in comparison with traditional dynamic replication methods.

Nomenclatura

FR	Factor de replicación
RD	Recursos disponibles
Tr	Tiempo de retención
Rt	Tiempo de respuesta
At	Tiempo de llegada
Et	Tiempo de salida
St	Tiempo de servicio
Wt	Tiempo de espera
Iat	Tiempo entre operaciones
API	Interfaz de programación de aplicaciones.
FR	Requerimiento funcional.
IaC	Infraestructura definida por código.
NFR	Requerimiento no funcional.
NIST	Instituto Nacional de Estándares y Tecnología.
SDS	Almacenamiento definido por software.
SO	Sistema operativo
VC	Contenedor virtual.
VM	Máquina virtual.
API	Interfaz de Programación de Aplicaciones
UX	Experiencia de usuario

1

Introducción

En este capítulo se presenta el contexto del trabajo, sus antecedentes y la descripción del problema que se aborda. A partir de ellos se presenta la hipótesis de investigación, los objetivos planteados para comprobarla y una descripción de la metodología de solución que se implementó para lograrlos.

1.1 Contexto

Hoy día, como parte esencial de las actividades cotidianas de personas e instituciones, el almacenamiento de datos forma parte importante en cualquier aplicación o sistema informático debido a que existe la necesidad de preservar datos para su uso posterior, ya sea consultándolos o analizándolos. En la literatura el almacenamiento de datos no es nuevo, existen diversos trabajos que abordan este tema [41, 46]. Las distintas formas de almacenar datos pueden agruparse como: 1) *almacenamiento tradicional*, 2. *almacenamiento por red* [32], 3. *almacenamiento en la nube* [51] y 4. *almacenamiento definido por software (SDS)* [43]. Los usuarios que interactúan con los sistemas de almacenamiento (SA) son clasificados en los roles de *productor* y/o *consumidor*, dependiendo de las

operaciones realizadas por el usuario, también es posible que un usuario pueda tomar ambos roles.

Normalmente los usuarios interactúan con los sistemas de almacenamiento (SA) mediante aplicaciones de software que les permiten realizar las operaciones básicas de carga y descarga de contenidos (archivos, bloques, objetos). Las interacciones realizadas por los usuarios de estas aplicaciones forman patrones de consumo y producción. En los patrones formados por las interacciones de los usuarios es posible que algunos de los contenidos producidos tengan una cantidad de accesos mayor a la del resto de contenidos, siendo éstos los principales causantes de los *encolamientos* como resultado de un nivel de concurrencia alto. La concurrencia es resultado de las múltiples peticiones realizadas por los usuarios. Múltiples operaciones llegando al sistema de almacenamiento al mismo tiempo producen un retraso en la entrega del contenido solicitado. Ejemplos de éstos podrían ser canciones recientes cargadas a la plataforma de Spotify o videos en Youtube, los cuales son accedidos por miles o millones de usuarios de manera simultánea. Algunas cifras que se encuentran en la literatura sobre la cantidad de usuarios consumiendo un video en Youtube rondan los 100 millones cada día, en donde se observa que los videos nuevos causan mayor impacto y son altamente consumidos [21]. Las diferentes plataformas que se utilizan de manera cotidiana, como redes sociales o plataformas de streaming invierten en el desarrollo de software e infraestructura para sistemas de almacenamiento que les permitan disminuir el consumo de almacenamiento, incrementar el rendimiento de la infraestructura y aumentar la *disponibilidad* de los datos almacenados aplicando métodos de replicación.

1.2 Sistemas de almacenamiento de alta demanda

En entornos de alto consumo de datos almacenados es necesario contar con esquemas de almacenamiento eficaces. Estos esquemas dependen del tipo de dato que se consume. Entre los diversos esquemas de almacenamiento se puede destacar los que se ilustran en la Figura 1.1.

En la Figura se observan los tres esquemas de almacenamiento que se implementan, de manera

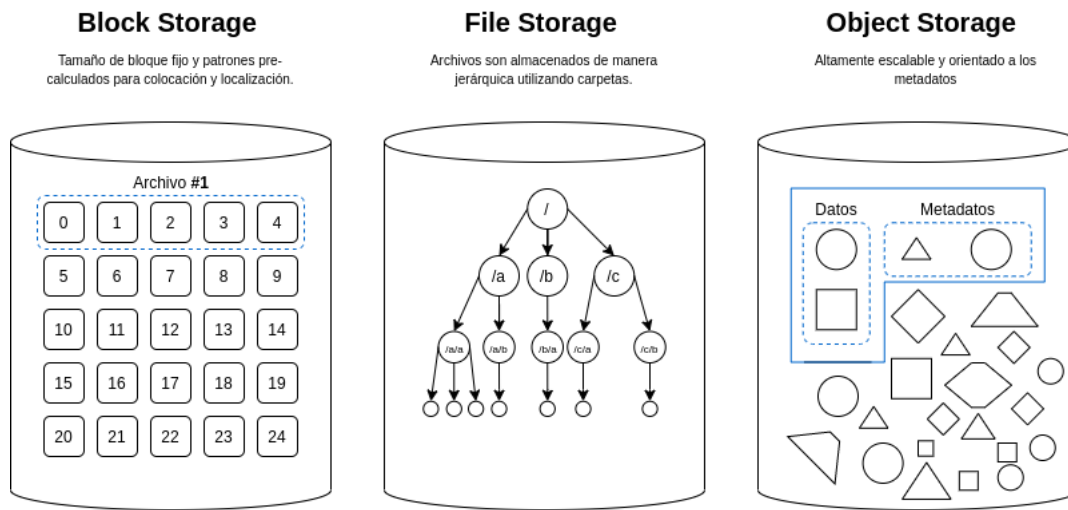


Figura 1.1: Esquemas de almacenamiento

general, dependiendo de las necesidades de los entornos: almacenamiento en bloque, almacenamiento de objetos y almacenamiento de archivos. Estos esquemas determinan cómo se procesan, almacenan, organizan y localizan los datos. Cada tipo de almacenamiento tiene capacidades y limitaciones únicas, lo que significa que se deberá seleccionar el tipo más adecuado a las necesidades de la organización o sistema que requieran una solución para almacenar datos de manera adecuada y eficiente.

Los sistemas de almacenamiento (SA) deben realizar dos operaciones básicas. La escritura de datos es la operación principal, la otra es la lectura de los datos desde un dispositivo de almacenamiento, la cual puede realizarse en tres formas diferentes: *bloque*, *archivo* u *objeto*. Ambas son conocidas como operaciones de colocación y localización, respectivamente.

El *almacenamiento en bloques*, también conocido como *almacenamiento a nivel de bloque*, se encarga de almacenar una secuencia de bytes de datos que contienen un número de registros enteros que tienen una longitud máxima (un tamaño de bloque). Una de las principales ventajas del almacenamiento en bloques es que ofrece el mayor rendimiento en las operaciones de escritura y lectura. Se selecciona el almacenamiento en bloque para situaciones en las que requieren un transporte de datos rápido, eficiente y fiable. Las desventajas del almacenamiento en bloques radica en limitaciones de escalabilidad, y la falta de metadatos que limitan la flexibilidad en la capacidad

de búsqueda y el análisis del contenido que podría describirse en los metadatos.

El *almacenamiento de archivos* se organiza de forma similar a como se organizan los archivos físicos en un archivador. Los archivos se colocan sistemáticamente en carpetas y se organizan mediante convenciones de nomenclatura basadas en características como extensiones, categorías o aplicaciones.

El *almacenamiento de objetos*, también conocido como *almacenamiento basado en objetos*, es una estrategia que gestiona y manipula el almacenamiento de datos como unidades distintas, llamadas *objetos*. Estos objetos se guardan en un único espacio, no están agrupados dentro de carpetas. En su lugar, el almacenamiento de objetos combina las piezas de datos que componen un archivo, añade todos sus metadatos relevantes a ese archivo y le adjunta un identificador único.

En este trabajo de tesis se abstraen los esquemas de almacenamiento, bajo la metáfora de *balls into bins*, y se emplea la palabra *esfera*, la cual hace referencia a un *bloque*, un *archivo* o un *objeto* dependiendo del esquema utilizado por el sistema de almacenamiento y las *cajas* se asocian a nodos de almacenamiento que se encargan de procesar las operaciones de carga y descarga. Los métodos de replicación pueden ser aplicados a cualquier esquema de almacenamiento debido a que el objetivo es crear réplicas de bloques, archivos u objetos para incrementar la disponibilidad de los elementos y obtener un beneficio en rendimiento y en tiempos de respuesta al momento de las localizaciones.

En los últimos años el aumento en el volumen de los datos ha sido provocado por diferentes factores, desde las actividades que realizan los usuarios hasta las actividades de aprovechamiento de los datos. Por ejemplo, los datos generados por dispositivos IoT (Internet of Things) que realizan la transmisión de datos recolectados sin intervención humana y de forma inalámbrica [53]. Otros factores, como los nuevos escenarios de la vida cotidiana de humanos y organizaciones, contribuyen a que el crecimiento de los datos sea exponencial.

La cuantificación de los datos creados, consumidos y replicados es denominada *esfera global de datos* (*global datasphere*) [53]. En 2006 se estimó un volumen de 160 *Exabytes* de la esfera global de datos. Los reportes estadísticos más recientes sobre la esfera global de datos realizados

por IDC (*International Data Corporation*) estimaban un aumento en volumen de datos de 33 Zettabytes(ZB) para 2018 y 175 ZB para 2025. En el pasado los usuarios consumidores eran responsables de la gestión de sus datos, pero su dependencia y su confianza en los servicios de cómputo en la nube actuales, especialmente desde las perspectivas de conectividad, rendimiento y disponibilidad, continúan aumentando, mientras que la necesidad de almacenar y gestionar datos localmente disminuye.

Patrones de utilización de almacenamiento en la nube

Uno de los principales usos del almacenamiento en la nube es el resguardo de datos en espacios de almacenamiento virtualizados, por lo general aportados por terceros (proveedores de Cloud Computing). Conceptualmente los usuarios que hacen uso del almacenamiento en la nube se clasifican como:

- **Productor:** El usuario que genera contenidos/archivos y los almacena en un espacio en la nube.
- **Consumidor:** El usuario que consume los contenidos/archivos de su interés que han sido almacenados en la nube por los usuarios productores.

Una opción eficiente y de bajo costo para la interacción entre usuarios consumidores y productores son las aplicaciones de almacenamiento en la nube. Las interacciones realizadas en este tipo de aplicaciones generan *patrones de acceso* a los recursos de la nube; estos patrones varían dependiendo de la aplicación utilizada, algunos ejemplos de estas aplicaciones son:

- **Aplicaciones de entrega de contenidos (EC):** Estas aplicaciones han sido diseñadas para acercar los contenidos a los usuarios. Tienen una estructura basada en el mecanismo de *publicación* y *suscripción*. La unidad de intercambio es un catálogo de contenidos, el cual es publicado (ofertado) por alguna organización para que los usuarios interesados se suscriban (accedan) a él. Los usuarios de estas aplicaciones adquieren el rol de productor o consumidor.

Cada productor está a cargo de la publicación de contenidos y cada consumidor del acceso a ellos. Algunos ejemplos de este tipo de aplicaciones son Netflix y Spotify, entre otros.

- **Aplicaciones de compartición de archivos (CA):** Estas aplicaciones no tienen una estructura definida, su comportamiento es similar al de una red social. La unidad de intercambio es una URL de un archivo, la cual es compartida entre los usuarios o un grupo de ellos. Los usuarios de estas aplicaciones adquieren el doble rol de productor/consumidor. Ejemplo de este tipo de aplicación es Dropbox.

En estos patrones se puede apreciar que la cantidad de operaciones de carga (ubicación) tiene un comportamiento opuesto con respecto a las operaciones de descarga. Las operaciones de carga se presentan en menor proporción, que van del 1 % al 20 % de las veces y las operaciones de descarga (localización) se presentan en mayor medida, en una proporción del 80 % al 99 % de las veces.

En la industria, para abordar esta situación, se emplea el almacenamiento *definido por software* (SDS), el cual ha atraído mucho la atención en años recientes. Es una arquitectura de almacenamiento que separa el software de almacenamiento de su hardware. A diferencia de los sistemas tradicionales de almacenamiento conectado a la red (NAS) o a la red de área de almacenamiento (SAN), el SDS suele estar diseñado para funcionar en cualquier sistema estándar pues elimina la dependencia del software del hardware propietario. Estas características son de especial interés en esta tesis.

En este contexto, es necesario aclarar que el SDS no hace total referencia a la virtualización, ni tampoco SDS es almacenamiento en la nube. Tanto el SDS como la virtualización del almacenamiento implican abstraer algo del hardware de almacenamiento. La virtualización del almacenamiento permite agrupar la capacidad de muchos dispositivos de almacenamiento para que parezca que todo el almacenamiento se encuentra en un único dispositivo. El SDS, en cambio, abstrae los servicios de almacenamiento, o el software de almacenamiento, y los separa del propio dispositivo. Es una capa de software que permite integrar diversos dispositivos y/o sistemas de almacenamiento y poderlos ver como una piscina de almacenamiento que tiene la propiedad de la *elasticidad*¹ con el objetivo de

¹La *elasticidad* es una característica que puede aplicarse a la mayoría de los servicios en la nube. Establece que la

proporcionar un almacenamiento unificado.

En escenarios de alta demanda de almacenamiento y consumo de archivos, las peticiones se atienden siguiendo el enfoque de teoría de colas. La teoría de colas (teoría de manejo de colas) se refiere al estudio matemático de la formación, funcionamiento y congestión de las líneas de espera (colas). En esencia, un sistema de colas o sistema de línea de espera está conformado por dos eventos:

- Alguien que solicita un servicio, normalmente denominado cliente.
- Alguien que completa o entrega los servicios, normalmente denominado servidor.

En los SA los elementos que se preservan en la línea de espera se conocen como operaciones de carga y descarga. Las cuales llegan al sistema de almacenamiento y son encoladas para ser atendidas siguiendo el esquema FIFO (First-In First Out)². Estas operaciones de carga y descarga, por regla empírica siguen el Principio de Pareto (las cargas representan una porción pequeña, entre 1 % a 20 %, mientras que las descargas representan la porción más amplia, que va desde el 80 % al 99 %). Con esto se puede entender que los SA experimentan una cantidad mayor de operaciones de descargas, ocasionando problemas de encolamiento, los cuales degradan la experiencia del usuario (*UX*), aumentando los tiempos de respuesta (*Rt*)³.

1.3 Métodos de replicación

La gestión de grandes volúmenes de datos se ha convertido en un reto para la comunidad científica, atrayendo su atención. Los volúmenes de datos deben de ser almacenados, y comúnmente se utilizan sistemas de almacenamiento que permiten gestionar el ciclo de vida de los contenidos almacenados, y preservándolos en dispositivos de almacenamiento (p. ej. *HDD*, *SSD*). Los SA

capacidad de almacenamiento y el rendimiento de cualquier servicio en la nube puede ampliarse o contraerse según las necesidades del usuario de la infraestructura y que ésta puede modificarse de forma dinámica como consecuencia de algún evento impulsado por el software.

²El enfoque de las colas es el conjunto de reglas bajo las cuales se procesan los elementos entrantes.

³El tiempo que tarda un sistema en reaccionar ante un determinado estímulo o acontecimiento.

son distribuidos por naturaleza, esto es, sus componentes se encuentran desplegados en distintas computadoras que podrían estar en diversas infraestructuras en distintas localidades. Esto se realiza por diversas razones, como tolerancia a fallas, incrementar el rendimiento, y/o crear redundancia⁴ para incrementar la disponibilidad. Cuando a la *redundancia* se le añade una capa de gestión del ciclo de vida de los elementos redundantes, entonces a esto se lo conoce como *replicación*⁵. La replicación es ampliamente estudiada en los entornos distribuidos (no sólo para almacenamiento, sino también para servicios) donde se almacenan múltiples copias (réplicas) de datos y se colocan en distintos nodos de almacenamiento (Sn). En estos entornos los costos asociados a la creación, transporte y consistencia de las réplicas son cuestiones muy importantes y desafiantes para los administradores de los sistemas de almacenamiento.

Un *método de replicación* define una serie de pasos que deben de seguirse de manera ordenada y sistematizada para llevar acabo el proceso de replicación. Los pasos más comunes son: 1) seleccionar los archivos, bloques u objetos a replicar, 2) asignar un factor de replicación (FR) a cada archivo, bloque u objeto, y 3) balancear y distribuir las réplicas de manera ortogonal.

La replicación reduce los tiempos de espera y aumenta la velocidad del acceso a los recursos, esto al mejorar la disponibilidad de los datos a través de la redundancia. Es ampliamente utilizada por sistemas de almacenamiento en la nube, los cuales reciben una gran cantidad de operaciones de forma concurrente (operaciones de carga y descarga que llegan en tiempos iguales, generalmente en forma de ráfagas). Amazon S3, GFS (*Google File System*) [31] y HDFS (*Hadoop Distributed File System*) [54] son ambientes de almacenamiento que utilizan métodos estáticos de replicación. Sin embargo, estos esquemas estáticos empiezan a presentar limitaciones ya que el almacenamiento en la nube tiene crecimiento exponencial, desde 2010 hasta la fecha [30].

De manera tradicional los sistemas de almacenamiento en la nube realizan al menos tres réplicas de datos, las cuales son distribuidas en piscinas estáticas de nodos de almacenamiento. Esto se

⁴La redundancia permite duplicar cualquier elemento o componente de un sistema.

⁵La replicación es lo que convierte a los elementos redundantes en elementos útiles y de alto consumo el sistema de almacenamiento.

conoce como *método estático de replicación* y generalmente utilizan cinco nodos de almacenamiento [31, 54]. Este método es adecuado para escenarios donde la carga de trabajo es conocida. Una de sus principales desventajas es el consumo de recursos de almacenamiento debido a que no es necesaria la creación de réplicas para todos los datos. La demanda de un archivo, no es constante, mas bien fluctúa sobre el tiempo y es por eso que mantener un *FR* adaptable es crucial para ofrecer un mejor servicio [22]. En la literatura se le conoce como *datos calientes* aquellos datos que son altamente demandados por los usuarios y *datos fríos* a los datos que no son accedidos con tanta frecuencia o que simplemente no son accedidos [4, 35].

1.4 Planteamiento del problema

Como se ha mencionado previamente, la replicación de datos mejora la disponibilidad y los tiempos de respuesta de los sistemas de almacenamiento en la nube. Sin embargo, administrar réplicas es costoso en términos de:

1. *Producción*, porque se requiere la realización de una operación de lectura y *FR* operaciones de escritura,
2. *Almacenamiento*, porque se requiere manejar las *FR* réplicas más el contenido original, su costo en espacio es el producto del tamaño del componente original por *FR*,
3. *Distribución*, porque se requiere enviar las *FR* réplicas a través, generalmente, de una red; su costo de transferencia es similar al costo de almacenamiento.

De esta forma, un método de replicación se vuelve una inversión única (durante la carga o colocación de las réplicas) de la cual se espera una ganancia en términos de disponibilidad (durante las descargas o localización). Se espera que la ganancia (disponibilidad) sea mayor que la inversión (costos de replicación), lo cual en la práctica generalmente se cumple ya que, por regla (empírica), los patrones de carga/descarga siguen el Principio de Pareto.

Los métodos de replicación estática aplican un FR homogéneo para todos los archivos, esto hace que la inversión sea constante pero las ganancias sean esporádicas ya que, por regla empírica, se espera que los usuarios también produzcan patrones de carga y descarga siguiendo el Principio de Pareto. Esto significa que las ganancias se presentan sólo entre un 1 y un 20 % de las descargas. Para resolver este desequilibrio se han propuesto métodos dinámicos de replicación, las cuales modifican FR dinámicamente midiendo la popularidad de cada archivo. En este caso las ganancias se van obteniendo gradualmente sobre el tiempo hasta que la popularidad se estabiliza en un *cenit* (generalmente siguiendo una distribución exponencial). En tal caso, el balance se mejora pero en mediano y/o largo plazo (hasta alcanzar el *cenit* de popularidad). Se han empleado algunos algoritmos multi-objetivo para determinar un FR óptimo, sin embargo estos algoritmos son costosos en términos computacionales y requieren un conjunto de datos extenso. Además, el enfoque de usar algoritmos multi-objetivo resulta complejo de implementar en escenarios de encolamientos súbitos producidos por ráfagas concurrentes o cuando el patrón de interacciones cambia. Estos últimos escenarios resultan en cargas de trabajo dinámicas como las que se presentan en las aplicaciones en la nube y resulta clave tomar decisiones en tiempo real.

En resumen, si se aplica un método estático se presenta una subutilización de recursos y un desequilibrio entre inversión (replicación) y ganancia (disponibilidad efectiva), mientras que las técnicas de popularidad o fuera de línea (optimización) aunque reducen el desequilibrio, las primeras técnicas consiguen en equilibrio en el mediano plazo, mientras que las últimas técnicas lo consiguen a corto plazo, pero en condiciones de cargas de trabajo estáticas.

Tomando en cuenta que los factores FR y RD determinan la probabilidad de disponibilidad de un sistema de almacenamiento, se plantean las siguientes preguntas de investigación:

1. ¿Cómo se puede calcular un FR que permita a un sistema de replicación mejorar la disponibilidad de contenidos?

2. ¿Cómo se puede ajustar el factor de disponibilidad de recursos (RD) a un FR variable para mejorar el consumo de almacenamiento?

1.5 Hipótesis de investigación

A partir de la problemática descrita, en esta tesis se asumen las siguientes premisas:

1. Aumentar el FR aumenta la probabilidad de disponibilidad de los datos.
2. El costo de la gestión de los datos aumenta con el aumento del parámetro FR .
3. Las operaciones de colocación disparan ráfagas de operaciones de localización, las cuales por regla empírica siguen un Principio de Pareto.
4. Distribuir réplicas en ubicaciones ortogonales aumenta la probabilidad de disponibilidad de datos.
5. Al aumentar la disponibilidad de los datos se disminuyen los tiempos de servicio (generalmente siguiendo una distribución exponencial, dependiendo de la concurrencia).
6. Al aumentar el factor de disponibilidad de recursos (RD) se aumenta el ancho del banda ⁶ del sistema, lo cual permite incrementar el FR del proceso de replicación.

Tomando en cuanto las premisas, la hipótesis de investigación es la siguiente:

Un método de replicación que defina FR y RD de manera dinámica mediante una estrategia que combine la replicación sistémica con la replicación de datos, basado en las interacciones de carga-descarga de usuarios, mejorará la disponibilidad de contenidos y reducirá los tiempos de respuesta.

⁶Es la capacidad máxima en el volumen de datos que puede fluir por el sistema.

1.6 Objetivos

Para determinar la validez de la hipótesis se plantearon los siguientes objetivos:

Objetivo General

- Crear un método adaptativo-reactivo de replicación para sistemas de almacenamiento de alta disponibilidad en la nube.

Objetivos Particulares

1. Crear un método de construcción y despliegue de esquemas de almacenamiento basado en la definición contextual de sus elementos y con elasticidad del parámetro RD para mejorar el ancho de banda del sistema de almacenamiento.
2. Definir una estrategia que combine la replicación sistémica (elasticidad del parámetro RD) con una replicación de datos adaptable, basada en las interacciones de carga y descarga de los usuarios (definición dinámica de FR) para mejorar la disponibilidad de los recursos almacenados.

1.7 Representación conceptual del prototipo

Para cumplir con los objetivos antes planteados, en la Figura 1.2 se muestra la representación conceptual del método propuesto y los componentes que lo integran:

- **Cliente (C)**: Componente que utilizan los usuarios para interactuar directamente con el sistema de almacenamiento y realizar operaciones de carga y descarga.
- **Gestor de réplicas (RM)**: Gestiona las réplicas de datos y sistémicas. Se encarga de su creación y distribución. Este componente también se encarga de gestionar las líneas de espera de los nodos de almacenamiento.

- **Piscina de almacenamiento (Sp):** Conjunto de nodos de almacenamiento, en los cuales se distribuyen las réplicas.
- **Nodos de almacenamiento (Sn):** Procesan las operaciones de escritura y lectura de contenidos.



Figura 1.2: Representación conceptual de los componentes que forman la solución propuesta.

La descripción detallada del método se presenta en el Capítulo 4 dando los pormenores de las etapas que constituyen al método y los conceptos que se emplean. Para ello es necesario conocer el

Marco Teórico (Capítulo 2) y el Trabajo Relacionado (Capítulo 3). Este método es el resultado de la metodología de trabajo que se definió para el desarrollo del trabajo de tesis.

1.8 Metodología de trabajo

A continuación se presente la metodología de trabajo que se siguió para alcanzar los objetivos planteados. Para ello se presenta un acercamiento conceptual a la solución de la problemática planteada en la Sección 1.4. Estos conceptos se utilizarán como guía para obtener los productos de la tesis. La metodología de trabajo consta de las siguientes etapas:

1. Revisión del estado del arte

Se recopilaron los trabajos relacionados con la problemática descrita y la solución propuesta. Además, se conocieron los conceptos de mayor relevancia para ayudar a la comprensión del trabajo. A partir de lo anterior se redactó el protocolo de tesis.

2. Diseño y desarrollo del método

Se diseñó y desarrolló un prototipo con los módulos del método propuesto: *cliente*, *gestor de réplicas* y *nodo de almacenamiento*.

3. Evaluación experimental

El prototipo desarrollado se evaluó variando sus parámetros de entrada. El desempeño se comprobó mediante las métricas 1) rendimiento de las soluciones generadas, 2) tiempo de servicio y 3) tiempo de respuesta.

4. Análisis de resultados

A partir de los resultados de la experimentación se realizó un análisis de los resultados. Además se realizó una comparación directa con soluciones de almacenamiento de datos en la nube de la literatura.

5. Conclusiones

Se realizó una interpretación de los resultados obtenidos, se contrastó el grado de cumplimiento de los objetivos alcanzados, se concluyó sobre los experimentos y se redactó este documento de tesis.

En la Figura 1.3 se muestra la metodología de trabajo descrita en los puntos anteriores.

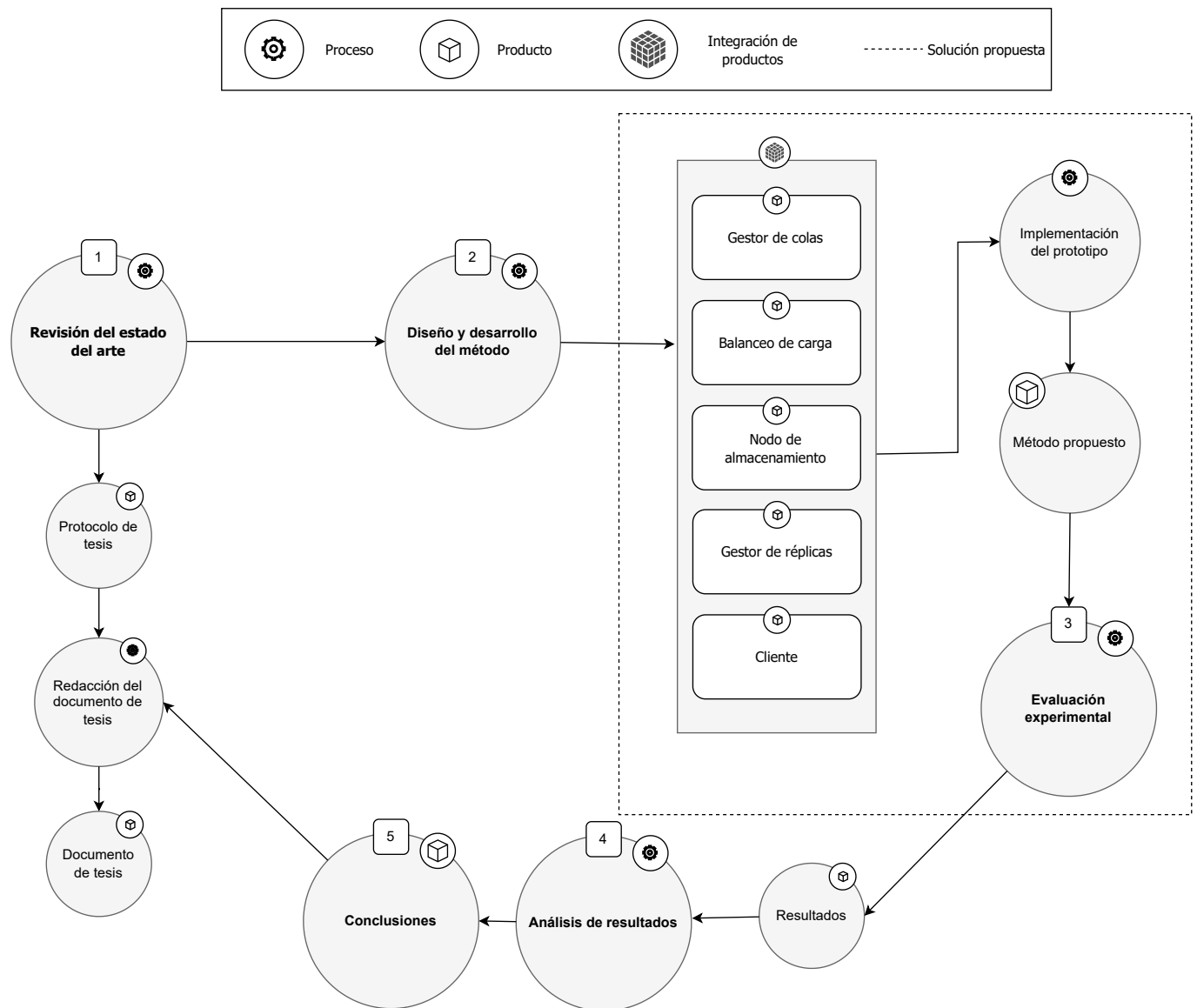


Figura 1.3: Metodología de trabajo.

1.9 Organización de la tesis

Este documento de tesis está conformado por 6 capítulos, los cuales se encuentran organizados de la siguiente manera: El Capítulo 1 presenta el contexto, problemática, hipótesis y objetivos de este trabajo de tesis. En el Capítulo 2 se realiza una descripción general del trasfondo teórico del trabajo de tesis. En el Capítulo 3 se presenta un resumen del estado del arte respecto a los métodos de replicación. En el Capítulo 4 se describe el diseño y el desarrollo de los componentes que integran el método propuesto. En el Capítulo 5 se describe la evaluación experimental realizada y los resultados obtenidos. Por último, en el Capítulo 6 se presentan las conclusiones, contribuciones, limitaciones y trabajo futuro identificado.

2

Marco teórico

En este capítulo se presentan los conceptos más importantes que están relacionados con el trabajo de tesis desarrollado. Éstos se describen en el orden en que se usan en el método propuesto.

2.1 Virtualización

La virtualización se define como una tecnología que introduce una capa de abstracción de software entre el hardware y el sistema operativo. La virtualización se clasifica como:

- **Virtualización completa:** En este enfoque el gestor de máquinas virtuales (VMM) se ejecuta sobre un sistema operativo (SO) anfitrión. El resultado es que en las máquinas virtuales, las aplicaciones se ejecutan sobre un SO, el cual fue instalado en un hardware virtual proporcionado por el VMM.
- **Virtualización basada en contenedores o sistema operativo:** También se conoce como *Single Kernel Image* (SKI). Este concepto implementa la virtualización ejecutando más

instancias del mismo SO de forma paralela utilizando el mismo hardware.

- **Paravirtualización:** A diferencia de la virtualización completa, en la *paravirtualización* el SO huésped debe ser modificado para poder ser operado en el entorno virtual. La paravirtualización es un subconjunto de la virtualización de servidores, que proporciona una interfaz de software entre el hardware anfitrión y el SO invitado modificado.
- **Virtualización de aplicación:** El usuario es capaz de ejecutar una aplicación localmente utilizando los recursos locales sin necesidad de lidiar con la complejidad de instalar completamente la aplicación. Las aplicaciones están diseñadas para ejecutarse en un pequeño entorno virtual que contiene únicamente los recursos necesarios para la ejecución de dicha aplicación.
- **Virtualización de recursos:** Virtualización de recursos específicos del sistema, como almacenamiento, CPU (procesamiento) y/o red.

2.2 Conceptos básicos de tecnologías en la nube

Con el paso de los años el cómputo en la nube (Cloud Computing) se ha convertido en un tema de gran relevancia por su modelo de oferta y consumo de recursos. Este concepto ha sido definido por el Instituto Nacional de Estándares y Tecnología (NIST) como un modelo de desarrollo que permite el acceso, bajo demanda, a un conjunto compartido de recursos computacionales configurables: redes, servidores, almacenamiento, aplicaciones y servicios, que pueden ser rápidamente provisionados y liberados con un esfuerzo mínimo de administración o de interacción con el proveedor de servicios. Las cinco características esenciales del cómputo en la nube, de acuerdo con el NIST, son:

- **Elasticidad:** Las capacidades pueden ser provisionadas y liberadas elásticamente, en algunos casos automáticamente. Dichas capacidades pueden adaptarse con base a la demanda, dando la impresión al consumidor de que los recursos son ilimitados.

- **Amplio acceso a la red:** Las capacidades están disponibles en la red y son accedidas a través de mecanismos estándares que promueven el uso de plataformas heterogéneas para clientes ligeros o pesados (teléfonos móviles, tablets, portátiles, estaciones de trabajo, etc).
- **Autoservicio bajo demanda:** Un consumidor puede abastecerse de capacidades de cómputo, como tiempo de servidor y almacenamiento en red, de forma automática según sea necesario, sin necesidad de la interacción humana con cada proveedor de servicios.
- **Servicio medido:** Los sistemas de nube controlan automáticamente y optimizan el uso de recursos de cómputo mediante el aprovechamiento de la capacidad de medición de algún nivel de abstracción, apropiado para el tipo de servicio (almacenamiento, procesamiento, ancho de banda, etc).
- **Piscina de recursos:** Los recursos computacionales del proveedor se reúnen para servir a múltiples consumidores mediante un modelo *multi-tenant*, con diferentes recursos físicos y virtuales dinámicamente asignados y reasignados con la demanda del consumidor.

Los tres modelos de servicio que ofrece la nube de acuerdo con el NIST son:

- **Software como servicio (SaaS -Software as a Service-):** Permite al consumidor utilizar las aplicaciones de algún proveedor. Éstas son accesibles desde diferentes dispositivos del cliente a través de una interfaz, como un navegador web.
- **Plataforma como servicio (PaaS -Platform as a service-):** Permite al consumidor desarrollar aplicaciones utilizando lenguajes de programación, bibliotecas, servicios y herramientas soportadas por el proveedor.
- **Infraestructura como servicio (IaaS -Infrastructure as a Service-):** Permite al consumidor disponer de procesamiento, almacenamiento, redes y otros recursos de cómputo, el consumidor es capaz de desplegar y ejecutar software arbitrario, que puede incluir sistemas operativos y aplicaciones.

Estos modelos de servicio se construyen sobre los siguientes modelos de despliegue:

- **Nube pública:** Es la infraestructura preparada para el uso abierto al público en general. Puede ser de propiedad, administrada y operada por una empresa u organización gubernamental.
- **Nube privada:** Es la infraestructura preparada para uso exclusivo de una sola organización que comprende múltiples consumidores (unidades de negocio). Puede ser de propiedad, administrada y operada por la organización o un tercero.
- **Nube híbrida:** Es la infraestructura compuesta de dos o más distintas infraestructuras (privadas, comunitarias o públicas) que permanecen como entidades únicas pero están unidas mediante tecnología estandarizada que habilita los datos y portabilidad de las aplicaciones.

Los modelos de despliegue de nube se implantan en los centros de datos, en donde los recursos de cómputo y procesamiento de su infraestructura son virtualizados como un sistema coherente llamado *piscina de recursos*, el cual es ofrecido a los usuarios o aplicaciones como un servicio.

2.3 Almacenamiento

En la literatura se hace referencia a diversos tipos de almacenamiento que se emplean hoy en día en diversos ámbitos. Estos tipos de almacenamiento dan un panorama general del tipo de almacenamiento en el que se contextualiza este trabajo de tesis.

2.3.1 Almacenamiento tradicional

Es la opción de almacenamiento en la que se utilizan unidades físicas locales para almacenar los datos en la ubicación primaria del usuario. El usuario suele utilizar el hardware basado en discos para almacenar los datos. Las tecnologías actuales de discos son electromecánicos (HDD) o de estado sólido (SSD).

2.3.2 Almacenamiento por red

En esta opción de almacenamiento se ponen a disposición de los usuarios diversos dispositivos de almacenamiento a través de una red. Son menos flexibles debido al acoplamiento fuerte que existe entre el hardware y el software que permite interactuar con el almacenamiento. A pesar de esto, ofrece un rendimiento dependiendo de la infraestructura y las características de la red. Ejemplos de dispositivos de almacenamiento por red son:

- *Network Attached Storage (NAS)*: El almacenamiento de área de red se refiere a un dispositivo de almacenamiento dedicado que se almacena en una red local. Los sistemas NAS son vendidos por los fabricantes con la garantía de ser dispositivos completos, listos para su uso inmediato. Por definición, esto significa también que no será tan flexible debido al acople entre el software y el hardware.
- *Storage Area Network (SAN)*: Una red de área de almacenamiento (SAN) es una red dedicada e independiente de alta velocidad que interconecta y entrega grupos compartidos de dispositivos de almacenamiento a múltiples servidores. Cada servidor puede acceder al almacenamiento compartido como si se tratara de una unidad directamente conectada al servidor. Una SAN se suele montar con cableado, adaptadores de bus de host y conmutadores SAN conectados a matrices de almacenamiento y servidores. Cada conmutador y sistema de almacenamiento de la SAN debe estar interconectado.

2.3.3 Almacenamiento en la nube

El almacenamiento en la nube es un modelo de cómputo en la nube para almacenar datos. Permite el acceso a los usuarios a través de un dispositivo que tenga acceso a Internet a través de un proveedor de cómputo en la nube, el cual gestiona y opera el almacenamiento de datos como un servicio. Se suministra bajo demanda, con capacidad y costos justo a tiempo, y elimina la necesidad de comprar y

gestionar su propia infraestructura de almacenamiento de datos. Esto le proporciona agilidad, escala global y durabilidad, con acceso a los datos “en cualquier momento y lugar”. El almacenamiento y otros recursos de cómputo pueden ser ofrecidos a los usuarios como IaaS (*Infrastructure as a Service*), PaaS (*Platform as a service*) o SaaS (*Software as a Service*).

Los servicios en la nube han revolucionado la computación mediante estos modelos de servicio, los que han permitido que las empresas y organizaciones desarrollen una infraestructura virtualizada y proporcionen software a través de la nube, independientemente del sistema operativo de los usuarios.

2.3.4 Almacenamiento definido por software

El almacenamiento definido por software (SDS) es un enfoque en el que los recursos se abstraen del hardware de almacenamiento físico subyacente y, por tanto, son más flexibles. La flexibilidad de los recursos se combina con la programabilidad para permitir que el almacenamiento se adapte rápida y automáticamente a las nuevas demandas. Esta programabilidad incluye la gestión de recursos basada en políticas y el aprovisionamiento automático de la capacidad de almacenamiento. La naturaleza independiente del software de este modelo de despliegue también facilita en gran medida los acuerdos de nivel de servicio (SLA) y la calidad del servicio, y hace que la seguridad, la gobernanza y la protección de los datos sean mucho más fáciles de implementar. Cuando se administra correctamente aumenta el rendimiento, la disponibilidad y la eficiencia.

2.4 Teoría de manejo de colas y tiempos de espera

El estudio de las líneas de espera es una rama de las Matemáticas que se encarga de entender el comportamiento de las líneas de espera, también conocidas como colas. Las líneas de espera se encuentran con facilidad en las actividades cotidianas, como por ejemplo en un cajero automático, cine o en cualquier actividad en la que se tiene que esperar por un turno para ser atendido. En SA es necesario realizar un estudio para conocer y caracterizar las líneas de espera, con lo que se puede

observar el rendimiento general del sistema y así comprender la experiencia que se llevan los usuarios consumiendo el servicio. En un SA lo importante es mantener el orden de las operaciones de escritura y lectura que serán procesadas por los nodos de almacenamiento. En la Figura 2.1 se observa una línea de espera que mantiene las operaciones enviadas por los clientes (c) y asignadas a un nodo de almacenamiento (sn). El SA realiza las siguientes operaciones:

- **Colocación:** Poner un archivo en un nodo de almacenamiento perteneciente a una piscina de almacenamiento.
- **Localización:** Ubicar un archivo en alguno de los nodos de almacenamiento perteneciente a una piscina de almacenamiento del SA.

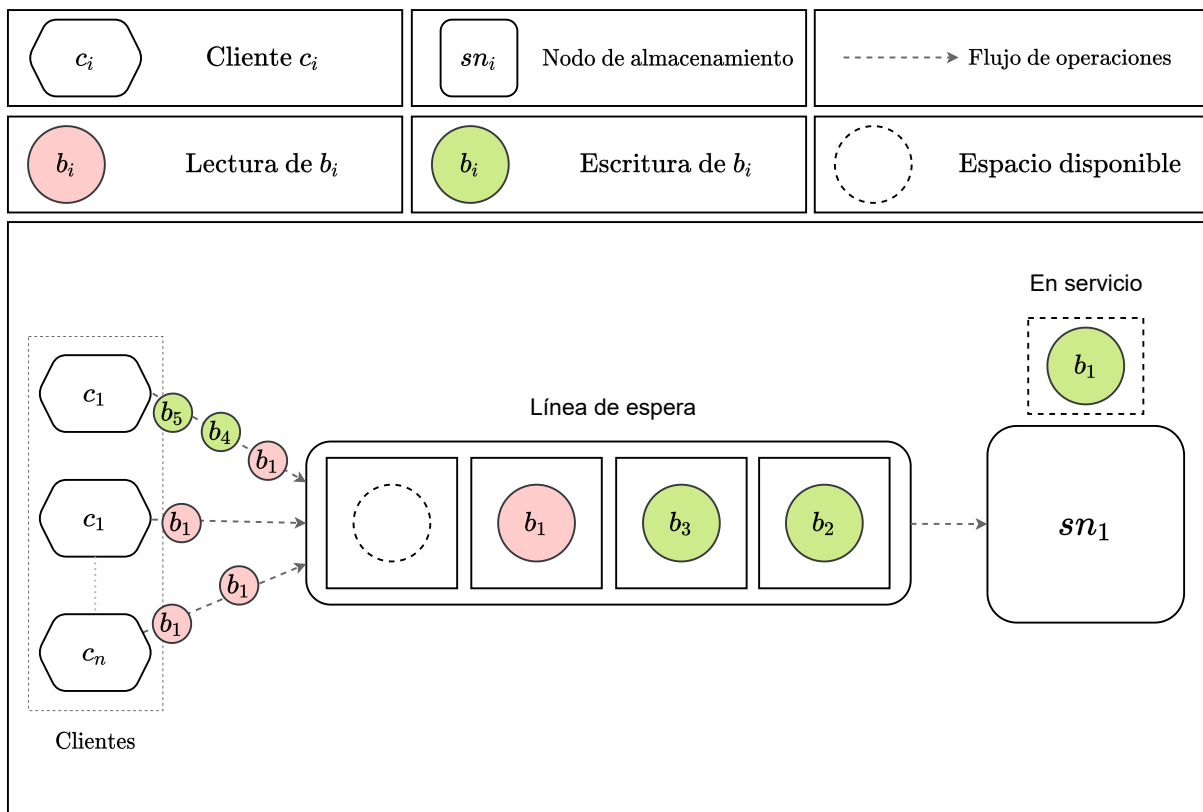


Figura 2.1: Representación conceptual de una línea de espera en el contexto de los sistemas de almacenamiento.

Las operaciones antes descritas son caracterizadas por los siguientes atributos [25]:

- **Identificador único de archivo:** Permite identificar de manera única a un archivo que ha sido colocado en el sistema.
- **Tamaño del archivo:** Tamaño del archivo en bytes.
- **Tiempo de llegada (At):** Tiempo registrado cuando la operación llega a la cola.
- **Tiempo de servicio (St):** Tiempo que se tarda un nodo en realizar una operación de colocación o localización.
- **Tiempo de espera (Wt):** Tiempo que la operación espera para ser procesada.
- **Tiempo entre peticiones (Iat):** Tiempo que existe entre una petición y otra.
- **Tiempo de salida (Et):** Tiempo en el que una operación finaliza al ser procesada por el nodo de almacenamiento.
- **Tiempo de respuesta(Rt):** Tiempo empleado por el sistema para realizar una operación de colocación o localización tomando en cuenta el tiempo que ha esperado para ser atendido.

Los tiempos antes definidos permiten entender el comportamiento de un SA. Como se observa en Figura 2.2, la petición dos (b_2) espera para ser atendida debido a que han llegado dentro del tiempo de servicio de la operación número uno (b_1). Esto significa que el tiempo de salida de la operación número uno es mayor que el tiempo de llegada de la operación número dos ($At(2) < Et(1)$). Este tiempo de espera impacta de manera negativa al SA produciendo tiempos adicionales (Wt) de espera que son absorbidos por las peticiones posteriores. Estos retrasos en la atención de peticiones degrada los tiempos de respuesta (Rt) de manera incremental conforme las peticiones se coloquen en espera, llegando a observarse un incremento exponencial en escenarios de no intervención (el sistema no reacciona ni resuelve el problema de encolamiento). Esto finalmente afecta la experiencia del usuario final (UX).

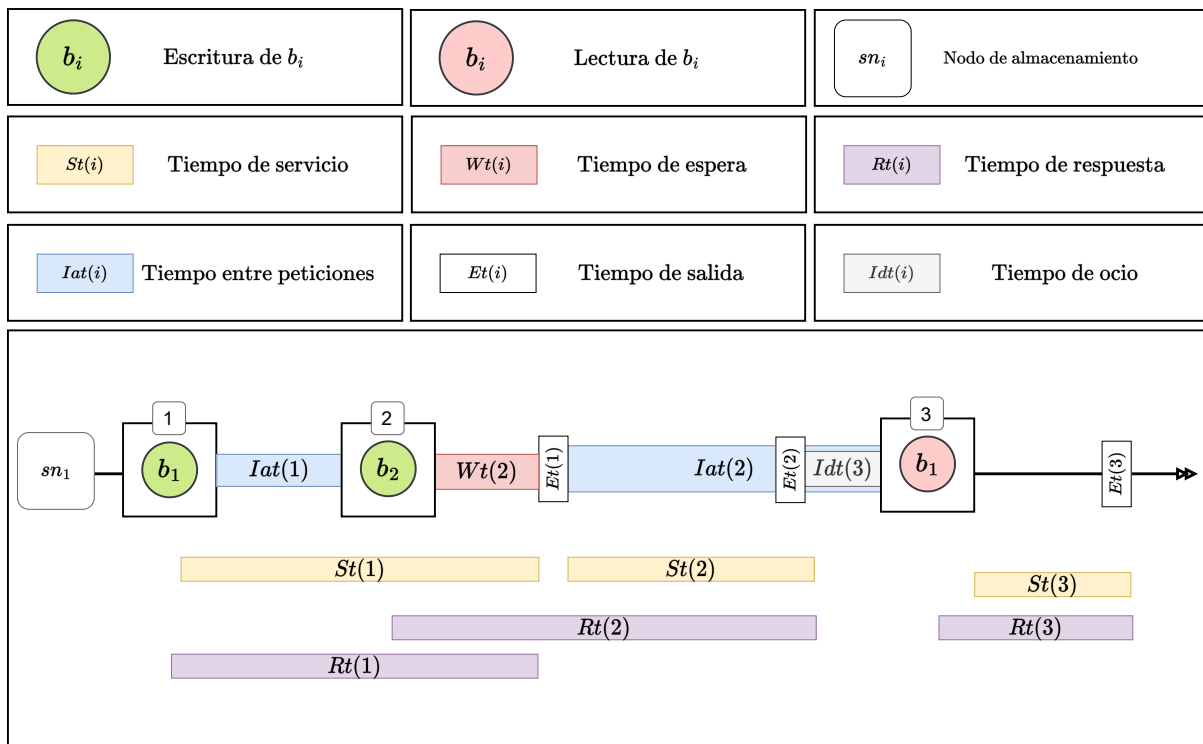


Figura 2.2: Representación conceptual de una línea de espera y los tiempos asociados a este tipo de eventos.

Una solución tradicional para enfrentar los problemas de encolamiento son los *sistemas de disponibilidad*, los cuales se basan en procesos de producción de redundancia. La redundancia que tradicionalmente se utiliza para mejorar el encolamiento de los sistemas de almacenamiento es la producción de réplicas. Una réplica es una copia exacta de un elemento que se encuentra en saturación o experimenta encolamiento (por utilización concurrente o atención de ráfagas). El proceso de producción de réplicas se realiza en dos fases: en la primera se crea la réplica y en la segunda fase se distribuye la réplica a un lugar diferente al lugar donde se ha colocado la réplica original (distribución ortogonal).

En este trabajo de tesis se consideran dos tipos de procesos de replicación:

- *replicación sistémica*, donde se crean réplicas de los nodos de almacenamiento. Esto implica la integración de un componente que se encargue de la distribución de las operaciones a los nodos pertenecientes a la piscina de almacenamiento, como se muestra en la Figura 2.3.
- *replicación de datos*, que consiste en la creación y distribución de copias de réplicas, las cuales se almacenan en distintos nodos de almacenamiento para así incrementar la disponibilidad al momento de la localización de las réplicas.

En la *replicación sistémica* la integración de nuevos nodos de almacenamiento permite incrementar de manera artificial los Iat de las operaciones, disminuyendo los tiempos de espera (Wt), como se observa en la Figura 2.4, en donde la línea de espera principal se ha dividido en tres líneas de espera, relativas a cada nodo de almacenamiento. Como se observa en la Figura 2.4, el beneficio se obtiene principalmente en las operaciones de escritura, gracias al balanceo de carga. Sin embargo, el problema del encolamiento persiste en las operaciones de lectura, debido a que suelen llegar en forma de ráfagas y con Iat pequeños, lo que degrada la UX debido al incremento de los tiempos de espera. Generalmente el encolamiento en los SA son ocasionados por las operaciones de lectura. Esto realmente es un problema debido a que se presentan en mayor medida. Para contrarrestar este problema, la *replicación de datos* ayuda a mitigar los efectos negativos de la alta concurrencia de

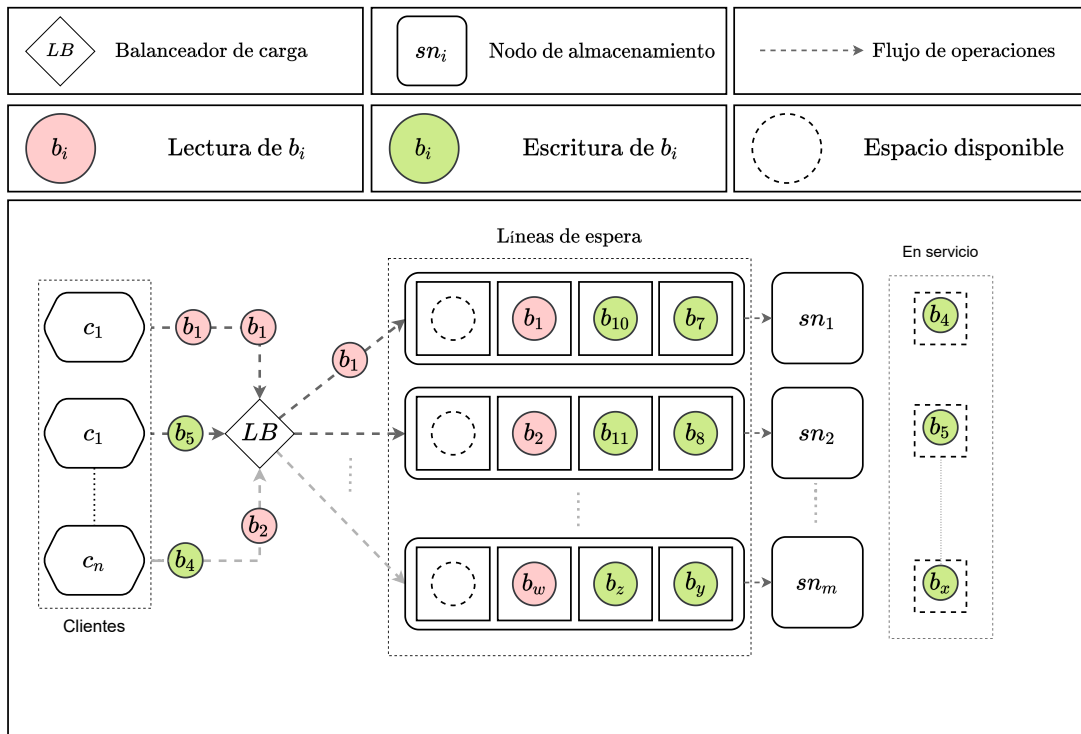


Figura 2.3: Representación de una línea de espera en conjunto con el componente de balanceo de carga.

operaciones de lectura.

En la Figura 2.5 se integra un gestor de réplicas (RM), que conceptualmente está conformado por un replicador (RP) encargado de seleccionar la cantidad de réplicas. La cantidad de réplicas se le conoce como *factor de replicación* (FR) [2, 33]. El gestor también está integrado por un balanceador de carga que deberá distribuir las FR réplicas de manera ortogonal, evitando que las réplicas sean colocadas en el mismo nodo de almacenamiento.

Desde la perspectiva de las líneas de espera independientes a cada nodo de almacenamiento, y como se observa en la Figura 2.6, la integración del componente RM permite que la replicación de datos se logre a través de la colocación de FR réplicas. A esto se le conoce como *carga de trabajo subyacente* (SW), que puede ser entendida como la inversión realizada por el sistema con la finalidad de obtener un beneficio a la hora de las lecturas, evitando el encolamiento e incrementando la disponibilidad de los recursos almacenados.

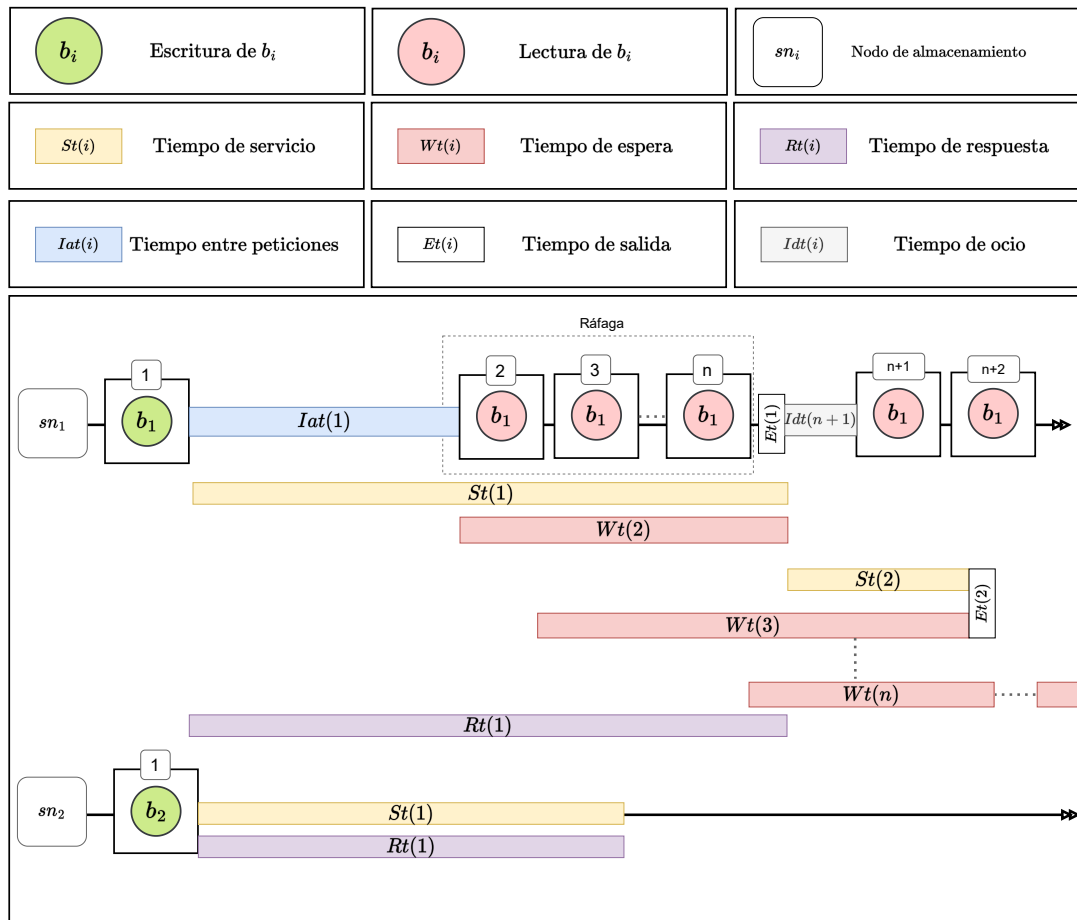


Figura 2.4: Representación conceptual de los tiempos asociados a una línea de espera con balanceo de carga.

En la Figura 2.6 se aplica un $FR = 3$ para la réplica b_1 , lo que genera una carga de trabajo subyacente (SW) que consiste en 3 operaciones de escritura, las cuales se encolarán en los nodos que almacenarán dichas réplicas. Si la replicación se realiza en el tiempo adecuado entonces el SA será capaz de brindar los beneficios de disminuir los tiempos de espera y los tiempos de respuesta.

Metáfora *balls-into-bins*

Para comprender de mejor manera cómo se ejecutan las tareas dentro de un sistema de almacenamiento que satisfaga adecuadamente las peticiones de carga y descarga de los usuarios

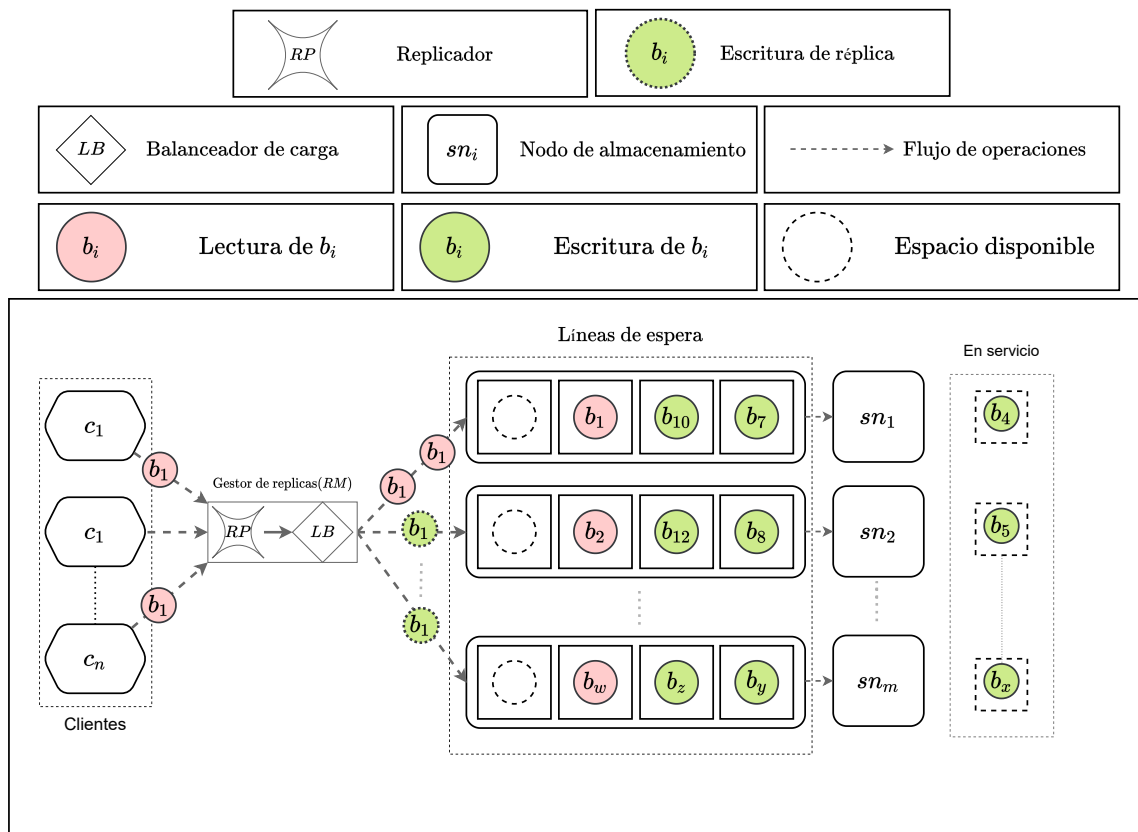


Figura 2.5: Representación conceptual de un sistema de colas con gestor de réplicas.

se puede usar la metáfora *balls-into-bins* (ver Figura 2.7). Esta metáfora consta de dos componentes principales: a) las *balls* (esferas o elementos) que pueden ser archivos, bloques, objetos, usuarios, etc. y b) los *bins* (cajas) que son aquellas entidades de almacenamiento a cargo de alojar las *balls* [47]. En los esquemas de almacenamiento se realizan operaciones funcionales basadas en mapas *ball-bin*, en donde se realizan dos operaciones principales: a) la *colocación*, en la cual se busca un bin para un *ball* y b) la *localización*, en la cual se busca un *ball* en un conjunto de *bins* [47].

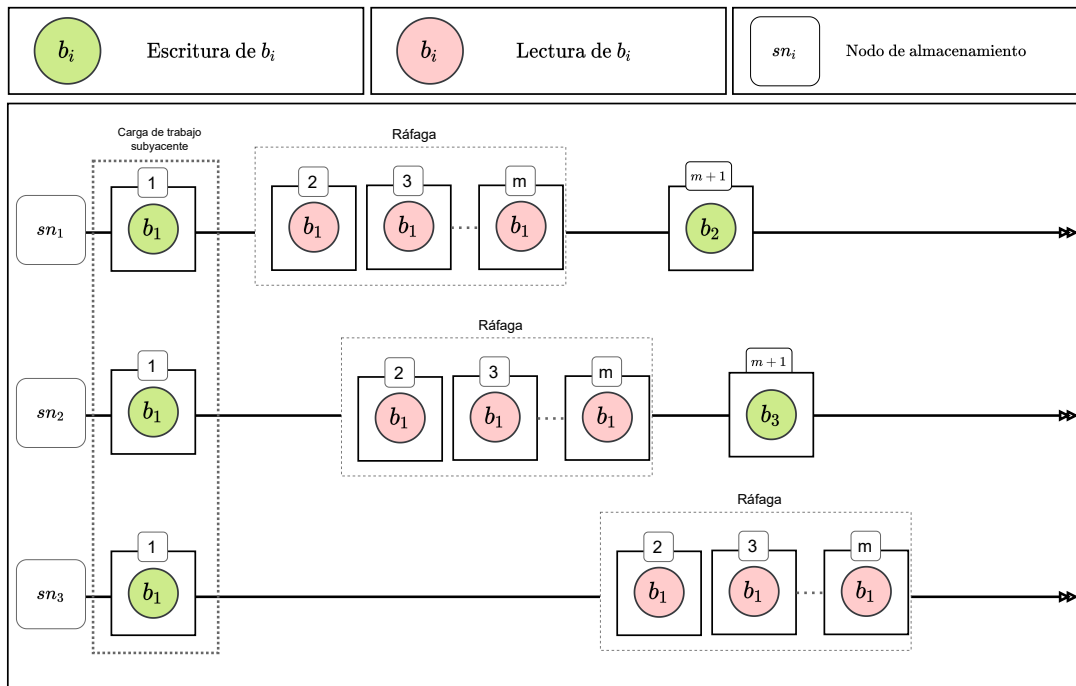


Figura 2.6: Representación conceptual de líneas de espera con carga subyacente resultado de la replicación.

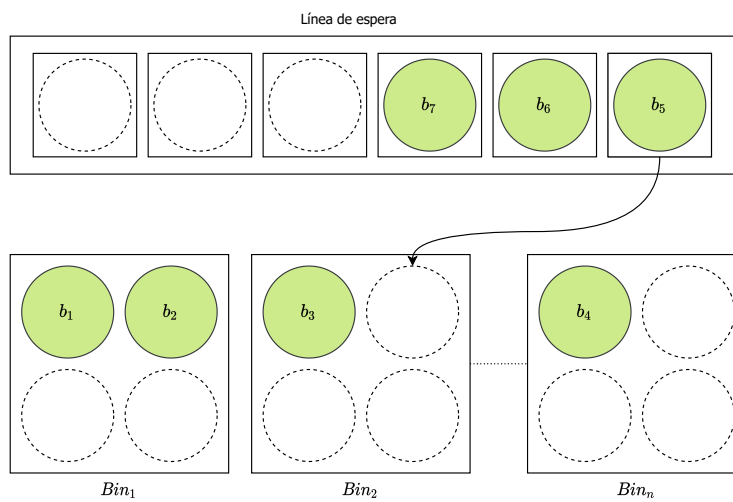


Figura 2.7: Diagrama conceptual de la metáfora *balls-into-bins*.

2.5 Replicación

Los sistemas de almacenamiento en la nube se han convertido en una tecnología prometedora para preservar datos en diferentes nodos de almacenamiento, los cuales se encuentran desplegados en un entorno de nube. Uno de los principales mecanismos en los sistemas de almacenamiento en la nube es la *replicación de datos*, la cual permite mejorar la disponibilidad, ancho de banda y tiempo de respuesta del acceso a los datos. La replicación se utiliza comúnmente en aplicaciones con datos compartidos masivamente y que necesitan ser localizados desde varios sitios en diversas áreas distribuidas geográficamente. Por lo tanto, los datos solicitados puede seguir siendo servidos desde otros sitios cuando uno de los sitios que mantiene su réplica falla. La replicación logra sus objetivos satisfaciendo las solicitudes de los sitios más cercanos para mantener la réplica de los datos deseados.

2.5.1 Factores de la replicación de datos

La replicación de datos es directamente afectada por la respuesta que se le dé a las siguientes preguntas:

- ¿Quién inicia el proceso de replicación?
- ¿Qué datos deberían de replicarse?
- ¿Dónde deberían de colocarse las nuevas replicas?
- ¿Cómo debe llevarse acabo el proceso de replicación?
- ¿Cuándo debería accionarse la replicación?

Considerando la primer pregunta, replicar todos los datos incrementa el costo de replicación y almacenamiento de los datos causando una degradación en la experiencia del usuario. No existe un enfoque razonable para encontrar los datos más populares debido a que puede depender de diversos

factores relativos al problema que se pretende abordar. Los *datos calientes* son aquellos que presentan la mayor demanda de operaciones de lectura.

Considerando la segunda pregunta, la creación de réplicas resultará en un incremento en la disponibilidad y también conseguirá impactar la capacidad de almacenamiento. Es por esto que una selección adecuada de un *FR* para la creación un número razonable de réplicas es un aspecto importante a tomar en cuenta en la replicación de datos. La creación de las réplicas debe de accionarse en un tiempo en específico, la decisión de accionar la replicación influye en la utilización de las réplicas. Esto es, una replicación precoz o una replicación tardía podría resultar en la creación de réplicas innecesarias.

Determinar la ubicación de las nuevas réplicas es otro factor que impacta a la replicación. Colocar las réplicas de manera aleatoria no incrementará el rendimiento del sistema [12]. Las réplicas deberían de colocarse en los nodos de almacenamiento que incrementen el rendimiento del sistema, lo cual se consigue mediante balanceadores de carga que toman la decisión de manera reactiva; esto se aborda con más detalle en la subsección 2.6.

2.5.2 Métodos de replicación

Los métodos de replicación de datos (Figura 2.8) son definidos como una serie de pasos que deben de seguirse de manera sistematizada. Pueden clasificarse en los siguientes grupos basados en la forma en la que administran el ciclo de vida de las réplicas, desde la selección de nodos de almacenamiento, selección los datos a replicar, producción de réplicas y la eliminación de la réplicas:

- **Estático:** Se mantiene un *FR* constante para todos los archivos. La distribución de las réplicas es de manera determinista.
- **Dinámico:** El *FR* cambia de manera dinámica, tomando en cuenta diversos factores que influyen en su selección. La distribución de las réplicas se realiza de manera dinámica:

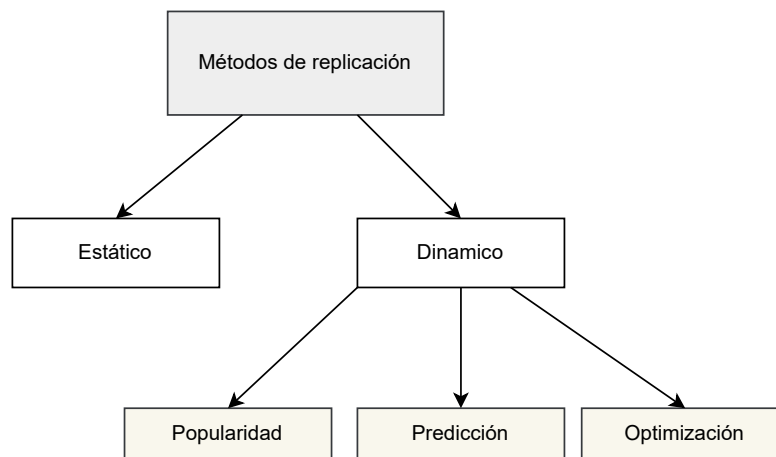


Figura 2.8: Métodos de replicación

- **Basado en la optimización:** La creación de las réplicas a partir de los archivos existentes se basa en el más optimizado.
- **Basado en la popularidad:** Se seleccionan los archivos más populares para ser replicados.
- **Basado en la predicción:** Los archivos se seleccionan basándose en un enfoque de aprendizaje (supervisado o no supervisado).

La selección del método de replicación más adecuado depende de factores como la estrategia de replicación, el entorno, la decisión de replicación y la carga de trabajo. La decisión para la selección de un método de replicación se toma con base en los diversos factores y características del entorno de almacenamiento.

2.6 Balanceo de carga

El balanceo de carga permite distribuir la carga de trabajo entre diferentes nodos de almacenamiento con la finalidad de conseguir una distribución uniforme de las operaciones. Los balanceadores de carga pueden clasificarse en dos grupos: estáticos y dinámicos, los cuales se ilustran

en la Figura 2.9. En los balanceadores de carga estáticos el tiempo que se requiere para encontrar una solución es alto, y se requiere un volumen de datos de entrada grande [11, 44, 62]. Los balanceadores probabilísticos son balanceadores estáticos debido a que no cambia el balanceo durante todo el ciclo de vida del SA. Asigna diferentes probabilidades a los nodos para ser seleccionados, lo que se observará en la distribución de operaciones es un sesgo asignando un mayor número de peticiones a los nodos con mayor probabilidad, lo que ocasionará cuellos de botella. Los balanceadores deterministas son aquellos que otorgan siempre la misma distribución de operaciones, sin mantener ninguna información de la distribución. Un algoritmo *pseudoaleatorio* asigna la carga generando números aleatorios entre $[0, RD - 1)$ o *round robin*, el cual distribuye la carga entre un conjunto de nodos. Estos dos algoritmos son un ejemplo de balanceadores estáticos deterministas.

Los balanceadores de carga dinámicos pueden ser distribuidos o centralizados. Los balanceadores distribuidos toman la decisión para la distribución entre un conjunto de componentes pertenecientes a una piscina de balanceadores (*Plb*). Si los componentes colaboran entre sí para llegar a un consenso entonces son denominados *cooperativos*. Los balanceadores denominados *no cooperativos* toman las decisiones de distribución de manera independiente de los nodos pertenecientes a *Plb*.

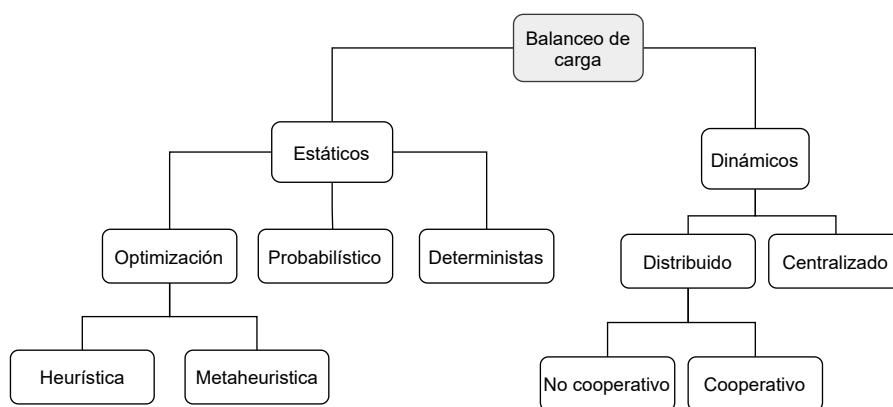


Figura 2.9: Balanceo de carga

3

Estado del arte

En esta sección se describen con más detalle algunos conceptos básicos del trabajo de tesis y se presenta el trabajo relacionado sobre los distintos sistemas de almacenamiento que utilizan estrategias de replicación dinámica.

3.1 Replicación de datos

El mecanismo de replicación determina qué archivos deben ser replicados, cuándo deben crearse las nuevas réplicas y dónde deben colocarse las nuevas réplicas. A continuación se describen los 2 principales categorías en las cuales se clasifican los mecanismos de replicación.

3.1.1 Replicación estática

Los mecanismos de replicación de datos estáticos siguen políticas deterministas, por ejemplo el factor de replicación (FR), así como la cantidad de recursos disponibles (RD), los cuales son

definidos en tiempo de diseño. Las estrategias estáticas son las más simples de implementar, pero a menudo no son las más utilizadas debido a que no son totalmente adaptables al entorno, no capturan las interacciones o no toman en cuenta alguna métrica del sistema o del entorno, con las cuales se pueda determinar un número de réplicas adecuado en tiempo de ejecución. En la literatura se encuentran diferentes sistemas de almacenamiento que utilizan un mecanismo estático, por ejemplo Ghemawat *et al.* (2003) [31] propusieron *Google File System* (GFS), el cual utiliza una estrategia estática de replicación. GFS considera diferentes factores para la colocación de las nuevas réplicas, como la utilización de disco para limitar la colocación de réplicas en los nodos de almacenamiento. La limitación de los mecanismos estáticos es que se utiliza un número de réplica fijo para todos los archivos, lo que puede no ser la mejor solución debido al constante cambio en las interacciones con los archivos que realizan los usuarios. Cidon *et al.* (2013) [23] propusieron *MinCopysets*, que utiliza un esquema de replicación estático y escalable. Los mecanismos utilizados para la distribución y durabilidad de los datos se encuentran desacoplados, permitiendo utilizar diferentes mecanismos en tiempo de diseño. Cada nodo es agrupado en una piscina de almacenamiento¹ con un factor de replicación constante, un nodo primario se encarga de distribuir los archivos en los nodos secundarios para ser almacenados.

Hassan *et al.* (2009) [5] propusieron un algoritmo evolutivo multi-objetivo (*MOE*) para la replicación de datos en sistemas almacenamiento en la nube a gran escala utilizando un método evolutivo para encontrar la mejor estrategia de replicación. Este MOE mejora el almacenamiento, la latencia y la fiabilidad del sistema, pero en su diseño no se consideró el costo total de energía del centro de datos como el principal objetivo de optimización [42]. El MOE depende de tomar información histórica del sistema y alimentarla a un motor, donde no sólo intenta mantener la fiabilidad, la latencia y el almacenamiento dentro de las limitaciones, sino que, además, intenta optimizar la latencia, la fiabilidad y el almacenamiento con el fin de encontrar diferentes compensaciones entre estos objetivos

¹En la literatura el concepto *piscina de almacenamiento* hace referencia a un conjunto de nodos de almacenamiento en un SA.

utilizando más de un objetivo.

Long *et al.* (2014) [42] propusieron una estrategia de gestión de replicación optimizada multi-objetivo (*MORM*) para sistemas de almacenamiento en la nube utilizando un algoritmo multi-objetivo. Se consideraron cinco factores, entre los que se incluyen la indisponibilidad promedio de archivos, el tiempo de servicio promedio, la variación de carga, el consumo de energía y la latencia de acceso promedio para capturar la compensación entre estos factores y la relación entre el número de réplicas, el diseño de réplicas y el rendimiento. *MORM* es un algoritmo de replicación fuera de línea (*offline*) que hace colocación de réplicas en forma aleatoria teniendo en cuenta la capacidad de los nodos de almacenamiento. *MORM* necesita conocimiento completo del tiempo de servicio y las tasas de acceso para todos los archivos asignados. Cuando los archivos llegan de manera ininterrumpida en forma de ráfagas con patrones impredecibles *MORM* ya no es práctico, porque las técnicas de replicación de archivos estáticos no son aplicables en tales condiciones.

Zeng y Veeravalli (2014) [63] presentaron una técnica de balanceo de carga óptima para centros de datos en la nube a gran escala, que se componen de miles de servidores de datos sin procesar (RDS) y cientos de servidores de metadatos (MDS) conectados por redes arbitrarias. El propósito de la técnica fue lograr el tiempo medio de respuesta (MRT) mínimo de las solicitudes de metadatos, que es uno de los índices de rendimiento más importantes que deben considerar los proveedores de servicios de datos. Esta técnica modela MDS como un sistema $M/M/1$. $M/M/1$ representa la longitud de la cola en un sistema que tiene un único servidor, donde las llegadas están determinadas por un proceso de Poisson y los tiempos de servicio de las operaciones tienen una distribución exponencial. El nombre del modelo se escribe en la notación de Kendall, el cual formula la función objetivo para el MRT de las solicitudes de metadatos que llegan a MDS.

3.1.2 Replicación dinámica

Las estrategias dinámicas para la replicación de datos en entornos de la nube colocan y eliminan automáticamente las réplicas de acuerdo con los cambios en el patrón de acceso de los usuarios, la

capacidad de almacenamiento o el ancho de banda. Toman decisiones inteligentes sobre la colocación de los datos en función de la información del entorno. Aunque estos enfoques presentan algunos inconvenientes, como la dificultad para recopilar información en tiempo de ejecución de todos los nodos de almacenamiento en una infraestructura de nube y mantener la consistencia del archivo de datos. Las estrategias de replicación dinámica de datos incluyen tres fases: (i) analizar y modelar la relación entre el número de réplicas y la disponibilidad del sistema, (ii) reconocer los datos populares y (iii) desencadenar un proceso de replicación cuando los datos superan un umbral.

Wei *et al.* (2010) [60] propusieron un modelo para captar la relación entre el número de réplicas y la disponibilidad denominado *Cost-effective Dynamic Replication Management* (CDRM). CDRM calcula y mantiene un número mínimo de réplicas para una condición de disponibilidad determinada. Li *et al.* (2011) [39] presentaron una estrategia de replicación dinámica de datos denominada *Cost-effective Incremental Replication* (CIR). CIR es una estrategia de confiabilidad de datos para aplicaciones basadas en la nube, en la que el enfoque principal es administrar de manera rentable el problema de fiabilidad de datos en un centro de datos. Sun *et al.* (2012) [58] propusieron la estrategia *Dynamic Data Replication Strategy* (D2RS) en tres fases: (i) definición de qué dato y cuándo tales datos deben replicarse para cumplir con los requisitos de los usuarios (reducción del tiempo de espera y la aceleración del acceso a los datos); (ii) cuántas nuevas réplicas se deben realizar en el sistema para cumplir con un requisito de disponibilidad determinado y (iii) dónde se deben colocar las nuevas réplicas para cumplir con la función del sistema, la tasa de ejecución exitosa y los requisitos de consumo de ancho de banda.

Bai *et al.* (2013) [9] propusieron una estrategia de administración de réplicas conocida como *Response Time-based Replica Management* (RTRM). Aquí la cantidad de réplicas está en función del tiempo de respuesta promedio. RTRM predice el ancho de banda entre los servidores de réplicas al recibir nuevas solicitudes. Estas estrategias toman en cuenta la creación, la selección y la distribución en los nodos de almacenamiento. Estas estrategias son disparadas una vez el umbral en el tiempo de respuesta es superado, accionando las estrategias de creación, selección y distribución.

Lin *et al.* (2013) [40] presentaron un algoritmo llamado *High-QoS first-replication* (HQFR). En este algoritmo se considera el problema de replicación de datos sensible a la calidad del servicio (QADR) para aplicaciones con uso intensivo de datos en sistemas en la nube. La principal tarea de QADR es definir de manera eficiente los requisitos de QoS de las aplicaciones en la replicación de datos y minimizar el costo de replicación de datos.

Chen *et al.* (2014) [20] propusieron *Scalable service-oriented replication* (SSOR), la cual es capaz de satisfacer los requisitos de coherencia en aplicaciones orientadas a servicios y basadas en la nube. Para abordar las importantes diferencias entre la replicación orientada a servicios (SOR) y la replicación orientada a datos (DOR) SSOR proporciona nuevos formalismos para definir servicios en SOR. SSOR se puede utilizar para garantizar la coherencia y gestionar los requisitos de coherencia para colecciones de servicios dentro de una región.

3.2 Balanceo de carga

El balanceo de carga se define como la distribución metódica y eficiente del tráfico de la red o de las aplicaciones en varios servidores. Un balanceador de carga intercepta las peticiones enviadas por los usuarios, divide la carga entre un conjunto de nodos, los cuales deben de recibir y procesar las peticiones. Además un balanceador de carga debe asegurarse de tener un registro de los nodos disponibles, en caso de que alguno de los nodos no responda éste no debe considerarse en la distribución de peticiones. Los balanceadores de carga se pueden clasificar en dos grupos principales: estáticos y dinámicos.

3.2.1 Balanceo de carga estático

En el balanceo de carga estático la distribución depende de la carga del nodo, en el tiempo de selección (anticipada), por lo que toman la decisión de selección de un nodo de manera *proactiva*. El principal inconveniente de los algoritmos de balanceo de carga estática es que el estado actual del

sistema no se tiene en cuenta a la hora de tomar decisiones y, por lo tanto, no es un enfoque adecuado en los sistemas distribuidos donde la mayoría de los estados del sistema cambian dinámicamente. Una de las aplicaciones en donde el balanceo de carga estático sobresale es en sistemas que tienen una carga de trabajo constante. Los balanceadores de carga estáticos pueden ser deterministas o probabilísticos. En los deterministas, la carga extra de un nodo específico será transferida siempre al mismo nodo, a diferencia de un enfoque probabilístico, donde cada nodo envía la carga de trabajo extra con probabilidad p a un nodo con probabilidad $1 - p$. Entre estos algoritmos destacan los siguientes:

- *Round-robin* [34]: Distribuye la carga equitativamente entre los nodos del sistema. Una de las ventajas de este algoritmo es que no requiere comunicación entre los nodos para realizar la selección de nodos en donde se distribuirá la carga.
- *Pseudo aleatorio* [49]: La distribución de la carga se realiza seleccionando de forma aleatoria los nodos pertenecientes al conjunto de nodos disponibles. Cuando mayor sea la cantidad de peticiones este algoritmo distribuirá la carga acercándose a una distribución uniforme.
- *Central Manager Algorithm (CMA)*: Un nodo central es encargado de tomar la decisión de la distribución de las nuevas peticiones. El nodo central también se encarga de recibir mensajes de los nodos cada vez que la carga ha cambiado.

3.2.2 Balanceo de carga dinámico

En el balanceo de carga dinámico la distribución se realiza en tiempo de ejecución, por lo que toman la decisión de selección de un nodo de manera *reactiva*. Se utiliza información del sistema para realizar una decisión en la distribución. Si los balanceadores de carga son capaces de cambiar utilizando el estado del sistema se consideran adaptativos. Las aplicaciones de un balanceador de carga dinámico se centran en sistemas que reciben cargas de trabajo que no son predecibles y son altamente cambiantes en el tiempo. Los beneficios obtenidos en rendimiento y distribución de la carga

son mayores en entornos cambiantes en comparación con los que se obtienen utilizando balanceadores estáticos. Los beneficios que se obtienen traen consigo complejidad en la implementación y diseño de los balanceadores dinámicos.

Los balanceadores de carga dinámicos se pueden subdividir en centralizados y distribuidos. En balanceadores de carga centralizados un solo nodo se encarga de obtener la información de todos los nodos, para después utilizarla en realizar la distribución de la carga correspondiente. Estos balanceadores son buenos en sistemas pequeños. En los balanceadores de carga distribuidos, a diferencia de los centralizados, no existe un nodo central, en su lugar múltiples nodos colectan información. La información mantenida por los nodos es actualizada cuando el estado de alguno de los nodos cambia. El intercambio de información se lleva a cabo utilizando algún modelo de comunicación entre nodos, lo cual añade un *overhead* extra al sistema. Los balanceadores dinámicos se comportan mejor en escenarios donde la carga no es predecible. Los beneficios se obtienen a cambio de complejidad, debido a que los balanceadores dinámicos necesitan un sistema para el monitoreo de los nodos con la finalidad de obtener información para tomar la decisión, esto también impacta incrementando el *overhead* del sistema.

Existen 2 mecanismos para llevar a cabo la recolección de información de los nodos. Los mecanismos más básicos y populares son *centralized pooling* y *agent*. Estos mecanismos utilizan *broadcasting*, es decir, cada nodo realiza un *broadcast* con su información, la cual podrá ser accedida por otros nodos. Los mecanismos de *agent* se han utilizado en años recientes, algunos ejemplos que se encuentran en la literatura son [13] y [18]. En éstos los agentes cooperan entre sí para balancear la carga. La importancia del balanceo de carga en los sistemas de gran escala han llevado al desarrollo de diferentes técnicas para otorgar una solución óptima a los problemas.

3.3 Consistencia de datos

La consistencia de datos en los sistemas de almacenamiento indica que cada réplica de datos es coherente después de operaciones simultáneas. Cuando una réplica cambia en alguno de los nodos, el sistema necesita cambiar todos los datos correspondientes en otros nodos para que la consistencia se pueda mantener y así los usuarios puedan observar siempre la última versión de los datos. En la literatura se definen diferentes modelos de consistencia de datos, a continuación se describen las dos más importantes.

3.3.1 Consistencia eventual

Es un modelo de consistencia que se utiliza en diferentes bases de datos distribuidas [1, 6, 26]. A pesar de todos los cambios que se realizan sobre las réplicas de los datos, requiere que se obtenga una versión coherente. La consistencia eventual es una forma específica de consistencia débil, donde los sistemas de almacenamiento garantizan que si no se realizan nuevas actualizaciones a una réplica, eventualmente todos los accesos devolverán el último valor actualizado. El sistema más popular que implementa la consistencia de datos eventual es el DNS (*Domain Name Server*) [29]. Las actualizaciones de un nombre de dominio se distribuyen de acuerdo con un patrón configurado. Eventualmente, todos los clientes verán el mismo estado.

Ahamad *et al.* (1995) [3] propusieron un protocolo de compromiso atómico de una fase para consistencia de datos. Este método requiere que todos los participantes estén regidos por un mismo control de concurrencia, donde se utilizan *distributed locks* [37] para realizar la sincronización de los datos. Un *lock* o mutex es una primitiva de sincronización que impone límites al acceso a un recurso cuando hay muchos hilos de ejecución. Por lo tanto un *distributed lock* es una primitiva de sincronización entre nodos pertenecientes a un sistema distribuido.

Bermbach y Tai. (2013) [10] propusieron un enfoque para evaluar cuando las réplicas se vuelven

obsoletas en un sistema de almacenamiento de datos distribuido. Esta propuesta la implementaron en Amazon S3. El enfoque tiene un *escritor* que escribe periódicamente una marca de tiempo local más un número de versión en el sistema de almacenamiento, en esto se considera la diferencia entre los tiempos de creación (*timestamps*).

Cooper *et al.* (2008) [24] desarrollaron un sistema de base de datos paralelo y distribuido geográficamente para aplicaciones web de Yahoo llamado PNUTS. PNUTS proporciona almacenamiento de datos organizado como tablas de hash ordenadas, permitiendo obtener una latencia baja para un gran número de solicitudes simultáneas, incluidas actualizaciones y consultas. El modelo de consistencia es mediante línea de tiempo por registro; esto es, para todas las réplicas de un registro dado aplicará todas las actualizaciones al registro en el mismo orden. Esto proporciona un modelo de consistencia que combina las transacciones serializadas y consistencia eventual.

3.3.2 Consistencia de datos fuerte

Un sistema de almacenamiento sigue un modelo de consistencia fuerte si todos los accesos a los datos son vistos por todos los procesos paralelos (o nodos, procesadores, etc.) en el mismo orden (secuencialmente).

Windows Azure Storage (WAS) [17] es un sistema de almacenamiento en la nube que brinda a los clientes la capacidad de almacenar cantidades aparentemente ilimitadas de datos durante cualquier período de tiempo. Los clientes de WAS tienen acceso a sus datos desde cualquier lugar en cualquier momento y solo pagan por lo que usan y almacenan. En WAS los datos se almacenan de forma duradera mediante la replicación local y geográfica para facilitar la recuperación ante desastres. WAS hace uso de diferentes modelos de consistencia, y tiene la capacidad de realizar lecturas, escrituras y eliminaciones condicionales utilizando un control de concurrencia [38] para los datos fuertemente consistentes. Por esto WAS proporciona las tres propiedades del teorema de CAP [16], el cual afirma que son difíciles de lograr al mismo tiempo la consistencia fuerte, la alta disponibilidad y la tolerancia al particionamiento.

3.4 Almacenamiento en la nube

Drago *et al.* [27] presentaron una variante de *Dropox*, el cual es un sistema de almacenamiento basado en la nube que permite reducir la cantidad de datos intercambiados mediante el uso de la codificación de datos. El sistema mantiene una base de datos de información de metadatos en cada dispositivo de forma local y comprime los fragmentos antes de enviarlos. Este *Dropox* posee dos componentes principales en su arquitectura, el primero es el control, el cual se encuentra bajo el cargo directo del proveedor *Dropbox Inc.*; el segundo componente son los servidores de almacenamiento de datos, los cuales se encuentran a cargo de *Amazon Elastic Compute Cloud (EC2)* y *Simple Storage Service (S3)*. No obstante, al contar con una arquitectura de nube pública, no existe garantía de que los datos no sean accedidos por terceros.

Calder *et al.* [17] presentaron *Windows Azure*, un servicio de almacenamiento en la nube de alta disponibilidad con una fuerte consistencia. *Windows Azure* permite a los clientes almacenar grandes cantidades de datos. Los datos son almacenados de forma “duradera” utilizando replicación local y geográfica, lo cual facilita la recuperación de los mismos en caso de algún desastre. Es escalable, cuenta con una alta consistencia, alta disponibilidad y tolerancia a fallos. Asimismo balancea la carga de forma automática, para ello divide y fusiona los rangos de partición de acuerdo con las demandas de tráfico de las aplicaciones. No obstante, para poder utilizar *Windows Azure* en toda su capacidad es necesario contar con experiencia en la plataforma, para así poder garantizar que todas las piezas funcionen de manera eficiente.

3.5 Infraestructura definida por código

La Infraestructura definida por código (IaC) es un enfoque basado en prácticas de desarrollo de software. Los cambios en la infraestructura son hechos por definiciones desplegadas en los sistemas a través de procesos no supervisados que son incluidos a través de la red. En este enfoque la

infraestructura es tratada como software y datos, esto permite a las organizaciones aplicar o utilizar herramientas de desarrollo de software, tales como los controladores de versiones, librerías para pruebas automáticas y herramientas de orquestación [48].

Terraform [14] es una herramienta de código abierto desarrollada por HashiCorp en lenguaje GO. Se utiliza para desplegar infraestructura desde una computadora, no es necesaria ninguna otra infraestructura adicional para ejecutarlo. *Terraform* cuenta con un archivo binario que traduce el contenido del archivo de configuración en llamadas a los distintos proveedores de nube (por ejemplo, Google Cloud, Amazon Web Service, Open Stack, etc.). La versión gratuita de *Terraform* es limitada, por lo que es necesario pagar una licencia para adquirir características superiores de manejo. Con *Terraform* no se puede volver a un estado anterior del sistema, por lo que si se presentan errores es necesario identificarlos y corregirlos al momento, o en su defecto volver a desplegar toda la infraestructura.

Openstack Swift [7] es un sistema de almacenamiento de objetos (*Object Storage system*) altamente escalable, durable y multi-usuario, diseñado para almacenar grandes cantidades de datos no estructurados (p.ej. documentos, contenido web, respaldos, entre otros) a bajo costo. *Openstack Swift* permite almacenar, recuperar y eliminar objetos en la nube y sus metadatos a través de una API HTTP. Los programadores pueden utilizar la API de *Openstack Swift* en lenguajes como *Java*, *Python*, *Ruby* y *C#*. En *Openstack Swift* todo el tráfico va a través de servidores proxy, lo cual provoca que la velocidad de transferencia disminuya y aumente la latencia. Estos problemas de latencia pueden causar que las réplicas de los datos no sean actualizadas simultáneamente y, por lo tanto, algunos usuarios tendrán versiones desactualizadas de los datos.

AWS CloudFormation [57] es un servicio que permite modelar y desplegar recursos de *Amazon Web Services* mediante la creación de plantillas (archivos YAML o JSON), los cuales describen los recursos que se desean administrar. Además, permite la construcción de la infraestructura y aplicaciones sin necesidad de realizar procesos manuales; se puede regresar a un estado anterior de forma automática en caso de errores. La Figura 3.1, muestra las etapas de construcción de una

solución con *AWS CloudFormation*. En la primera etapa se crea el archivo de configuración, el cual pasará a ser revisado y compilado en una segunda etapa. Posteriormente, en la tercera etapa se podrá utilizar *AWS CloudFormation* desde la terminal o desde el navegador. Finalmente, en la cuarta etapa, *AWS CloudFormation* construirá y configurará los recursos especificados.

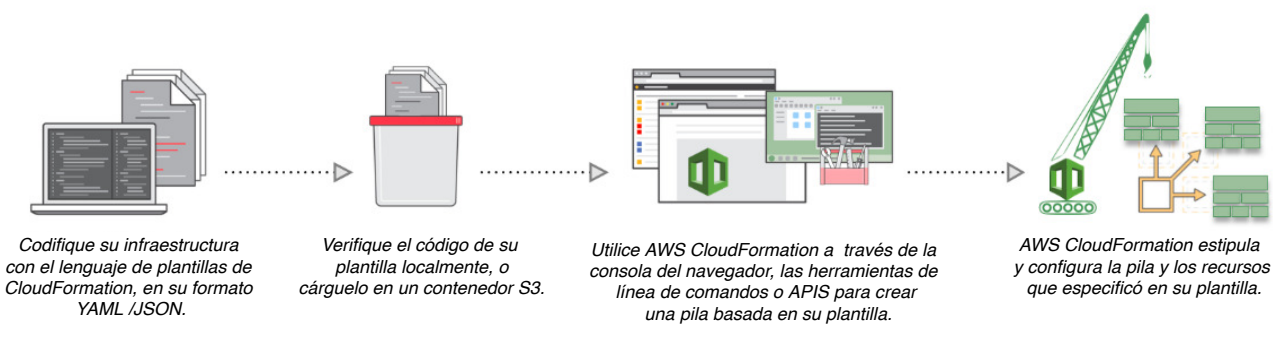


Figura 3.1: Etapas de construcción de *AWS CloudFormation*

3.6 Almacenamiento definido por software

Los sistemas de almacenamiento definidos por software emergieron como una propuesta para una nueva categoría de productos de software de almacenamiento. Pueden ser una tecnología independiente, o bien un elemento dentro de un centro de datos definido por software.

El término SDS (*Software-defined storage*) se deriva del término redes definidas por software², la cual fue utilizada para describir un enfoque de tecnología de red que abstrae varios elementos de redes, y además crea una abstracción en el software. El enfoque almacenamiento definido por software abstrae y simplifica la gestión de redes en servicios virtuales. A continuación se muestra una lista de los atributos de los SDS que normalmente se encuentran en el mercado [19]:

²Es un paradigma de red en donde la inteligencia de la red está lógicamente centralizada en los controladores basados en software y los dispositivos de red se convierten en simples dispositivos de reenvío de paquetes que se pueden programar a través de una interfaz abierta [50].

- Permite que los clientes “armen” una solución de acuerdo a sus necesidades.
- Puede funcionar con cualquier hardware o con hardware desarrollado específicamente para este tipo de soluciones.
- La mayoría de estas soluciones permite la ampliación del espacio de almacenamiento.
- Permite la construcción incremental de servicios de almacenamiento.
- Proporciona una gestión automatizada.
- Incluyen una interfaz para que los usuarios puedan generar su solución.
- Permite el etiquetado de los metadatos para controlar el tipo de almacenamiento y servicios de datos aplicados, esto mediante una gestión a nivel de servicio. Además el nivel de granularidad puede ser modificado con el tiempo.
- Permite el establecimiento de políticas para administrar los servicios de almacenamiento y datos.

IBM [61] desarrolló *IBM System Storage DS8000* como un sistema de almacenamiento de alto rendimiento, capacidad y resistencia. ofrece alta disponibilidad, brinda soporte a múltiples plataformas y provee herramientas de administración simplificadas. El consolidar múltiples sistemas de almacenamiento en un solo equipo ayuda a simplificar el entorno de almacenamiento. No obstante, su interfaz es difícil de usar y no especifica de forma clara los requerimientos de uso, además de ser costosa.

Red Hat [55] presentó *Ceph*, un proyecto de código abierto que utiliza la tecnología almacenamiento de definido por software. *Ceph* proporciona soluciones unificadas definidas por software de almacenamiento, archivos y almacenamiento de objetos. La idea se centra en proporcionar un sistema de almacenamiento distribuido escalable y de alto rendimiento. Además, provee características tales como confiabilidad, codificación de borrado, niveles de caché y conteo. Sin

embargo, *Ceph* posee características de configuración muy limitadas y no brinda un reporte detallado de los errores que suelen ocurrir en el sistema. Por otro lado, no existe un mecanismo que distribuya el archivo de configuración a todos los nodos existentes, lo cual se vuelve una tarea muy tediosa entre más grande sea el sistema.

3.7 Resumen del estado del arte

En esta sección se presenta el resumen de los trabajos en el estado del arte que se menciona en este Capítulo. En la Tabla 3.1 se muestran, de manera breve, las características de algunos sistemas de almacenamiento, tales como:

- **Elasticidad:** Propiedad de un sistema que permite adaptarse a los cambios en la carga de trabajo mediante el aprovisionamiento y des-aprovisionamiento de recursos de manera autónoma, de manera que en cada momento los recursos disponibles coincidan con la demanda actual.
- **Balanceo de carga:** El proceso de redistribuir la carga de trabajo general del sistema entre todos los nodos del sistema distribuido para mejorar tanto la utilización de recursos de cómputo como el tiempo de respuesta del trabajo, mientras se evita una situación en la que algunos nodos están sobrecargados y otros están sub-cargados o inactivos.
- **Métodos de replicación:** Es la estrategia con la cual se lleva a cabo la creación de las réplicas de los datos (archivos).
- **Modelo de consistencia:** Especifica un contrato entre el usuario y el sistema en el que el sistema garantiza un nivel de consistencia/coherencia en los resultados de la lectura, escritura o actualización de la memoria, y que serán predecibles.

Parámetro de comparación	Abreviacion	Significado
Entorno	ENV	Especifica el entorno distribuido utilizado en el trabajo, es decir, si la técnica se propone para los sistemas de almacenamiento por red o los sistemas de almacenamiento en la nube
Arquitectura	ARQ	Especifica el tipo de arquitectura utilizada en la obra. Las diferentes arquitecturas utilizadas son: Multi-nivel jerárquico, Árbol de varios niveles, Maestro-esclavo y arquitecturas basadas en grafos (DAG).
Disponibilidad	AVA	Este parámetro especifica si el trabajo se centra en mejorar la disponibilidad. S significa que el trabajo se centra en aumentar la disponibilidad y N significa que el trabajo no se centra en el aspecto de la disponibilidad.
Minimización de costos	CMI	Este parámetro especifica si la estrategia de replicación se centra en minimizar el coste de la replicación. S significa Sí y N significa No
Tipo	TY	Este parámetro especifica si es un Modelo (MO), Simulación(SIM), Estrategia (ES) o Prototipo (PT)
Tiempo de respuesta	RT	Este parámetro especifica si el trabajo se centra en mejorar el tiempo de respuesta de las tareas enviadas. S significa Sí y N significa No.
Balanceo de carga	LB	Este parámetro especifica si la carga de trabajo está equilibrada en los nodos de almacenamiento. S significa Sí y N significa No.

Tabla 3.1: Descripción de las abreviaciones utilizadas para los parámetros de comparación.

Referencia	Año	ENV	ARQ	AVA	CMI	TY	RT	LB
[52]	2001	Grid	Multi-nivel jerárquico	S	N	SIM	S	N
[15]	2011	Grid	DAG	S	N	ES	S	N
[58]	2012	Cloud	Multi-nivel jerárquico	S	N	MO	S	S
[36]	2016	Cloud	Multi-nivel jerárquico	N	S	MO	N	N
[44]	2016	Cloud	Multi-nivel jerárquico	S	N	SIM	S	S
[28]	2019	Cloud	Multi-nivel jerárquico	N	S	MO	N	N
[56]	2020	Cloud	DAG	S	N	PT	S	S
[2]	2020	Cloud	Multi-nivel jerárquico	S	N	MO	N	N
[45]	2021	Cloud	Multi-nivel jerárquico	S	N	SIM	S	S

Tabla 3.2: Comparativa de diferentes sistemas de replicación de la literatura.

4

Método adaptativo-reactivo de replicación para sistemas de almacenamiento de alta disponibilidad

En esta sección se describe el método propuesto para dar solución a la problemática planteada. Se describen los componentes y módulos de la arquitectura del método propuesto. Además se proporcionan detalles de la implementación del método.

4.1 Glosario

Dado que en la descripción del método se emplean diversos conceptos importantes, a continuación se describen de manera breve estos conceptos clave.

- **Método de replicación:** Define una serie de etapas que al seguirse de forma sistematizada se consigue realizar la replicación de elementos. Los métodos de replicación abarcan la selección de la cantidad de réplicas (FR), la distribución y tónicas para la transferencia de réplicas.

- **Método de construcción:** Define una serie de etapas para la construcción de un sistema mediante un lenguaje de programación, cláusulas declarativas o archivos de configuración.
- **Replicación sistémica:** Consiste en la creación y despliegue de nodos, que son asignados a una máquina física de la cual también se le asignan recursos como almacenamiento en disco, memoria RAM y cantidad de núcleos.
- **Técnicas de replicación:** Conjunto de procedimientos que indican la forma en la que se lleva la transferencia de réplicas a los nodos de almacenamiento.
- **Replicación dinámica:** Cambia la cantidad de réplicas en tiempo de ejecución basado en cuáles puede cambiar con base en diversos criterios.
- **Replicación estática:** Mantiene una cantidad constante de réplicas durante el ciclo de vida del sistema de almacenamiento.
- **Factor de replicación:** Variable que indica la cantidad de réplicas (esferas) que son almacenadas de un contenido en un sistema de almacenamiento.
- **Elasticidad:** Es la capacidad de ajustar los recursos necesarios para hacer frente a las cargas de forma dinámica. De este modo, cuando la carga aumenta, se añade más recursos y, cuando la demanda disminuye, se reducen y se eliminan los recursos innecesarios.
- **Ancho de banda:** Es la capacidad máxima en el volumen de datos que puede fluir por el sistema.
- **Rendimiento:** La cantidad de datos que fluyen a través del sistema.
- **Recursos disponibles:** Cantidad de nodos que se encuentran disponibles para colocar réplicas.
- **Probabilidad de disponibilidad:** Define la probabilidad que se tiene para acceder a una réplica, la cual está en función de la cantidad de réplicas.

4. Método adaptativo-reactivo de replicación para sistemas de almacenamiento de alta disponibilidad

53

- **Alta disponibilidad:** Es la característica de los sistemas que tiene como objetivo asegurar un nivel de rendimiento operativo. La disponibilidad incluye la cantidad de tiempo que un sistema es accesible y cuánto tiempo necesita el sistema para responder una petición de un usuario.
- **Experiencia de usuario:** Cómo lo usuarios interactúan con el sistema y la experiencia que tienen al hacer uso del sistema.
- **Tiempo de servicio:** Cantidad de tiempo que le cuesta a un nodo procesar una operación de carga o descarga.
- **Tiempo de respuesta:** Cantidad de tiempo total que le cuesta al sistema procesar una operación de carga o descarga.
- **Tiempo de espera:** Cantidad de tiempo que las operaciones tienen que esperar para ser procesadas.
- **Adaptativo:** La habilidad de cambiar el comportamiento del sistema en base a un criterio dado para hacerle frente a los cambios que han acontecido en el sistema.
- **Reactivo:** La característica de responder ante cualquier situación que haya acontecido en el sistema.
- **Tiempo de retención:** Cantidad de tiempo que las réplicas tienen que esperar antes de ser publicadas para comenzar a ser consumidas.
- **Tiempo de llegada:** Estampa de tiempo que indica la llegada de una operación a una línea de espera.
- **Tiempo de salida:** Estampa de tiempo que indica la finalización de una operación en el sistema.
- **Tiempo entre operaciones:** La cantidad de tiempo transcurrido entre dos operaciones.

- **Balanceo de carga:** Es el proceso de la distribución de carga entre un conjunto de trabajadores (nodos).
- **Piscina de almacenamiento:** Conjunto de nodos de almacenamiento de un sistema de almacenamiento.
- **Cliente:** Componente que interactúa con el sistema de almacenamiento para realizar operaciones de carga y descarga.
- **Ortogonalidad:** En el contexto de los sistemas de almacenamiento, la distribución de réplicas es ortogonal cuando no existe una réplica almacenada dos o más veces en el mismo nodo.
- **Ráfaga:** Un conjunto de operaciones que llegan al sistema de manera agolpada, esto es con tiempos entre peticiones pequeños o con tiempos de llegada iguales.
- **Réplica:** Una esfera que hace referencia a un bloque, archivo u objeto, que se ha colocado en un nodo de almacenamiento a partir de la esfera original.

4.2 Descripción general

El método se contextualiza asumiendo el escenario de un sistema de almacenamiento en la nube, el cual está conformado por los componentes que se definen en la Sección 1.7. En el método se emplea la metáfora *balls-into-bin*, donde las *balls* (esferas) son los elementos de interés del usuario que se almacenan en forma de archivos, bloques u objetos; y b) los *bins* (cajas) que son los nodos de almacenamiento que alojan las esferas. Los *bins* son elementos que se replican con la finalidad de incrementar el ancho de banda para atender de mejor manera las cargas y se distribuyen en la infraestructura disponible. Las esferas (réplicas) son elementos que se replican para incrementar el ancho de banda para atender las descargas. Las decisiones para accionar la replicación son tomadas por el gestor de réplicas (*RM*). El método tiene como objetivo mejorar la disponibilidad de acceso

4. Método adaptativo-reactivo de replicación para sistemas de almacenamiento de alta disponibilidad

55

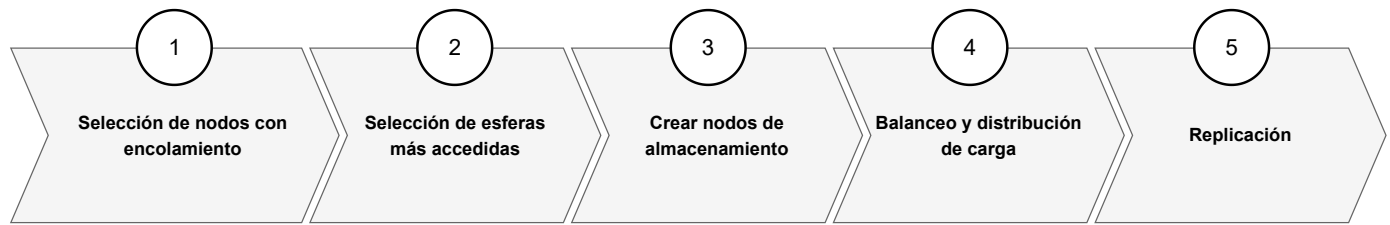


Figura 4.1: Etapas del método de replicación propuesto.

a datos empleando para ello *replicación a nivel de datos*, y a través de *replicación sistémica*, añadir la propiedad de elasticidad al sistema para agregar más nodos de almacenamiento y así adaptarse a la carga de trabajo (operaciones de carga y descarga). Utilizando estas operaciones de los usuarios, es posible ajustar el factor de replicación (FR), el cual indica la cantidad de réplicas (esferas) que deben crearse. Si $FR > RD$, siendo RD la cantidad de recursos disponibles, entonces se deben crear $FR - RD$ nodos de almacenamiento (cajas) nuevos, adaptando así un RD a un FR variable, lo cual dota al sistema de dinamismo para hacer frente a entornos de alta concurrencia y mejorar la disponibilidad de las esferas.

Para lograr lo anterior, el método se compone de cinco etapas graduales que se van ejecutando de manera secuencial, como se indicada en la Figura 4.1.

Etapa 1. Se seleccionan los nodos de almacenamiento (cajas) que experimentan encolamientos. Esta selección se realiza tomando todos los nodos de almacenamiento con tiempo promedio de espera mayor a la media total.

Etapa 2. Se seleccionan las réplicas (esferas) más accedidas en los nodos que experimentan encolamientos.

Etapa 3. Se crean $FR - RD$ nuevos nodos de almacenamiento (cajas), si y solo si el $FR > RD$ (se requieren nuevas esferas).

Etapa 4. Se lleva acabo el balanceo y distribución de las réplicas (esferas) y nodos (cajas) que han sido replicados.

Etapas 5. Se definen y ejecutan la creación de las réplicas (esferas) y su distribución en los nodos de almacenamiento (cajas) mediante un conjunto de cláusulas. Con estas cláusulas se describen las actividades a realizar en cada una de las etapas anteriores.

4.3 La replicación en un sistema de almacenamiento

La replicación es una tarea que se realiza en sistemas de almacenamiento con alta demanda de operaciones de carga y descarga. Lo que se busca es que los usuarios consumidores satisfagan sus necesidades de acceso a la información en el menor tiempo posible. En esta sección se describe todo el contexto, incluyendo las actividades y operaciones que se ejecutan para llevar a cabo replicación en un sistema de almacenamiento. De manera resumida las variables que se involucran en un sistema de replicación se describen en la Tabla 4.1.

Tabla 4.1: Variables involucradas en la definición de un sistema de replicación.

Nombre	Descripción
rs	Esquema de replicación
sp	Piscina de almacenamiento
$idSP$	Identificador de una piscina de almacenamiento
B	Conjunto de esferas
B'	Subconjunto de esferas
b	Esfera que hace referencia a un archivo, bloque u objetos.
idB	Identificador de una esfera
$sizeB$	Tamaño en bytes de una esfera
sn_i	Nodo de almacenamiento i
$idSN$	Identificador de un nodo de almacenamiento
$diskCap$	Capacidad total en disco
$memCap$	Capacidad total en memoria
$\bar{st}$	Tiempo de servicio promedio
$\bar{wt}$	Tiempo de espera promedio
$what$	Subconjunto de esferas que serán replicadas
who	Nodo de almacenamiento que inicia la replicación.
$where$	Subconjunto de nodos de almacenamiento en donde se colocan las réplicas
how	Tupla que expresa la técnica y la manera de transferencia de las réplicas
$when$	Expresa el tiempo en el que se debe de accionar la replicación
$SServ$	Conjunto de piscinas de almacenamiento
$SNodes$	Conjunto de nodos de almacenamiento
$SNodes'$	Subconjunto de nodos de almacenamiento
$QSNodes$	Conjunto de nodos de almacenamiento con encolamiento.

Para llevar acabo la replicación es necesario definir el servicio de almacenamiento, el cual se compone de varias piscinas de almacenamiento. Una piscina de almacenamiento (sp) es un conjunto de nodos de almacenamiento, que sigue un esquema de replicación. Es importante mencionar que para acceder a una tupla, como la ilustrada en la Ecuación 4.1, con $p_1(e)$ nos referimos al elemento x , con $p_2(e)$ al elemento y y así sucesivamente.

$$e = (x, y, z) \quad (4.1)$$

En el SA una esfera se denota como b , y es una tupla que hace referencia a un archivo, bloque u objeto, se define como indica la Ecuación 4.2. Cada nodo de almacenamiento (sn) tiene un subconjunto de esferas almacenadas (B') como se expresa en la Ecuación 4.3. La unión de todas las esferas almacenadas en los nodos de almacenamiento da como resultado el conjunto total de esferas almacenadas en el sistema (B), lo que se expresa con la Ecuación 4.4:

$$b = (idB, sizeB) \quad (4.2)$$

$$B'_i = \{b_1, b_2, \dots, b_n\} \quad (4.3)$$

$$B = \bigcup_{i=1}^n B'_i \quad (4.4)$$

donde idB_i es el identificador único de la esfera b_i , $sizeB_i$ es el tamaño de la esfera b_i , B' es un subconjunto de esferas en un nodo, y B es el conjunto global de esferas en el sistema de almacenamiento.

Un nodo de almacenamiento (sn , -caja-) se define mediante una tupla, como se muestra en la Ecuación 4.5. La tupla incluye un identificador único ($idSN$), capacidad en disco ($diskCap$) y memoria ($memoryCap$), tiempo promedio de servicio ($\bar{st}$) y tiempo promedio de espera ($\bar{wt}$), así como el conjunto de esferas almacenadas en el nodo (B').

$$sn = (idSN, diskCap, memoryCap, \bar{st}, \bar{wt}, \mathbf{B}') \quad (4.5)$$

La estructura lógica de una piscina de almacenamiento (sp) se define mediante una tupla como se muestra en la Ecuación 4.9, incluyendo un identificador único ($idSP$); un balanceador de carga (lb), como se define en la Ecuación 4.8 (los balanceadores disponibles se describen en la sub-sección 4.4.4.1); un conjunto de nodos de almacenamiento ($\mathbf{SNodes}'$) y un esquema de replicación (rs), el cual indicará cómo es que debe llevarse a cabo el proceso de replicación por defecto en la piscina de almacenamiento.

$$\mathbf{SNodes}' = \{sn_1, sn_2, \dots, sn_m\} \quad (4.6)$$

$$\mathbf{SNodes} = \bigcup_{i=0}^n \mathbf{SNodes}'_i \quad (4.7)$$

$$lb = Random \vee RoundRobin \vee TwoChoices \vee SortingUF \vee LeastHits \vee MinWT \quad (4.8)$$

$$sp = (idSP, lb, \mathbf{SNodes}'_k, rs) \quad (4.9)$$

donde rs hace referencia a un esquema de replicación y es definido en la Ecuación 4.42

Un servicio de almacenamiento ($\mathbf{SServ}$) se define, por tanto, como un conjunto de piscinas de almacenamiento:

$$\mathbf{SServ} = \{sp_1, sp_2, \dots, sp_n\} \quad (4.10)$$

Las definiciones antes mostradas son utilizadas en las etapas del método.

4.4 Etapas

En esta sección se describen a detalle cada una de las cinco etapas que conforman el método adaptativo-reactivo de replicación.

4.4.1 Etapa 1. Selección de nodos con encolamiento

Esta etapa se detectan los nodos de almacenamiento (sn) que están experimentando encolamiento debido a la alta concurrencia de peticiones de carga o descarga. Para ello es necesario identificar los nodos cuyo tiempo promedio de espera supere la media de los tiempos de espera de todos los nodos de almacenamiento. Para obtener el tiempo de espera promedio del nodo o caja ($WT(sn)$) se debe primero obtener el tiempo promedio de servicio del nodo o caja ($ST(sn)$), el cual se obtiene a través de la Ecuación 4.11:

$$ST(sn) = \frac{\sum_{i=1}^n St(i)}{n} \quad (4.11)$$

donde $St(i)$ es el tiempo de servicio de la operación i , este tiempo representa cuánto le ha tomado al nodo (caja) realizar la operación i y se calcula con la Ecuación 4.12.

$$St(i) = Et(i) - At(i) \quad (4.12)$$

donde $Et(i)$ representa el tiempo de salida o finalización de la operación i . Para calcular el tiempo de salida de las operaciones de carga y descarga, se utiliza la Ecuación 4.13:

$$Et(i) = \begin{cases} At(i) + (Wt(i) + ST(sn)) & \text{si } At(i) < Et(i-1) \\ At(i) + ST(sn) & \text{en otro caso} \end{cases} \quad (4.13)$$

El tiempo de salida se puede calcular de dos formas, dependiendo si $At(i) < Et(i - 1)$. Esto es, si el tiempo de arribo de la operación i es menor que el tiempo de la operación $i - 1$, significa que la operación i ha llegado mientras la operación $i - 1$ estaba siendo procesada. En este caso se añade el tiempo de espera que se calcula con la Ecuación 4.14. Es importante mencionar que se utiliza el tiempo de servicio promedio del nodo ($ST(sn)$) debido a que se desconoce el tiempo de servicio de las operaciones encoladas, porque éstas aún no han sido procesadas. Utilizando el tiempo de servicio promedio por nodo se puede obtener un estimado del tiempo de espera y con base a este tiempo de espera realizar acciones en el sistema. $At(i)$ representa el tiempo en el cual la operación se registró en el sistema y se conoce como tiempo de llegada o tiempo de arribo, el cual es una estampa de tiempo que se registra al momento de que la operación llega a la línea de espera.

Para calcular el tiempo de espera de cada operación ($Wt(i)$), se utiliza la Ecuación 4.14:

$$Wt(i) = \begin{cases} 0 & si \ i = 1 \\ 0 & si \ (i > 1) \wedge At(i) > Et(i - 1) \\ Et(i - 1) - At(i) & si \ (i > 1) \wedge (At(i) < Et(i - 1)) \end{cases} \quad (4.14)$$

(4.15)

Es importante observar que el tiempo de espera se calcula dependiendo de los 3 casos posibles siguientes:

- Caso 1: Cuando es la primera operación en cola, entonces el $Wt(1) = 0$
- Caso 2: Si $i > 1$ y el tiempo de llegada es mayor que el tiempo de salida de la operación $i - 1$, entonces $Wt(1) = 0$
- Caso 3: Si $i > 1$ y el tiempo de llegada es menor que el tiempo de salida de la operación $i - 1$ entonces $Wt(1) = Et(i - 1) - At(i)$

En lo anterior se aprecia que la primera operación no observará tiempo de espera, ni tampoco tendrán que esperar aquellas peticiones en donde se cumpla que el tiempo de llegada es mayor al tiempo de salida de la última operación. Sin embargo, las operaciones en donde el tiempo de llegada sea menor al tiempo de salida de la operación anterior experimentarán tiempos de espera.

Una vez que se obtiene el tiempo de servicio promedio ($ST(sn)$) para cada nodo de almacenamiento (caja) entonces se podrá obtener el tiempo promedio de espera para cada nodo o caja ($WT(sn)$) utilizando para ello las operaciones que se encuentran en ese preciso momento del calculo en la línea de espera. El tiempo de espera promedio se obtiene para cada uno de los nodos mediante la Ecuación 4.16.

$$WT(sn) = \frac{\sum_{i=1}^n Wt(i)}{n} \quad (4.16)$$

También se calcula el tiempo de espera global ($WTGlobal$), como se muestra en la Ecuación 4.17.

$$WTGlobal = \frac{\sum_{sn \in \text{SNodes}} WT(sn)}{|\text{SNodes}|} \quad (4.17)$$

donde **SNodes** es el conjunto de todos los nodos de almacenamiento (cajas), para los cuales se obtiene el promedio de tiempos de espera $WT(sn)$. Utilizando los tiempos de espera relativos a los nodos se obtiene el tiempo de espera global ($WTGlobal$).

Asi pues, para seleccionar los nodos de almacenamiento que están presentando problemas de encolamiento, es necesario observar los tiempos de espera de cada uno de los nodos de almacenamiento, apoyándonos de las Ecuación 4.16 para calcular el tiempo de espera promedio relativo al nodo (caja) y de la Ecuación 4.17 para conocer el tiempo de espera global. Todos aquellos nodos de almacenamiento que superen el tiempo de espera global serán nodos de almacenamiento que superan el tiempo de espera global. Esto se expresa con la Ecuación 4.18, la cual expresa al

conjunto de aquellos nodos de almacenamiento que superan la media global de tiempo de espera del sistema.

$$QSnodes = \{sn | sn \in SNodes \wedge WT(sn) > WTGlobal\} \quad (4.18)$$

4.4.2 Etapa 2. Selección de esferas más accedidas

Una vez que se han filtrado los nodos con tiempo de espera superior al tiempo de espera global ($WTGlobal$) es posible filtrar las esferas que han sido más accedidas. El criterio es seleccionar las esferas que han superado la media de accesos, entonces se seleccionan aquellas que sobrepasan la media de accesos global. Las esferas que sobrepasan la media global de accesos serán las principales causantes de los encolamientos al estar recibiendo mayor cantidad de accesos. El promedio de accesos por nodo de almacenamiento se define con la Ecuación 4.19.

$$AVGDownloads(sn) = \frac{\sum_{b \in p_6(sn)} downloads(b)}{|p_6(sn)|} \quad (4.19)$$

donde *downloads* es una función que recibe como parámetro la esfera *b* y obtiene la cantidad de descargas que le corresponden a dicha esfera. La esfera *b* debe pertenecer a las esferas que han sido colocadas en el nodo (caja) *sn*, así que $p_6(sn)$ hace referencia a la posición 6 de la tupla que define al nodo (caja), el valor de esta posición es el conjunto de esferas (B') que han sido colocadas en el nodo (caja).

Las esferas que superen la media del nodo de almacenamiento en donde se han colocado serán las esferas que deberán replicarse. El conjunto de esferas que cumple la condición se define con la Ecuación 4.20.

$$\mathbf{MD} = \{(b, sn) | (sn \in QSNodes) \wedge (b \in p_6(sn)) \wedge downloads(b) > AVGDownloads(sn)\} \quad (4.20)$$

Se creará una nueva réplica de las esferas que pertenezcan al conjunto **MD** adaptando el *FR* que se haya asignado previamente a la hora de la carga inicial de la esfera. Cuando una esfera es cargada en el sistema se le asigna un *FR* inicial, éste puede cambiar en tiempo de ejecución (*FR* dinámico). Este proceso toma en cuenta únicamente las descargas que han sido realizadas en el último intervalo de tiempo t , esto evita que se creen réplicas de más, sólo las justas para evitar una subutilización de la infraestructura de almacenamiento. Entonces observando el último intervalo de tiempo se pueden tomar únicamente los accesos más recientes a la esfera.

4.4.3 Etapa 3. Crear nodos de almacenamiento

En esta etapa se verifica que las esferas que superan la media de accesos puedan seguir replicándose en la piscina de almacenamiento, en los nodos (cajas) en los cuales no se han colocado réplicas manteniendo así la propiedad de la ortogonalidad. Esto es, si se cumple $FR < RD$, lo cual se interpreta como si la cantidad réplicas colocadas hasta el momento es menor que la cantidad de nodos (cajas), entonces es posible seguir colocando más réplicas mientras no se cumpla que $FR \geq RD$. En caso contrario, $FR \geq RD$, lo cual se interpreta como que ya no es posible seguir colocando réplicas debido a que esto no preservaría la característica de la ortogonalidad, perdiendo así el SA los beneficios de la replicación. Por eso, en este punto es cuando el sistema debe tomar la decisión de lanzar nuevos nodos de almacenamiento, la cantidad está determinada por $FR - RD$, lo cual dará como resultado la cantidad de nodos que necesita el sistema para poder satisfacer un *FR* dado. Esta es la característica de elasticidad, con base en los valores de *FR* se aumentan los nodos de almacenamiento del sistema con la finalidad de seguir colocando réplicas. A la replicación de nuevos

4. Método adaptativo-reactivo de replicación para sistemas de almacenamiento de alta disponibilidad

65

nodos (cajas) se le denomina **replicación sistémica**, la cual se ilustra de manera conceptual en la Figura 4.2.

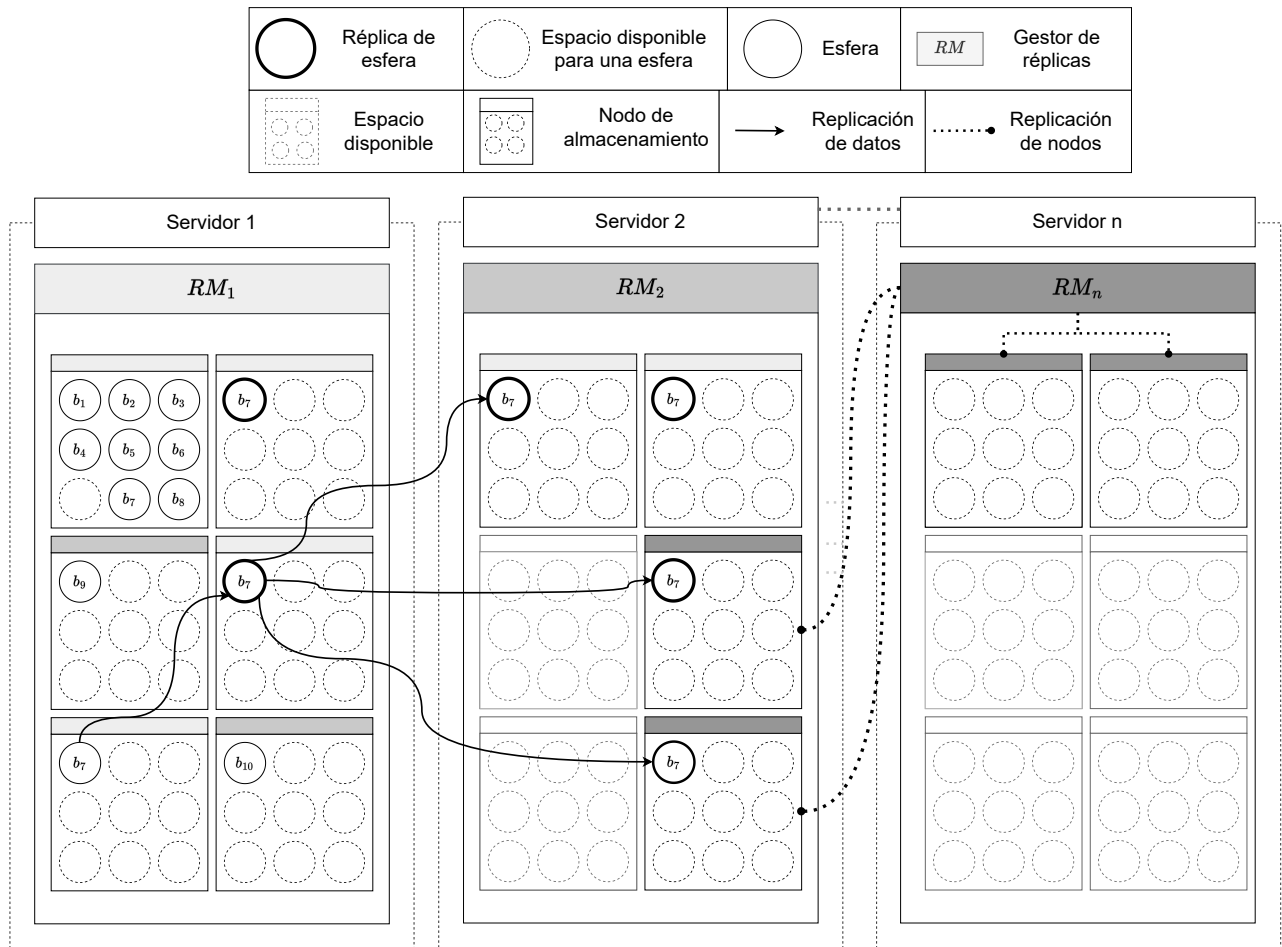


Figura 4.2: Representación conceptual de la replicación sistémica y replicación de datos.

Como se observa en la Figura 4.2, los gestores de réplicas (RM) se despliegan en un servidor, los nodos de almacenamiento que tienen que gestionar pueden estar en diferentes servidores. Al incrementar el número de nodos de una piscina de almacenamiento se incrementa el ancho de banda para las cargas y se incrementa el almacenamiento total del sistema.

4.4.4 Etapa 4. Balanceo y distribución de carga

Durante la operación del sistema de almacenamiento las esferas y las cajas deben ser balanceadas para otorgar una adecuada experiencia al usuario final y evitar la subutilización de réplicas. Las esferas necesitan ser balanceadas dependiendo del tipo de operación (carga o descarga). Los nodos necesitan ser balanceados con base en los recursos disponibles en los servidores de almacenamiento.

4.4.4.1. Balanceo de carga: Esferas

Las esferas que serán colocadas en un nodo tienen que preservar la propiedad de la ortogonalidad al momento de ser colocadas, utilizando la capacidad de almacenamiento del nodo como criterio para decidir cuál es el mejor nodo para colocar, y así utilizar la capacidad de almacenamiento de manera uniforme. Para balancear las operaciones de descarga, el conjunto de nodos disponibles se reduce a los nodos donde se han colocado réplicas. En lugar de utilizar la capacidad en disco se utiliza la capacidad en memoria principal. Los siguientes algoritmos de balanceo de carga son utilizados tanto para cargas como para descargas, respetando los criterios de balanceo del tipo de operación (carga/descarga):

- *Round-robin*: De manera cíclica distribuye las operaciones de carga. Este algoritmo no permite que el sistema active la elasticidad debido a que no permite integrar más nodos de almacenamiento.
- *Pseudo-random*: Se genera un número pseudo-aleatorio correspondiente a un índice en una lista de nodos. Este algoritmo permite que el sistema sea elástico permitiéndole agregar más nodos de almacenamiento.
- *Two-choices*: En este algoritmo se generan dos números aleatorios que corresponden a índices de una lista de nodos. Se selecciona aquel que tenga el menor *factor de utilización* UF ¹ [8].

¹El estado de los recursos de almacenamiento presentes en el sistema puede ser representado mediante su factor de utilización (UF), el cual proporciona un valor entre [0-1].

- *SortingUF*: Este algoritmo acomoda la lista de nodos de manera ascendente utilizando el *UF*. [8]
- *Min-WT*: Este algoritmo balancea la carga dependiendo del tiempo de espera promedio en cola de los nodos de almacenamiento.
- *Least-Hits*: Este algoritmo selecciona al nodo que contenga la réplica con menos descargas.

4.4.4.2. Balanceo de carga: Nodos

Los nodos de almacenamiento se balancean utilizando alguno de los siguientes algoritmos de balanceo de carga:

- *Round-robin*: Distribuye los nodos de manera cíclica sin tomar en cuenta ningún otro criterio.
- *Pseudo-random*: Distribuye los nodos de manera pseudo aleatoria, se genera un número aleatorio entre 0 y el número total de servidores físicos.

Para llevar acabo el balanceo, cada máquina física es etiquetada para que pueda ser utilizada en los algoritmos de balanceo. Estas etiquetas permiten gestionar los recursos utilizados de cada máquina física.

4.4.5 Etapa 5. Replicación

En esta etapa se realiza la replicación de las esferas en el sistema de almacenamiento. La replicación se realiza con base en algunos aspectos del sistema de almacenamiento que son importantes en el momento que es necesaria la replicación. Una técnica de replicación describe la forma en la cual las réplicas se transfieren a los nodos de almacenamiento (sn). Existen dos maneras para llevarla a acabo:

- *Replicación activa*: Todas las réplicas deben de ser escritas correcta y completamente en los nodos de almacenamiento para que puedan estar disponibles para su consumo por parte de los consumidores.
- *Replicación pasiva*: Las réplicas que se colocan con éxito en los nodos se hacen disponibles. Esto sucede de forma gradual hasta poner todas las réplicas a disposición de los consumidores.

El uso de una técnica de replicación tiene un costo asociado a las operaciones de escritura y lectura. Este costo depende de las tareas que sean necesarias en tales operaciones.

4.4.5.1. Replicación activa

En esta replicación el aspecto clave es que las FR réplicas deben de ser escritas correctamente para que éstas se pongan a disposición de los consumidores y ellos puedan iniciar la descarga del contenido publicado. La consistencia se garantiza y los encolamientos se reducen, pero la latencia observada por el usuario incrementa, esto debido al tiempo de retención (Tr). El tiempo de retención Tr expresa la cantidad de tiempo en la que las réplicas son retenidas antes de ponerse a disposición de los usuarios y éstos comiencen a descargar. Si la replicación activa se realiza de manera secuencial entonces puede ser expresada como indica la Ecuación 4.21.

$$Tr_{seq} = \sum_{i=1}^{FR} Rt(W(r_i(b_x))) \quad (4.21)$$

donde $W(x)$ denota una escritura de $r_i(bx)$, $r_i(bx)$ es la réplica i de la esfera bx , de la cual se obtiene su tiempo de respuesta mediante la función $Rt(x)$.

Si la escritura de las réplicas se realiza de forma paralela, el Tr se expresa como indica la Ecuación

4.23:

$$\mathbf{RepSet} = \{Rt(W(r_1(bx))), Rt(W(r_2(bx))), \dots, Rt(W(r_{FR}(bx)))\} \quad (4.22)$$

$$Tr_{par} = \max(\mathbf{RepSet}) \quad (4.23)$$

donde $\mathbf{RepSet}$ es el conjunto de los tiempos de respuesta de las escrituras de las FR réplicas, de este conjunto se extraer el mayor tiempo de respuesta utilizando la función $\max$, que recibe como parámetro un conjunto del cual extrae el mayor elemento.

Costo de escritura

En la replicación activa cada petición del usuario consumidor es procesada por todos los nodos de almacenamiento de forma determinista, de tal manera que todos los servidores producirán la misma respuesta y terminarán en el mismo estado final. La Figura 4.3 muestra un diagrama con todas las operaciones realizadas por los componentes que interfieren para llevar acabo una replicación activa con un $FR = 3$ y así conseguir una probabilidad de disponibilidad (PA). En esta Figura se observa un ejemplo de una operación de carga en un sistema de almacenamiento que tiene un $RD = 3$, lo cual significa que tiene 3 nodos de almacenamiento, y tiene configurado un $FR = 3$, lo que significa que el sistema realizará 3 réplicas y serán distribuidas en los RD nodos de almacenamiento pertenecientes a la piscina de almacenamiento. El flujo de la operación inicia desde el usuario, quien realiza la operación de escritura de manera local. Después seguido se realiza la operación de balanceo de carga para seleccionar los nodos de almacenamiento donde se colocarán las FR réplicas. Después se procede a transferir las réplicas a los nodos, lo cual se puede realizar en modo *secuencial* o en *paralelo*. Con la replicación activa se consigue un probabilidad de disponibilidad $PA = \frac{FR}{RD}$, en el ejemplo que se ilustra en la Figura se intenta conseguir un $PA = 1$.

El costo de una carga utilizando la replicación activa está dada por la Ecuación 4.26, y está conformada por el costo de balanceo de carga (Ecuación 4.24) y el costo de cada una de las réplicas

puede ser determinada, debido a que depende de factores externos que no pueden ser controlados o configurados en el sistema de almacenamiento. El mensaje para solicitar el balanceo de carga y el mensaje de respuesta son afectados por la latencia, y ésta dependerá del tiempo que les tome a los mensajes llegar a su destino.

Después de realizar el balanceo de carga éste dará como resultado los nodos que se han seleccionado para almacenar las réplicas. El costo de escritura de una réplica se expresa con la Ecuación 4.25:

$$Rt(W(r_i(bx))) = St(W(r_i(bx))) + Push(r_i(bx)) + \lambda(ACK(r_i(bx))) \quad (4.25)$$

donde $St(W(r_i(bx)))$ hace referencia al tiempo de servicio de la escritura de la réplica i (r_i) de la esfera b_x . Este tiempo de servicio está en función del tamaño de la esfera, esto es, entre mayor sea el tamaño de la esfera también mayor será el tiempo de servicio debido a que le costará más realizar el trabajo de escritura. $Push(r_i(bx))$ hace referencia a la transferencia de la réplica i (r_i) de la esfera bx , y esté en función del tamaño de la esfera bx . Por el último, la respuesta de la escritura de la réplica i (r_i) de la esfera bx se denota como $\lambda(ACK(r_i(bx)))$, la cual está afectada por la latencia (λ).

El costo de escritura de la técnica de replicación activa se puede realizar de 2 modos: *secuencial* o *paralelo*. Es por eso que la ecuación del costo de escritura se divide en 2 ecuaciones, una para cada modo. El costo de la escritura de un contenido utilizando el modo *secuencial* está dado por la Ecuación 4.26:

$$Rt(W(bx)) = St(WLocal(bx)) + Rt(LBW(bx)) + St(R(bx)) + Tr_{seq} \quad (4.26)$$

donde $St(WLocal(bx))$ es el tiempo de servicio de escritura local, operación que sucede en el cliente (usuario consumidor). También se suma el tiempo de balanceo de carga ($Rt(LBW(bx))$), definido

en la Ecuación 4.24. Para poder transferir las réplicas es necesario realizar una lectura, la cual se denota como $St(R(bx))$. Por último se suma el tiempo de retención para el modo secuencial (Tr_{seq}), expresado en la Ecuación 4.21

Esta técnica puede también escribir las réplicas de forma paralela con un cambio en la fórmula que expresa el tiempo de retención, como se observa en la Ecuación 4.27

$$Rt(W(bx)) = St(WLocal(bx)) + Rt(LBW(bx)) + St(R(bx)) + Tr_{par} \quad (4.27)$$

donde Tr_{par} , que se ha definido en la Ecuación 4.23, hace referencia al tiempo de retención de manera paralela, donde únicamente se toma el tiempo máximo de escritura de las réplicas. **Costo de lectura**

De la misma manera que la escritura de la esfera (b_x) en la replicación activa, en la lectura también se tienen 2 modos. La lectura dependerá del modo que se haya configurado en el sistema, esto es, si las escrituras se realizan en modo secuencial entonces las lecturas también serán secuenciales. El costo de una lectura utilizando el modo secuencial se expresa en la Ecuación 4.28.

$$Rt(R(bx)) = Rt(LBR(bx)) + St(R(bx)) + Tr_{seq} + \lambda(ACK) \quad (4.28)$$

donde $Rt(LBR(bx))$ es el balanceo de carga para realizar una lectura, el cual es definido de manera similar al de escritura definido en la Ecuación 4.24. La lectura de la esfera bx implica el tiempo de servicio de lectura en el nodo de almacenamiento seleccionado por el balanceo de carga y se expresa como $St(R(bx))$. Todas las lecturas son afectadas por el tiempo de retención (Tr_{seq}) debido a que permanecerán encoladas hasta que todas las réplicas se hayan escrito de manera exitosa; en el modo secuencial se suma el tiempo de retención expresado en la Ecuación 4.21. Por último, se suma la

respuesta con el contenido ACK , que es afectada por latencia (λ), que está en función del tamaño del contenido.

En el modo de lectura paralelo se realiza un cambio en la Ecuación que expresa el costo del tiempo de retención. En lugar de esperar a que terminen de ser escritas todas las réplicas, únicamente se esperará el tiempo de escritura máximo como se expresa en la Ecuación 4.29.

$$Rt(R(f_x)) = Rt(LBR(b_x)) + St(R(b_x)) + Tr_{par} + \lambda(ACK) \quad (4.29)$$

4.4.5.2. Replicación pasiva

En la técnica de replicación pasiva la publicación de las réplicas se realiza de manera gradual, lo cual reduce el tiempo de respuesta que observa el usuario. En consecuencia, esto ocasionará encolamientos en las descargas en caso de que la demanda sea alta para el contenido publicado. En la Figura 4.4 se muestran todas las operaciones realizadas por los componentes del sistema de almacenamiento para alcanzar una $PA = 1$ de manera paulatina e incrementándola gradualmente. El flujo de operación comienza en el cliente (usuario consumidor) realizando 1) una escritura local; 2) seguido de una operación al gestor de réplicas (RM) para que realice el balanceo de carga y dé como respuesta los nodos de almacenamiento en donde se van a colocar las FR réplicas y que serán distribuidas en los RD nodos de la piscina de almacenamiento; 3) se realiza la transferencia de las réplicas a los nodos. A diferencia de la replicación activa, la replicación pasiva consigue una probabilidad de disponibilidad (PA) de manera gradual, como se puede apreciar en la Figura, mientras se van terminando de escribir las réplicas la PA se va incrementando en $1/RD$. **Costo de escritura**

El costo de una escritura haciendo uso de la replicación pasiva, al igual que la replicación activa, es desglosada en 2 partes para después sumarmas y obtener el costo total de una escritura. A diferencia de la replicación activa, la replicación pasiva no tiene que esperar a que todas las réplicas terminen de ser escritas, éstas estarán disponibles para consumirse conforme vayan terminando de escribirse

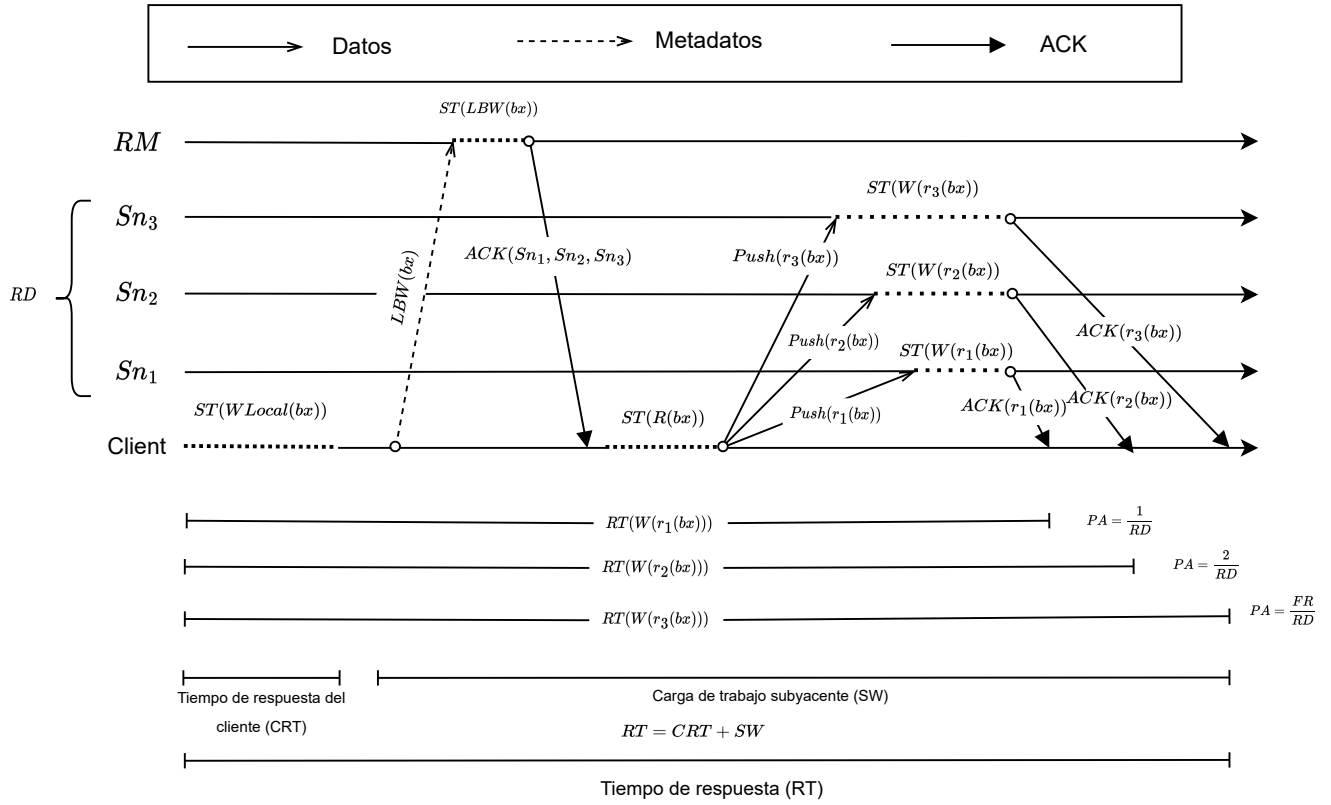


Figura 4.4: Esquema de replicación pasiva: interacción entre cliente y nodos de almacenamiento(sn).

en los nodos de almacenamiento que se seleccionaron para ponerlas. La Ecuación 4.30 expresa el costo de escribir la primera réplica, las escrituras posteriores no afectarán en el costo debido a que gradualmente irán terminando y poniéndose a disposición de los consumidores.

$$Rt(W(r_1(bx))) = St(W(r_1(bx))) + Push(r_1(bx)) + \lambda(ACK(r_1(bx))) \quad (4.30)$$

En la Ecuación 4.30 se observa el costo de realizar la escritura de la primera réplica, en donde $St(W(r_1(bx)))$ hace referencia al tiempo de servicio de la escritura de la primera réplica r_1 de la esfera bx . En esta Ecuación también se contabiliza el tiempo de transferencia de r_1 de la esfera bx ($Push(r_1(bx))$), esta transferencia está en función del tamaño del contenido. Por último se suma la latencia del mensaje de respuesta de la escritura de r_1 ($\lambda(ACK(r_1(bx)))$).

El costo de una escritura, entonces, es la suma de todas las operaciones que se realizan. Esto se

expresa en la Ecuación 4.31:

$$Rt(W(bx)) = St(WLocal(bx)) + Rt(LBW(bx)) + St(R(bx)) + Rt(W(r_1(bx))) \quad (4.31)$$

donde $St(WLocal(bx))$ hace referencia al tiempo de servicio que le toma al cliente escribir de manera local la esfera bx . A esto se le suma el tiempo que le toma al gestor tomar la decisión de los nodos de almacenamiento en donde se van a colocar las réplicas ($Rt(LBW(bx))$), lo cual se definió en la Ecuación 4.24. A esto también se le suma el tiempo de servicio de la lectura de la esfera bx ($St(R(bx))$). Por último se suma el tiempo de respuesta de la escritura de r_1 ($Rt(W(r_1(bx)))$), como se expresa en la Ecuación 4.30.

Costo de lectura

El costo de lectura de un contenido utilizando la replicación pasiva se expresa con la Ecuación 4.32.

$$Rt(R(b_x)) = Rt(LBW(b_x)) + Rt(W(r_1(b_x))) + St(R(b_x)) \quad (4.32)$$

donde $Rt(LBW(b_x))$ hace referencia al tiempo de respuesta que le toma al gestor réplicas (RM) tomar la decisión para seleccionar un nodo de almacenamiento para realizar la descarga. A esto se le suma el tiempo de respuesta de la escritura de la primera réplica (r_1) de la esfera bx ($Rt(W(r_1(bx)))$). Por último se añade el tiempo de servicio de la lectura ($St(R(bx))$), que es el tiempo que le toma al nodo de almacenamiento (seleccionado por el balanceador de carga) para devolver el contenido(esfera).

4.4.5.3. *Replicación como Código*

En esta sección se explica cómo se materializan las replications *activa* y *pasiva* antes explicadas. Estas replications se llevan a cabo mediante código (*Replicación como Código*). Para ello se emplea un conjunto de cláusulas declarativas que permiten expresar el esquema de replicación deseado. El conjunto de cláusulas corresponde a un conjunto de preguntas que contextualizan el flujo en un sistema de almacenamiento que requiere replicación de datos.

1. *¿Qué archivos se van a replicar?* Se eligen los archivos que deben de ser replicados. Éstos pueden ser seleccionados manualmente por el usuario o puede provenir de la salida de alguna aplicación que haya seleccionado los archivos con base en algún criterio, por ejemplo seleccionado los archivos más accedidos.
2. *¿Quién inicia la replicación?* Se selecciona el nodo de almacenamiento o los nodos de almacenamiento que pueden comenzar la replicación. Es necesario que los nodos que inician la replicación contengan en memoria o disco los archivos que se van a replicar.
3. *¿Dónde se deben colocar las nuevas réplicas?* Se seleccionan los nodos en donde se colocarán las réplicas. Es importante que las réplicas preserven la característica de ortogonalidad. Normalmente la respuesta que se le da a esta pregunta es utilizando un balanceador de carga.
4. *¿Cómo debe efectuarse la replicación?* La transferencia de las réplicas hacia los nodos de almacenamiento se puede efectuar de 2 maneras: *activa* o *pasiva*, pero en el método propuesto se plantean 2 técnicas distintas para efectuar la replicación: *activa-elástica* y *pasiva-elástica*.
5. *¿Cuándo debe accionarse la replicación?* La replicación puede accionarse de forma 1) *reactiva*, lo cual hace referencia a efectuar la replicación en el momento de la carga de un archivo; 2) *diferida*, la cual realiza la replicación en segundo plano y puede retenerse hasta que se cumpla algún criterio; 3) *proactiva*, la replicación se realiza de manera anticipada, antes de que lleguen las operaciones de descarga.

4. Método adaptativo-reactivo de replicación para sistemas de almacenamiento de alta disponibilidad

77

En el método propuesto, la materialización de alguno de los esquemas de replicación descritos anteriormente se define mediante una estructura 5-tupla, la cual captura el contexto de la disponibilidad de un servicio de almacenamiento ($SServ$). La definición de este esquema se basa en un conjunto de cláusulas que modelan el contexto de cómo se realizará la replicación. El diseñador del esquema de replicación da respuestas a preguntas genéricas tales como *¿qué?*, *¿quién?*, *¿dónde?*, *¿cómo?* y *¿cuándo?* hacer la replicación. Se espera que estas preguntas no solo sean respondidas por un diseñador mediante código sino que también se espera que puedan ser respondidas por *bots* (aplicaciones o servicios de software).

Para evitar ambigüedades en las respuestas, a continuación se definen las expresiones con las que se puede dar respuesta a las preguntas antes mencionadas. Cabe recordar que una replicación se compone de dos operaciones básicas: 1) replicación de datos o nodos, definida mediante la creación de FR copias exactas de una esfera b ; y 2) distribución de las FR réplicas en los nodos de almacenamiento.

La pregunta *¿qué?* permite al sistema de replicación realizar la generación de réplicas, lo cual se hace en dos fases. Primero se pregunta por las esferas que se van a tomar en cuenta para generar las réplicas; esta pregunta se debe interpretar de la siguiente forma: *¿qué datos se van a replicar?*, su respuesta se expresaría como indica la Ecuación 4.33.

$$what = \{b_0, b_1, \dots, b_n\} | what \subset \mathbf{B} \quad (4.33)$$

recordar que $\mathbf{B}$ es el conjunto global de esferas en el sistema de almacenamiento.

Para completar la generación de réplicas en todo el sistema de almacenamiento se agrega a la respuesta el parámetro FR , lo cual resulta en la Ecuación 4.34:

$$\forall b \in what, \sum_{b \in what} FR * p_2(b) \quad (4.34)$$

Como se puede ver, al completar esta respuesta, el sistema de replicación puede calcular el costo de cada operación de replicación ($FR * p_2(b)$) en términos de almacenamiento y de transferencia. Es importante responder a esta pregunta en dos fases para que cada esfera pueda recibir un FR distinto, si es requerido, lo cual dota al sistema de replicación de dinamismo, lo que resulta clave en tiempo de ejecución. En el caso de que un *bot* sea quien responde a esta pregunta, el criterio que se utilizaría para determinar los datos a replicar sería la media aritmética de la cantidad de accesos al dato (esfera) de interés, aunque este criterio puede ser determinado por el diseñador del *bot*. Como resultado, aquellos datos cuya media de accesos sea superior a la media serían candidatos a replicarse automáticamente. Es importante seleccionar los datos que se replican mediante un criterio estadístico de accesos debido a que una elección de datos con bajo ratio de accesos podría representar una utilización innecesaria de recursos de almacenamiento, lo cual afecta directamente a la segunda operación de replicación: el transporte de las réplicas.

La pregunta *¿quién?* permite al sistema de replicación habilitar a cualquier nodo a comenzar el proceso de generación de réplicas. Esta pregunta se plantea de la siguiente forma: *¿quién inicia la replicación?*. Por tanto *¿quién?* se refiere al nodo encargado de comenzar la replicación, al cual se le conoce como *nodo pivote*, debe tener almacenados los datos a replicar, lo cual se expresa en la Ecuación 4.35.

$$who = sn | what \subset p_6(sn) \quad (4.35)$$

$$(4.36)$$

donde $p_6(sn)$ hace referencia al elemento 6 del vector sn , que es el conjunto B' , el que contiene las esferas que están almacenadas en el nodo (caja) denotado como sn . Si el nodo no contiene alguna réplica de los datos no puede formar parte de los nodos de almacenamiento que inician la replicación.

La pregunta *¿dónde?* se refiere a la segunda operación de un proceso de replicación, la cual tiene que ver con el transporte y escritura de las réplicas en un nodo distinto al que contiene el

4. Método adaptativo-reactivo de replicación para sistemas de almacenamiento de alta disponibilidad

79

dato (esfera) original y distinto a cualquier nodo que ya contenga a una esfera replicada. Esto para preservar la ortogonalidad de la distribución de réplicas, lo cual resulta clave para mantener la eficacia del sistema de replicación. Por tanto, la pregunta se formula de la siguiente manera: *¿dónde se deben colocar las nuevas réplicas?*, la cual se expresa con la Ecuación 4.37.

$$where = \{sn_1, sn_2, \dots, sn_{FR}\} | \forall sn \in where, [what \not\subset p_6(sn)] \quad (4.37)$$

La respuesta a esta pregunta se espera sea dada por alguno de los algoritmos de balanceo de carga, incorporados al método de replicación.

La pregunta *¿cómo?* hace referencia a la técnica de replicación y al tipo de transferencia de las réplicas, formulándose de la siguiente manera *¿cómo debe efectuarse la replicación?*, la cual se expresa como indica la Ecuación 4.38.

$$how = (rt, tt) \quad (4.38)$$

$$rt = ACTIVE \vee PASSIVE \vee ACTIVE_ELASTIC \vee PASSIVE_ELASTIC \quad (4.39)$$

$$tt = PUSH \vee PULL \quad (4.40)$$

La pregunta *¿cuándo?* tiene como finalidad que los diseñadores y/o *bots* definan la política de ejecución de la generación y distribución de réplicas en el sistema de almacenamiento. Por tanto, la pregunta se formula de la siguiente forma: *¿cuándo debe de accionarse la replicación?*. Para responder se debe hacer referencia a la manera de cómo se debe accionar la replicación: *reactiva*, *diferida*, *proactiva*, como se indica en la Ecuación 4.41. Por ejemplo, dos posibles respuestas a esta pregunta podrían ser: *REACTIVE*, que básicamente significa que la replicación debe de realizarse en ese preciso momento, o *DEFERRED*, que se interpreta como la realización de la replicación en

segundo plano.

$$when = REACTIVE \vee DEFERRED \vee PROACTIVE \quad (4.41)$$

Así pues, respondiendo a las cinco preguntas anteriores se puede indicar cómo debe materializarse el *esquema de replicación* (rs). En consecuencia, un esquema de replicación se declara como una 5-tupla que contiene las respuestas a las preguntas (Ecuación 4.42).

$$rs = (answerWho, answerWhat, answerWhere, answerHow, answerWhen) \quad (4.42)$$

donde todos los elementos que conforman a un esquema de replicación (rs) son respuestas a las 5 preguntas que se han definido para llevar acabo el proceso de la generación de replicas.

4.5 Implementación

Para observar el desempeño del método propuesto se implementó un prototipo funcional. En esta sección se dan los pormenores de la implementación realizada. Se describe la plataforma empleada así como los elementos que constituyen al prototipo.

4.5.1 Arquitectura del prototipo

El prototipo desarrollado consta de varios componentes y módulos. Un componente es una parte que constituye el sistema y son desplegados en forma de contenedores virtuales. Los módulos forman parte de los componentes, son piezas de software que se encargan de realizar una tarea específica.

4.5.1.1. Componentes

La arquitectura está conformada por los tres componentes descritos a continuación:

4. Método adaptativo-reactivo de replicación para sistemas de almacenamiento de alta disponibilidad

81

- **Cliente:** Componente que permite la interacción con el sistema de almacenamiento facilitando la realización de operaciones de carga y descarga.
- **Gestor de réplicas (RM):** Componente que se encarga gestionar las réplicas. También de balancear las operaciones en los nodos de almacenamiento pertenecientes a una piscina de almacenamiento y analiza los tiempos de espera (Wt) para accionar la replicación de datos o sistémica.
- **Nodo de almacenamiento (sn):** Los nodos de almacenamiento son agrupados para formar piscinas de almacenamiento, las cuales son gestionadas por el RM . Se encargan de procesar las operaciones de carga y descarga.

4.5.1.2. Módulos

Cada componente está formado por módulos, los cuales realizan tareas específicas de acuerdo a su nivel y al componente al que pertenecen. En la Figura 4.5 se muestra la pila de módulos correspondiente al gestor de réplicas.

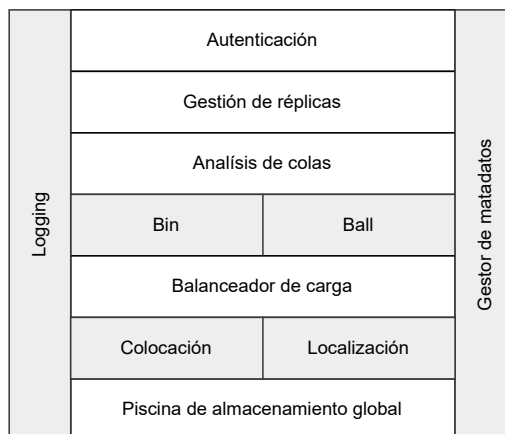


Figura 4.5: Módulos del gestor de réplicas.

A continuación se da una breve descripción de cada módulo del gestor de réplicas.

- **Autenticación:** Módulo encargado de llevar a acabo la autenticación basada en tokens para permitir el acceso al sistema y realizar operaciones de carga y descarga.
- **Gestión de réplicas:** Este módulo lleva la gestión de las réplicas creadas, almacenando metadatos necesarios para su correcta localización y interpretación a la hora de ser descargadas.
- **Análisis de colas:** Módulo encargado de obtener los tiempos relacionados a las colas de espera.
- **Balanceador de carga:** Módulo que permite el balanceo de cargas y descargas a los nodos preservando la propiedad de ortogonalidad. El balanceador también es capaz de balancear nuevos nodos de almacenamiento en máquinas que pertenecen a un *cluster* de servidores de almacenamiento.
- **Piscina de almacenamiento global:** Grupo de piscinas de almacenamiento, las cuales contienen nodos de almacenamiento. Una piscina de almacenamiento es un conjunto de nodos de almacenamiento.
- **Logging:** Módulo que registra todos los eventos ocurridos en el gestor de réplicas, para su posterior análisis.
- **Gestor de metadatos:** Módulo que lleva el control de los metadatos de las réplicas y de los nodos de almacenamiento. Esto permite a los otros módulos consumir los metadatos necesarios para llevar a cabo su función principal.

Las réplicas se crean siguiendo un método de replicación, en el cual influyen factores como el esquema de replicación, la cantidad de réplicas (determinada por el FR) y la selección de los nodos que almacenarán a las réplicas.

4.5.2 Construcción del sistema de replicación

A continuación se describe el método de construcción del sistema de replicación. Éste consta de 5 fases que se siguen de manera secuencial, como se muestra en la Figura 4.6, y las cuales se describen a continuación.

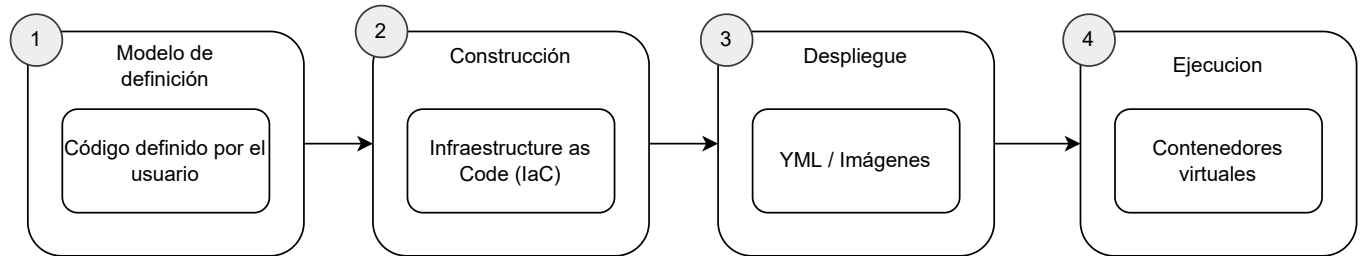


Figura 4.6: Diagrama de los pasos para la construcción del sistema de replicación con una configuración declarada.

4.5.2.1. Modelo contextual de definición

La implementación de un sistema de replicación de acuerdo al método propuesto se realiza siguiendo un modelo contextual de definición. Mediante este modelo es posible definir todo el contexto del sistema de almacenamiento en el que se pone el sistema de replicación. El modelo permite definir los componentes que interactúan entre sí, los cuales se declaran como objetos que intercambian mensajes para construir una piscina de almacenamiento dentro del sistema de almacenamiento.

Se definieron cinco clases siguiendo este modelo contextual de programación para definir los componentes del sistema de replicación, y otras seis clases que expresan atributos más complejos de los componentes. El diagrama de clases (Figura 4.7) muestra la jerarquía de las clases, los atributos y métodos de los componentes.

Las clases que expresan los atributos complejos de los servicios se describen a continuación:

- **Enviroments:** Esta clase permite expresar las variables de entorno del sistema operativo en donde se esté ejecutando el servicio. Contiene un método llamado *toOsEnv* que convierte el

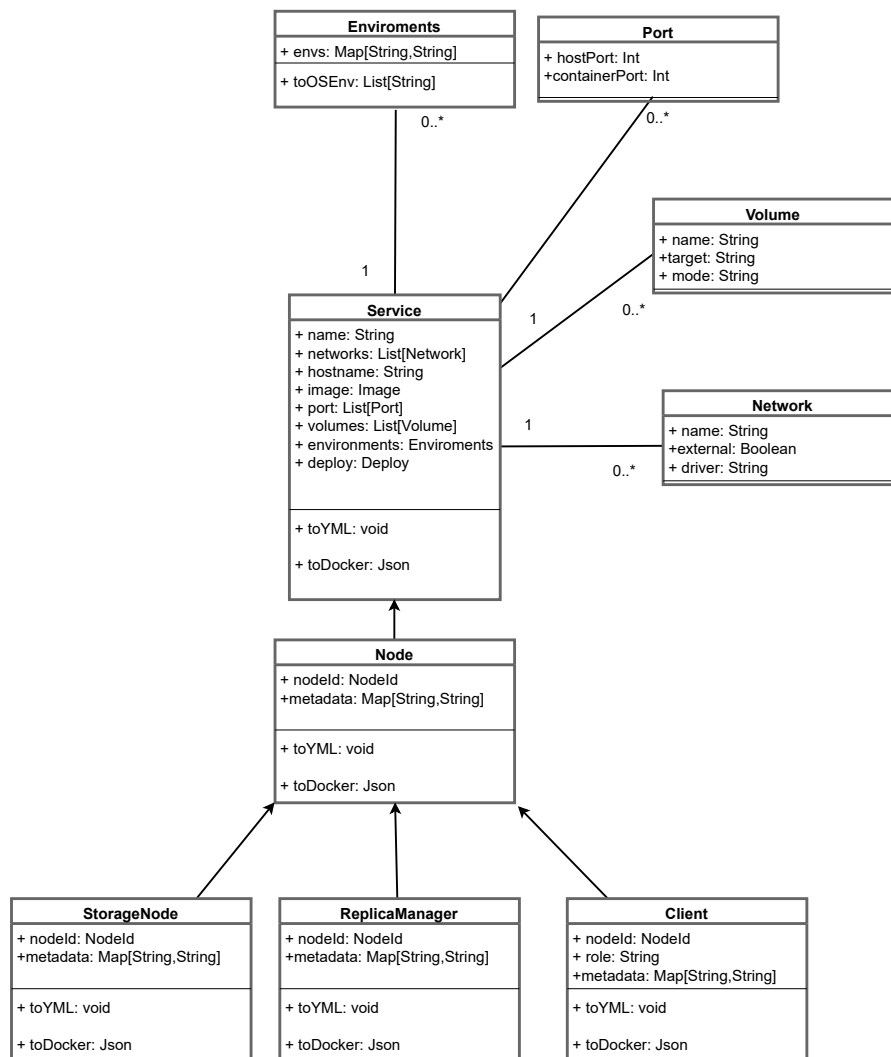


Figura 4.7: Diagrama de clases de los nodos.

mapa de variables de entorno en una lista con el formato requerido por el sistema operativo.

Por ejemplo $TEST- > 1$ tiene que ser trasladado a $TEST = 1$.

- **Port:** Esta clase permite expresar la relación de puertos de la máquina física host al contenedor².

- **Volume:** Esta clase permite expresar los volúmenes de almacenamiento, hay 3 tipos de

²El mapeo de puertos permite asignar un puerto de la máquina en donde reside el contenedor virtual a uno dentro del sistema operativo que se instaló en el contenedor

volúmenes;

- *Bind mount*: Este mecanismo permite asignar una ruta de la máquina host a una ruta dentro del contenedor.
 - *Tmpfs mount*: Es un mecanismo que permite la persistencia de datos en memoria principal.
 - *Volumes*: Es el mecanismo preferido para la persistencia de los datos generados y utilizados por los contenedores Docker.
- **Network**: Esta clase permite declarar redes con las cuales los servicios pueden comunicarse, se permiten distintos controladores de redes:
- *bridge*: El controlador de red por defecto. Si no se especifica un controlador, este es el tipo de red que está creando. Las redes puente se utilizan normalmente cuando sus aplicaciones se ejecutan en contenedores independientes que necesitan comunicarse.
 - *host*: En el caso de los contenedores autónomos, elimina el aislamiento de red entre el contenedor y el host Docker y utiliza directamente la red del host.
 - *overlay*: Las redes superpuestas conectan varios demonios Docker entre sí y permiten que los servicios de enjambre se comuniquen entre sí. También puede utilizar las redes superpuestas para facilitar la comunicación entre un servicio desplegado con Docker Swarm y un contenedor independiente, o entre dos contenedores independientes en diferentes demonios de Docker.
 - *ipvlan*: Las redes IPvLAN ofrecen a los usuarios un control total sobre el direccionamiento IPv4 e IPv6. El controlador de VLAN se basa en esto al ofrecer a los operadores un control completo del etiquetado de VLAN de capa 2, e incluso del enrutamiento L3 de IPvlan para los usuarios interesados en la integración de redes subyacentes.

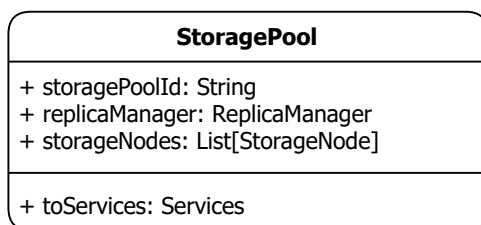


Figura 4.8: Atributos de una piscina de almacenamiento.

- *macvlan*: Las redes *macvlan* permiten asignar una dirección MAC a un contenedor, haciéndolo aparecer como un dispositivo físico en su red. El demonio Docker enruta el tráfico a los contenedores por sus direcciones MAC. El uso del controlador *macvlan* es a veces la mejor opción cuando se trata de aplicaciones heredadas que esperan estar directamente conectadas a la red física, en lugar de ser enrutadas a través de la pila de red del host Docker.
 - *none*: Normalmente se utiliza junto con un controlador de red personalizado. *none* no está disponible para los servicios de desplegados con Docker Swarm.
- **Deploy**: Esta clase permite expresar los recursos y la colocación de los nodos. Su definición de atributos y métodos se puede ver en la Figura 4.9

Cuando se crean las instancias del gestor de réplicas y los nodos de almacenamiento, éstos se agrupan en una piscina de almacenamiento (*sp*), los atributos de *sp* se muestran en la Figura 4.8.

Una entidad *StoragePool* tiene como atributos un identificador único, un gestor de réplicas y una lista de nodos de almacenamiento. Esto último es opcional debido a que también se puede asignar la cantidad de nodos de almacenamiento a una de las variables de entorno del sistema operativo donde se encuentra en el contenedor virtual. El gestor de réplicas se encargará de lanzarlos con una configuración por defecto. El método *toServices* interpreta lo que se declare en un archivo de configuración para generar las instancias de los nodos de almacenamiento y el gestor de réplicas. Para ello *toServices* recibe un archivo de configuración JSON. A continuación se muestra un fragmento del archivo de configuración que indica cómo se declara el ambiente del gestor de réplicas.

4. Método adaptativo-reactivo de replicación para sistemas de almacenamiento de alta disponibilidad

87

```
rmEnviroments = Environments(  
  "UPLOAD_LOAD_BALANCER" -> "ROUND_ROBIN",  
  "DOWNLOAD_LOAD_BALANCER" -> "ROUND_ROBIN",  
  "MAX_CONNECTIONS" -> "100000",  
  "BUFFER_SIZE" -> "65536",  
  "RESPONSE_HEADER_TIMEOUT_MS" -> "390000",  
  "API_VERSION" -> "2",  
  "LOG_PATH" -> "/app/logs"  
)
```

En esta definición se declaran las variables de entorno del sistema operativo, las cuales se implementan como un mapa o diccionario. Los valores de estas variables son los que se utilizan para configurar el gestor de réplicas. En este archivo de configuración también se deben de declarar las características de hardware que utilizará el gestor de réplicas y que, en la segunda fase de construcción, se gestionarán específicamente para los nodos de almacenamiento, como se indica a continuación.

```
rmDeploy = Resources(limits = Limits(cpus = "1", memory = "5G"))  
rmDeploy = Deploy(placement = None, resources = rmDeploy)
```

Cuando se requiera usar la configuración por defecto, se utiliza la palabra reservada *None* para hacer referencia que ese atributo estará vacío. El intérprete es capaz de detectarlos para después rellenarlos con la configuración por defecto.

Es importante también declarar los puertos que se expondrán y serán utilizados por los nodos, como se muestra a continuación:

```
defaultPort = Port(hostPort = 3000, containerPort = 3000)  
rmPorts = Ports(ports = List(defaultPort))
```

Todos los nodos deben conectarse a una red para poder comunicarse. Pueden conectarse diferentes redes, un ejemplo de declaración de una red se muestra a continuación:

```
defaultNet = Network(name = "my-net",external =true,driver="overlay")  
rmNets = Networks(networks = List(defaultNet))
```

Los nodos también pueden tener volúmenes en donde pueden escribir o leer archivos, siendo este un atributo requerido. Se declaran como se muestra a continuación:

```
logVol = Volume(hostPath = "/log", dockerPath = "/log", mode="rw")
rmVolumes = Volumes(volumes = List(logVol) )
```

Por último los puertos, las redes, variables de entorno, configuración de despliegue y volúmenes declarados son asignados al nodo como se muestra a continuación, en este ejemplo al nodo *rm-0*.

```
rm = ReplicaManager(
  nodeId      = NodeId("rm-0"),
  ports       = rmPorts,
  networks    = rmNets,
  volumes     = rmVolumes,
  environments = rmEnvs,
  deploy      = rmDeploy
)
```

El gestor de réplicas es uno de los atributos de la clase piscina de almacenamiento (*StoragePool*), como se muestra en la Figura 4.8, de la misma forma en la que se declara un gestor de réplicas, los nodos de almacenamiento también son declarados y son asignados al atributo correspondiente del objeto piscina de almacenamiento, como se indica a continuación.

```
pool = StoragePool(
  storagePool= "pool-0",
  replicaManager = rm,
  storageNodes = List(...)
)
```

Las piscinas de almacenamiento tienen un método para convertir el gestor réplicas y los nodos de almacenamiento en un objeto de la clase *servicio*, que por herencia es la superclase, como se muestra en la Figura 4.7

4.5.2.2. Construcción

Esta fase contempla el proceso de gestión de las máquinas físicas pertenecientes al cluster de servidores de almacenamiento. Esta gestión se lleva a cabo a través de archivos de definición legibles por las máquinas en lugar de configurar manualmente el hardware físico o usar herramientas de configuración interactivas. Como resultado de la primera fase (*modelo contextual de definición*) se interpreta el atributo *deploy* de los servicios que se declararon, con lo que se especifican los recursos necesarios para los nodos de almacenamiento, estos recursos son reservados para tales servicios.

En los componentes gestor de réplicas y los nodos de almacenamiento es indispensable que se declare la cantidad de recursos como la cantidad de memoria RAM, cantidad de núcleos, la maquina física en donde debe de colocarse y las políticas de reinicio en caso de presentar una falla, como se observa en la Figura 4.9. |

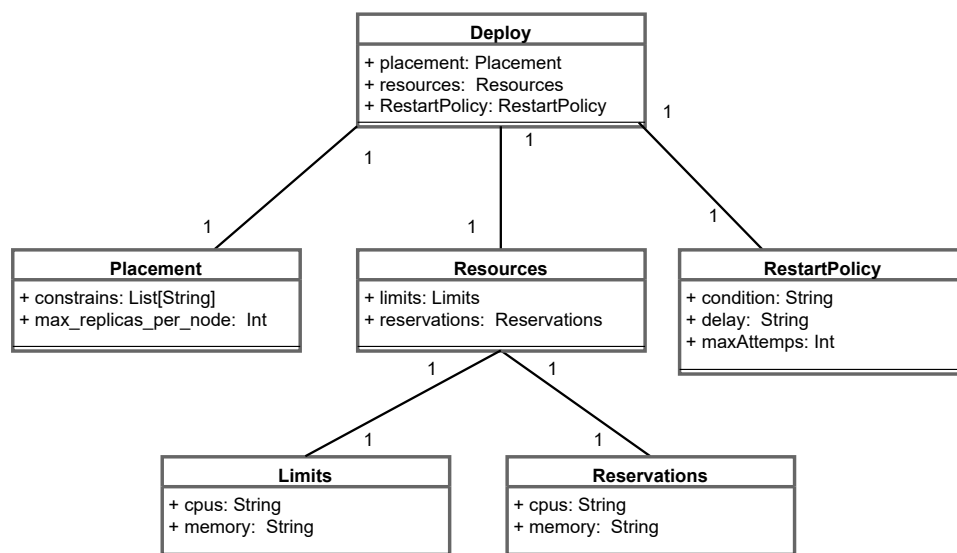


Figura 4.9: Diagrama de clases que representa la asignación de recursos, colocación y políticas de reinicio.

En la Figura 4.9 se observa como la clase *Deploy*, contiene una relación con *Placement*, en dicha clase se expresa una lista de cadenas de texto en donde se expresa una condición para colocar un

nodo, por ejemplo a cada máquina física se le puede otorgar un identificador único. Un ejemplo de esto sería "node.id == maquina1", lo cual indica que el atributo *id* del nodo debe de ser igual a "maquina1" para colocar el componente. *RestartPolicy* es la clase donde se expresa la política para reiniciar los componentes que han sido desplegados y la cantidad de intentos antes de dejar de intentar reiniciar, en busca de que la acción de reiniciar deje en un estado válido al nodo que ha sido desplegado. La clase *Resources* permite expresar la cantidad de memoria RAM y la cantidad de núcleos asignados al contenedor, esta también permite reservar memoria RAM y núcleos en caso de que los que han sido asignados se superen y poder hacer frente a sobrecargas de trabajo.

4.5.2.3. Despliegue

En esta fase se interpreta el archivo de configuración descrito en la Fase 1 para generar un archivo YML que siga las *Compose Specifications*³ y permita desplegar los servicios declarados como *contenedores virtuales Docker*, utilizando los recursos definidos para cada servicio. En esta fase es importante que se utilice un *orquestador de contenedores virtuales*. La orquestación de contenedores automatiza el aprovisionamiento, el despliegue, redes, escalamiento y la gestión del ciclo de vida de los contenedores. En la implementación del método propuesto se utilizó *Docker Swarm*⁴ como orquestador.

El YML tendrá una estructura bien definida, con el cual se consigue definir imágenes de contenedor virtual que pueden ser reutilizadas para generar instancias de contenedores virtuales que se basen en la misma imagen. Una imagen es un archivo utilizado para ejecutar código en un contenedor. Las imágenes actúan como un conjunto de instrucciones para construir un contenedor virtual, puede entenderse como una *plantilla*. En la Figura 4.8 el objeto de una piscina de almacenamiento tiene un método que interpreta el gestor de réplicas y los nodos de almacenamiento, preparándolos para

³Una *Compose Specification* permite definir una aplicación basada en un contenedor virtual, independiente de la plataforma de contenedores.

⁴Es una herramienta de orquestación de contenedores virtuales, lo que significa que permite al usuario gestionar múltiples contenedores virtuales desplegados en múltiples máquinas anfitrionas

pasarlos como parámetro al objeto que representa un *Compose File*, como se muestra en la Figura 4.10.

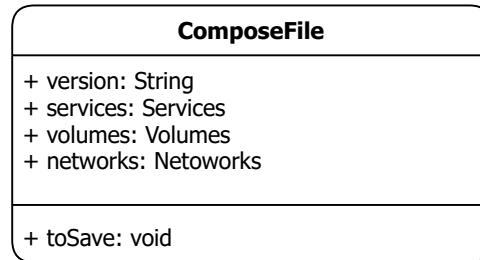


Figura 4.10: Clase que representa un *Compose File*.

Se crea una instancia de la clase *ComposeFile* donde los atributos son: la versión del *ComposeFile*, los servicios que serán declarados al momento de guardar el *ComposeFile* utilizando el método *toSave*. También se pueden declarar volúmenes en caso de que no se hayan creado manualmente. De igual manera que los volúmenes, las redes pueden ser declaradas en este objeto, sin embargo, existe la opción de utilizar redes previamente creadas, como se muestra en el ejemplo siguiente, donde se indica que se utilizarán las que ya han sido creadas en el gestor de réplicas.

```
cf = ComposeFile(
    version = "3",
    services = pool.toServices,
    volumes = Volumes.empty,
    networks = rmNets
)
cf.toSave
```

Al finalizar esta fase se generará un archivo YML que contendrá todos los nodos declarados respetando las *Compose Specifications* [59].

4.5.2.4. Ejecución

En esta fase, mediante el uso de las imágenes de contenedor definidas en la fase anterior, se pueden desplegar en forma de contenedores virtuales. Para ello, en la implementación del método, se

utilizó la plataforma de Docker como herramienta de virtualización. En este punto los componentes del sistema de replicación han sido desplegados en forma de contenedores virtuales y se encuentran en ejecución. Así, el sistema de replicación es capaz de generar nuevos nodos de almacenamiento según los vaya requiriendo, utilizando para ello las mismas clases antes definidas.

Para crear estos nuevos nodos no se repite el proceso de generación de archivos *YML*, en tiempo de ejecución únicamente hace uso de una *API* definida para ello. Esta API permite la interacción con Docker empleando cualquier lenguaje de programación en donde se hayan definido las clases. Las nuevas instancia de los nodos se crean utilizando esas clases ya definidas, las que son interpretadas e instanciadas en tiempo de ejecución utilizando el método *toDocker* definido en las mismas clases. Este método convierte la clase en un formato válido para la *API* de Docker y ésta pueda obtener los valores declarados y desplegar el componente en forma de contenedor virtual.

5

Evaluación experimental y resultados

En este capítulo se describen los procedimientos de evaluación del prototipo que implementa el método propuesto con el fin de conocer su desempeño. De igual manera, se describen los elementos necesarios para la creación de los escenarios experimentales, así como las configuraciones utilizadas en cada escenario. Además, se presentan los resultados obtenidos de cada uno de los experimentos realizados.

5.1 Entorno de evaluación

El prototipo que implementa el método de replicación propuesto fue evaluado en dos etapas. En la primera etapa se evaluó el rendimiento del método propuesto en un ambiente reducido con experimentación controlada. En la segunda etapa se evaluó el método en una comparación directa con otros métodos propuestos en la literatura que implementan estrategias de replicación tradicionales, tales como replicación activa y pasiva; ambas siguiendo la norma tradicional de replicación (3 réplicas para cada 5 nodos del sistema de almacenamiento).

5.1.1 Infraestructura empleada

El despliegue del prototipo fue realizado sobre una infraestructura de nube privada cuyas características se describen en la Tabla 5.1. En esta infraestructura se despliegan los componentes: gestor de réplicas, nodos de almacenamiento y clientes, en forma de contenedores virtuales. Las instancias indicadas son de un cluster de Docker Swarm. Los componentes del sistema se distribuyen como se indica. Los equipos *disys18*, *disys19* y *disys20* fueron utilizados para el despliegue de nodos de almacenamiento que conforman la piscina de almacenamiento.

Nombre de estancia	Núm. de núcleos	RAM	Cap. almacenamiento (TB)
disys17 (Replica master)	12	64	18
disys18 (Storage nodes)	12	64	2.7
disys19 (Storage nodes)	12	64	5.4
disys20 (Storage nodes)	12	64	5.4

Tabla 5.1: Infraestructura empleada para desplegar el sistema de almacenamiento.

Los componentes del prototipo fueron desplegados en forma de contenedores virtuales Docker. El gestor de réplicas (*RM*) se desplegó en el servidor físico *disys17* utilizando todos los recursos de hardware disponibles. Los nodos de almacenamiento se balancearon utilizando el algoritmo *Round Robin* en los servidores físicos denominados *disys18*, *disys19* y *disys20*, a cada uno de éstos se les asignó una capacidad de disco de 100GB, 4GB de memoria RAM y 2 núcleos.

5.1.2 Lanzador de cargas

Se implementó un cliente (*bot*) para producir y consumir cargas de trabajo sintéticas, que fueron enviadas y recibidas al/del prototipo. El cliente puede tomar el rol de *productor* al crear cargas de trabajo conteniendo únicamente operaciones de carga de archivos. El rol de *consumidor* lo adquiere

cuando la carga de trabajo contiene únicamente operaciones de descarga de archivos. Los clientes clasificados como *consumidores* tienen la opciones de realizar descargas de todos los archivos que se han cargado con éxito en el sistema de almacenamiento. Estos clientes *consumidores* también han sido configurados para producir cargas de trabajo concurrentes, donde dicho cliente realiza N descargas de los archivos publicados. Como resultado el cliente consumidor crea tráfico concurrente de descargas de archivos de manera dinámica.

Con el propósito de realizar una evaluación exhaustiva, el cliente (*bot*) fue configurado para recibir y enviar cargas de trabajo y procesarlas como si proviniera de usuarios reales (humanos o aplicaciones).

5.1.3 Generador de cargas de trabajo sintéticas

El cliente (*bot*) contiene un componente para generar cargas de trabajo. Las cargas de trabajo se producen en forma de *trazas*, las cuales se crean usando un *script* desarrollado en Python. Este *script* es encapsulado en el mismo contenedor virtual usado para encapsular el cliente (*bot*) del sistema de almacenamiento. Una traza básicamente es un archivo CSV que describe una carga de trabajo (un conjunto de operaciones del mismo tipo, ya sea de carga o descarga, incluyendo los campos descritos en la Tabla 5.2). El objetivo es crear una ráfaga de carga/descarga y observar qué sucede con el sistema de almacenamiento.

Los parámetros que recibe el generador de trazas son: 1) la cantidad de cargas; 2) el nombre de la distribución de datos que simulará la llegada o salida de las operaciones de carga/descarga, la cual se utilizará para asignar la cantidad de cargas/descargas de cada archivo, por defecto se utiliza la distribución de *Pareto*; 3) el tamaño promedio de los archivos; y 4) el tiempo promedio entre peticiones, el cual es exponencialmente distribuido. En la Tabla 5.2 se muestran los atributos que conforman las trazas que fueron generadas para la experimentación, que están conformadas por operaciones de carga y descarga.

Característica	Descripción
OPERATION_ID	Identificador único de operación
OPERATION_TYPE	Dos posibles valores: UPLOAD o DOWNLOAD
BALL_ID	Identificador único del archivo (esfera)
ARRIVAL_TIME	Tiempo de llegada de la operación
BALL_SIZE	Tamaño del archivo (esfera)
USER_ID	Identificador único del usuario que envía la operación
CONSUMER_ID	Identificador único del consumidor, en las operaciones de carga estará vacío

Tabla 5.2: Atributos de una traza sintética.

Una traza hace referencia a un archivo (esfera) que se cargará o descargará. Estos archivos se crearon mediante un *generador de archivos*, que también es un componente del cliente (*bot*). Este componente del cliente se ejecuta cuando el cliente tiene el rol de *producer*. El componente recibe como parámetros de entrada una traza de descarga, de la cual se obtienen los tamaños y los identificadores de los archivos que serán enviados al sistema de almacenamiento (prototipo). El contenido de los archivos se crea mediante una combinación de caracteres, letras y números elegidos de forma pseudo-aleatoria.

5.1.4 Métricas

Para realizar la evaluación del prototipo se programó en el cliente (*bot*) un componente para capturar las siguientes métricas:

- **Tiempo de respuesta (Rt):** Esta métrica permite evaluar el rendimiento del cliente y representa la cantidad de tiempo requerido para realizar una operación de carga o descarga pasando por todos los componentes requeridos, desde el cliente y el balanceo de carga hasta llegar al nodo de almacenamiento encargado de llevar a cabo la operación de carga o descarga.

- **Rendimiento (Th):** Esta métrica mide el volumen de datos que se procesa en el sistema de almacenamiento por una unidad de tiempo (MB/s).
- **Porcentaje promedio de acceso a réplica por nodo ($PPRxN$):** Esta métrica permite identificar el porcentaje de utilización promedio de las réplicas para cada nodo de almacenamiento. Si esta métrica es mayor a 100, entonces se interpreta una subutilización ($100 - PPRxN$). Se espera que esta métrica sea igual a 100 %, lo que indica el uso de todas las réplicas.

5.1.5 Configuraciones definidas

Las configuraciones para evaluar el rendimiento y el tiempo de respuesta del prototipo implementado son las siguientes, las cuales son declaradas utilizando el modelo contextual de definición.

- *Activa:* Para esta configuración se asignó un $FR = 3$ y un $RD = 5$, que es la configuración tradicional en un sistema de replicación, según la literatura. La creación de las réplicas se realiza de manera paralela, son retenidas hasta que las FR terminan de escribirse en los nodos de almacenamiento.
- *Pasiva:* Para esta configuración se asignó un $FR = 3$ y un $RD = 5$, pone gradualmente las réplicas creadas a disposición de los usuarios consumidores.
- *Activa-elástica:* Para esta configuración se utilizan los tiempos de espera para lanzar nuevos nodos y nuevas réplicas (FR y RD dinámicos). Las réplicas creadas no se ponen a disposición hasta que todas las réplicas son creadas con éxito.
- *Pasiva-elástica:* Para esta configuración se utilizan los tiempos de espera para lanzar nuevos nodos de almacenamiento y nuevas réplicas (FR y RD dinámicos). Las réplicas creadas

son puestas a disposición de los usuarios consumidores tan rápido como los nodos de almacenamiento notifiquen al gestor de réplicas *RM*.

Es importante recordar que las configuraciones que se propusieron en esta investigación son *activa-elástica* y *pasiva-elástica*, las cuales utilizan los tiempos de espera para accionar las réplicas. En el balanceador de carga se utilizó el algoritmo *Min-WT* para balancear las operaciones de descarga a los nodos con menor tiempo de espera, para las operaciones de carga se utilizó el algoritmo *SortingUF* [8]. En las configuraciones sin elasticidad se utilizó el algoritmo *Least-Hits* para las operaciones de descarga y el algoritmo *SortingUF* para las operaciones de carga.

5.2 Evaluación

Se realizó una evaluación en la cual se realizaron variaciones experimentales, las cuales se pueden ver en la Tabla 5.4, tales variaciones tienen las características definidas en la Tabla 5.3. La variación del nivel de concurrencia de descargas de archivos representa el incremento en el número de clientes que envían operaciones de carga/descarga de manera concurrente al prototipo. Se utilizaron los niveles de concurrencia 3, 6 y 12, lo cual puede ser entendido como la cantidad de usuarios que toman el rol de *consumidores* y realizan operaciones de descarga a los contenidos cargados previamente por un cliente *productor*. Las operaciones de carga fueron lanzadas por un cliente, el cual tomó el rol de *productor* cuya tarea principal fue lanzar las operaciones de carga definidas en una traza sintética que contiene operaciones de carga. Estas operaciones de carga hacen uso de archivos que pertenecen a un conjunto de archivos que se denomina *lote*. Se crearon 3 lotes de archivos variando su tamaño promedio: 1MB, 10MB y 100MB. Para todos los experimentos se realizó una repetición de 31 ejecuciones para obtener la mediana de los valores de las métricas definidas.

Abreviación	Descripción
NC	Nivel de concurrencia, indica la cantidad de clientes que realizan descargas de manera simultánea a los contenidos cargados al sistema.
TR	Técnica de replicación, puede tomar los siguientes valores: Sin replicación (NOREP), Activa (ACTIVE), Pasiva (PASSIVE), activa elástica (ACTIVE-ELASTIC) o pasiva elástica (PASSIVE-ELASTIC)
TPA	Tamaño promedio de archivos

Tabla 5.3: Descripción de abreviaciones de las configuraciones definidas.

5.2.1 Evaluación de los tiempos de respuesta de las configuraciones definidas

Se analizaron los tiempos de respuesta de las configuraciones definidas (*activa*, *pasiva*, *activa-elástica* y *pasiva-elástica*) para las operaciones de carga, descarga y global (carga de trabajo total que integra cargas y descargas). Para las cargas y descargas se obtiene la mediana para cada tipo de operación; para ello se filtran las operaciones y se obtienen los tiempos de respuesta. Para el cálculo del tiempo de respuesta de la carga de trabajo global se toma el registro de todas las operaciones que incluyen carga y descarga para obtener el tiempo de respuesta. Es importante mencionar que el tiempo de respuesta global no es igual a la suma de los tiempos de respuesta de las operaciones de carga y descarga ya que el volumen de las operaciones de carga y descarga son diferentes, cada operación de carga dispara múltiples operaciones de descarga, lo cual significa que las descargas tienen más peso en el rendimiento global del sistema de almacenamiento que las operaciones de carga. Por tanto, la operación global representa un patrón de producción y consumo de contenidos/archivos, el cual es realizado por una operación de carga (*productor* publicando un contenido/archivo) disparando un conjunto de descargas (producidas por los clientes cuando se suscriben y consumen un contenido

Identificador de experimento	NC	TR	TPA
C3_NOREP-1MB	3	–	1MB
C3_NOREP-10MB	3	–	10MB
C3_NOREP-100MB	3	–	100MB
C6_NOREP-1MB	6	–	1MB
C6_NOREP-10MB	6	–	10MB
C6_NOREP-100MB	6	–	100MB
C12_NOREP-1MB	12	–	1MB
C12_NOREP-10MB	12	–	10MB
C12_NOREP-100MB	12	–	100MB
C3_ACTIVE-1MB	3	Activa	1MB
C3_ACTIVE-10MB	3	Activa	10MB
C3_ACTIVE-100MB	3	Activa	100MB
C6_ACTIVE-1MB	6	Activa	1MB
C6_ACTIVE-10MB	6	Activa	10MB
C6_ACTIVE-100MB	6	Activa	100MB
C12_ACTIVE-1MB	12	Activa	1MB
C12_ACTIVE-10MB	12	Activa	10MB
C12_ACTIVE-100MB	12	Activa	100MB
C3_PASSIVE-1MB	3	Pasiva	1MB
C3_PASSIVE-10MB	3	Pasiva	10MB
C3_PASSIVE-100MB	3	Pasiva	100MB
C3_ACTIVE_ELASTIC-1MB	3	Activa elástica	1MB
C3_ACTIVE_ELASTIC-10MB	3	Activa elástica	10MB
C3_ACTIVE_ELASTIC-100MB	3	Activa elástica	100MB
C6_ACTIVE_ELASTIC-1MB	6	Activa elástica	1MB
C6_ACTIVE_ELASTIC-10MB	6	Activa elástica	10MB
C6_ACTIVE_ELASTIC-100MB	6	Activa elástica	100MB
C12_ACTIVE_ELASTIC-1MB	12	Activa elástica	1MB
C12_ACTIVE_ELASTIC-10MB	12	Activa elástica	10MB
C12_ACTIVE_ELASTIC-100MB	12	Activa elástica	100MB
C3_PASSIVE_ELASTIC-1MB	3	Pasiva elástica	1MB
C3_PASSIVE_ELASTIC-10MB	3	Pasiva elástica	10MB
C3_PASSIVE_ELASTIC-100MB	3	Pasiva elástica	100MB
C6_PASSIVE_ELASTIC-1MB	6	Pasiva elástica	1MB
C6_PASSIVE_ELASTIC-10MB	6	Pasiva elástica	10MB
C6_PASSIVE_ELASTIC-100MB	6	Pasiva elástica	100MB
C12_PASSIVE_ELASTIC-1MB	12	Pasiva elástica	1MB
C12_PASSIVE_ELASTIC-10MB	12	Pasiva elástica	10MB
C12_PASSIVE_ELASTIC-100MB	12	Pasiva elástica	100MB

Tabla 5.4: Descripción de las combinaciones definidas

publicado).

La Figura 5.1 muestra una comparación de las medianas correspondientes al tiempo de respuesta por configuración (eje vertical). Como se puede observar, la configuración sin replicación *C3-NO_REP-100MB* produce el tiempo de respuesta más elevado debido al impacto del encolamiento de las descargas, expresado en términos de tiempos de espera, los cuales son sumados a los tiempos de respuesta de cada operación realizada en los nodos de almacenamiento. En las configuraciones restantes, donde la replicación es habilitada, se observan tiempos de respuesta menores a la

configuración base (*C3-NO_REP-100MB*), lo cual muestra la eficacia de las técnicas de replicación propuestas para reducir los tiempos de espera de las peticiones.

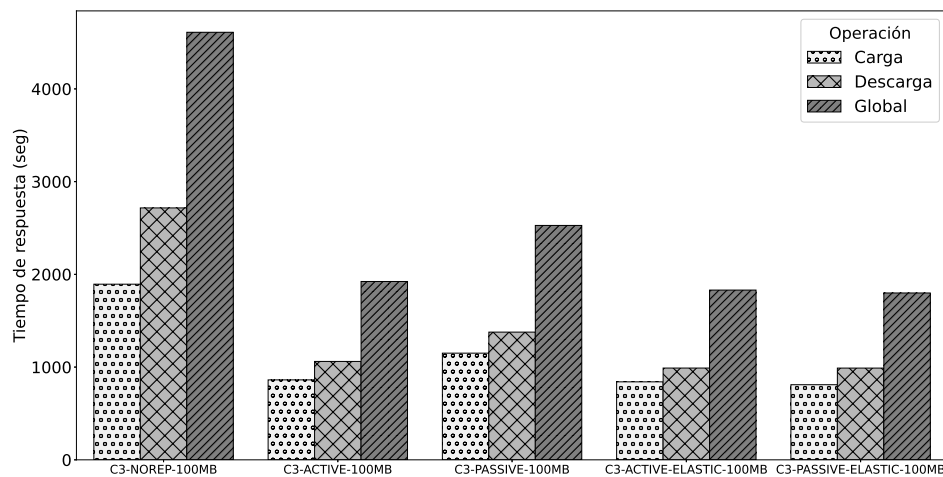


Figura 5.1: Tiempos de respuesta para las cinco configuraciones con tres clientes concurrentes.

La Figura 5.2 muestra la comparación de las medianas correspondientes al tiempo de respuesta (eje vertical), en donde se incrementa la concurrencia (eje horizontal). Cuando se añaden más clientes que realizan descargas concurrentes se incrementa la cantidad de operaciones de descarga que el sistema procesa. Con 3 clientes concurrentes la cantidad de descargas es de 3000, para 6 clientes son 6000 y para 12 clientes son 12000, representando en porcentaje de operaciones del 75 %, 85 % y 92 % respectivamente para cada nivel de concurrencia.

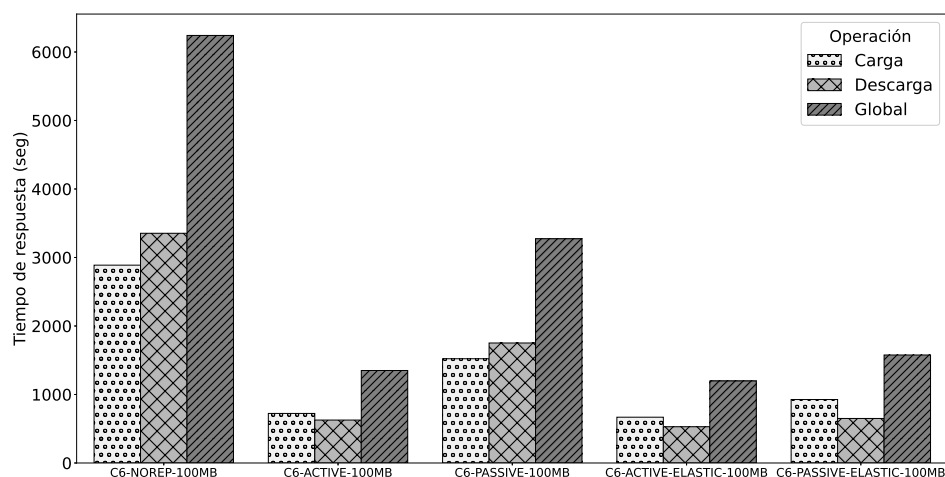


Figura 5.2: Tiempos de respuesta para las cinco configuraciones con seis clientes concurrentes.

La Figura 5.3 presenta los tiempos de respuesta para las cinco configuraciones incrementando el nivel de concurrencia a 12, lo que significa que 12 clientes realizarán operaciones de descarga de manera concurrente a los contenidos publicados. Como se puede observar, a mayor concurrencia mayor la eficiencia de las técnicas elásticas propuestas, específicamente de las técnicas *pasiva-elástica*.

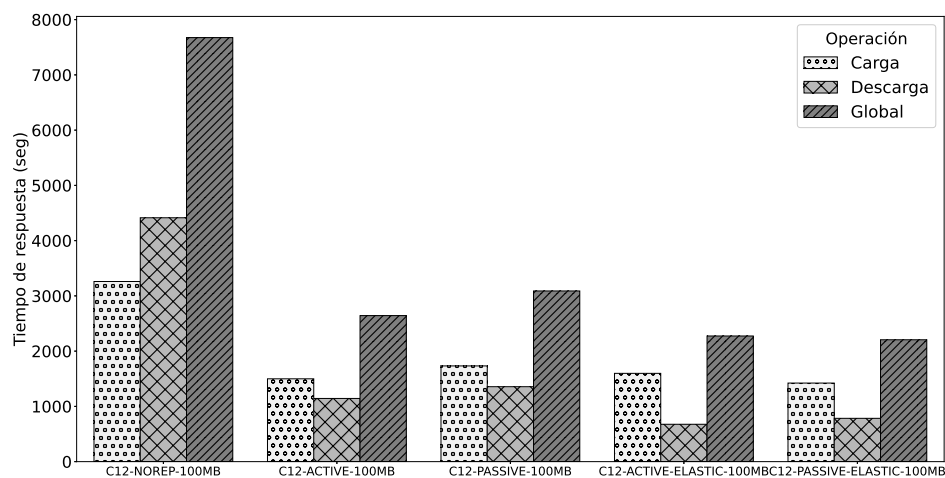


Figura 5.3: Tiempos de respuesta para las cinco configuraciones con doce clientes concurrentes.

La Figura 5.4 presenta, en el eje vertical, los porcentajes de ganancia para los niveles de concurrencia evaluados (eje horizontal). Se puede observar que conforme se incrementa el nivel de

conurrencia, las configuraciones que emplean la técnica de replicación *pasiva* mejoran su eficiencia. Hay que recordar que este tipo de técnica gradualmente produce y distribuye réplicas, las cuales son publicadas/consumidas cuando han sido escritas correctamente en un nodo de almacenamiento. Esta técnica básicamente reduce la carga subyacente y hace disponible las réplicas gradualmente, lo cual mejora el rendimiento gradualmente el sistema de almacenamiento. En cambio, las técnicas activas producen una mayor carga de trabajo subyacente pero producen un ancho de banda mayor. De lo anterior, se deduce que las técnicas activas son invasivas pero mejoran el tiempo de respuesta en el corto plazo (al abrir un ancho de banda mayor), mientras que las técnicas pasivas tratan de reducir el impacto del proceso de replicación a cambio de una liberación de un incremento de ancho de banda en forma gradual. Las configuraciones que emplean la técnica de replicación activa comienzan a degradar su rendimiento conforme se incrementa el nivel concurrencia, lo cual degrada el rendimiento del sistema en escenarios de alta concurrencia, caso contrario con las técnicas de replicación pasiva.

En la Figura 5.5 se muestra el porcentaje de ganancia (eje vertical) de las configuraciones definidas con respecto a la configuración sin replicación, donde se puede observar que las configuraciones que emplean técnicas de replicación pasiva tienen mejores tiempos de respuesta. Siendo ese tipo de replicación muy efectiva en escenarios de alta concurrencia debido a que es menos invasiva que las técnicas activas. Cuando el nivel de concurrencia aumenta, también aumenta la carga de trabajo subyacente (producción y distribución de réplicas) así como la retención de las réplicas que contribuyen al aumento del tiempo de respuesta de las técnicas de replicación activas. En este tipo de escenario estas técnicas dejan de ser lo efectivas que son en escenarios de concurrencia reducida (por ejemplo cuando 3 o 6 clientes consumen archivos simultáneamente).

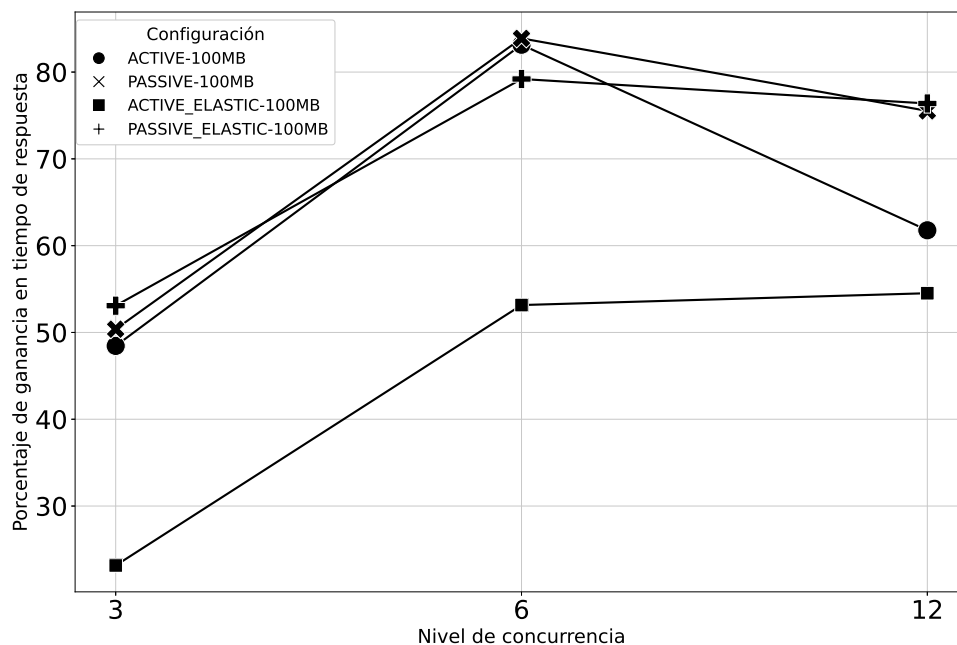


Figura 5.4: Porcentajes de ganancia con respecto a la configuración sin replicación incrementando el nivel de concurrencia.

Con el fin de dimensionar el efecto de la carga subyacente así como el impacto de la interrupción de las técnicas de replicación en el rendimiento del sistema de almacenamiento, se comparó el rendimiento de las técnicas elásticas con la mejor configuración de replicación tradicional (activa con RD fijo y FR variable en función de la popularidad de los contenidos). Se calculó el porcentaje de diferencia y se representó en la Figura 5.5 en el eje vertical, para los diferentes niveles de concurrencia (eje horizontal). Se puede observar que, cuando el nivel de concurrencia aumenta, las técnicas elásticas propuestas, mejoran los tiempos de respuesta en un 13 % para la replicación *activa-elástica* y en un 14 % para la replicación *pasiva-elástica*, lo cual ilustra los efectos de la carga subyacente en el rendimiento del sistema de almacenamiento descrito previamente. Esto lo consigue la replicación pasiva incrementando la probabilidad de disponibilidad gradualmente, evitando la retención de las

réplicas e inyectando la carga subyacente en forma gradual.

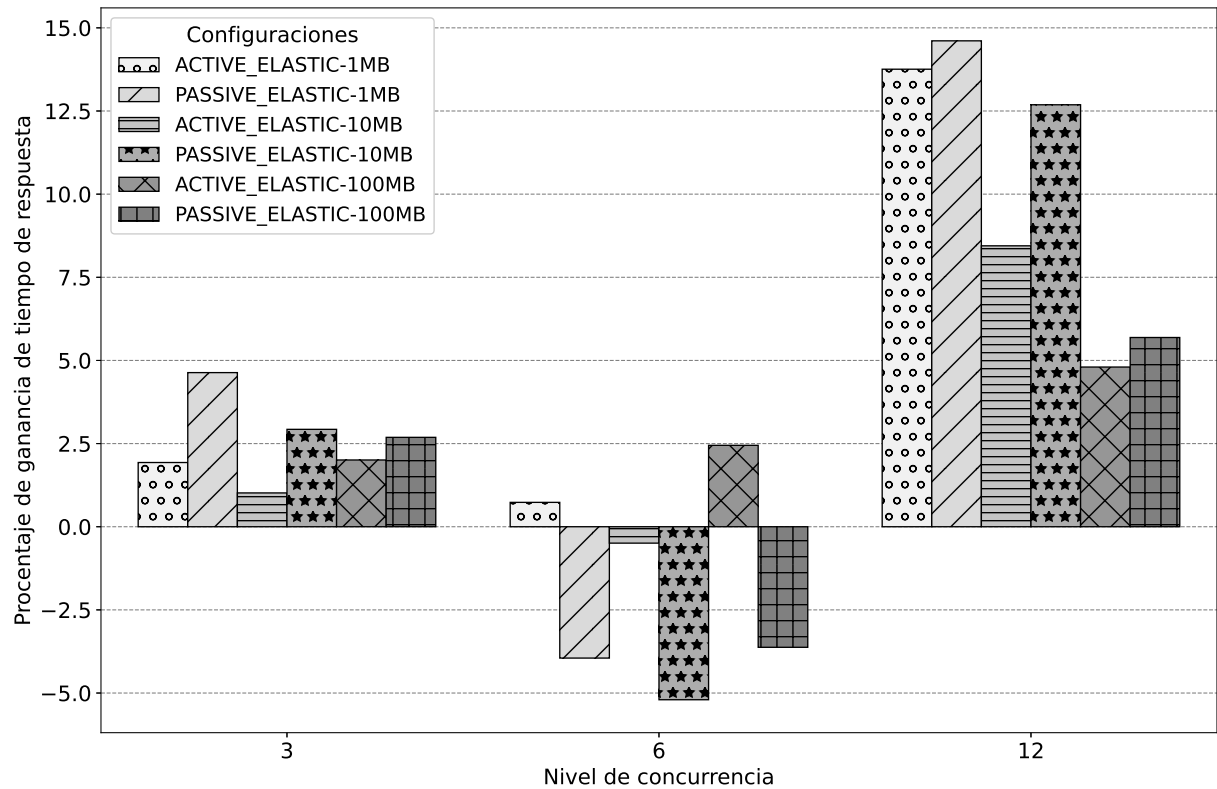


Figura 5.5: Porcentaje de ganancia con respecto a la replicación activa.

El tamaño de los archivos tiene un impacto significativo en el comportamiento de un sistema de almacenamiento de alta disponibilidad, debido a que si es menor el tamaño del archivo los nodos de almacenamiento procesarán más rápido las operaciones de carga/descarga. Sin embargo cuando el tamaño de los archivos se incrementa también se incrementa la cantidad de tiempo que le toma a un nodo de almacenamiento realizar una operación de carga/descarga. Esto a su vez eleva los tiempos de espera conforme se incrementa el tamaño debido al incremento del tiempo de servicio cuando se tiene una configuración que realiza replicación. Esto también hará que la carga de trabajo

subyacente (*SW*) sea más nociva para el sistema, incrementando los tiempos de espera y degradando la experiencia del usuario (*UX*).

5.2.2 Análisis de patrones de producción y consumo

El *rendimiento* de un sistema de almacenamiento de alta disponibilidad se mide por la cantidad de accesos producidos por los patrones de producción y consumo manejados por dicho sistema en una unidad de tiempo. El rendimiento también puede definirse como la cantidad de datos movidos satisfactoriamente de un lugar a otro en un período determinado (megabytes por segundo, MB/s). El rendimiento en las operaciones de carga indica cuántos megabytes son escritos por segundo y en las descargas cuantos megabytes son transferidos por segundo. El rendimiento global es la cantidad de bytes que el sistema procesa en un patrón de producción/consumo de operaciones de carga y descarga.

La Figura 5.6 muestra que, tal como era esperado, la configuración sin replicación produce el menor rendimiento en la comparación realizada, debido al encolamiento en los nodos producido por los accesos a un archivo en un tiempo dado, lo cual es determinado por la concurrencia. Por ejemplo, sin replicación, un archivo producido en un escenario de concurrencia será accedido 3, 6 o 12 veces simultáneamente produciendo un encolamiento para los accesos al nodo de almacenamiento, lo cual produce un crecimiento exponencial de los tiempos de espera, afectando significativamente a los tiempos de respuesta. En las configuraciones donde la replicación está habilitada el rendimiento del sistema mejora conforme se incrementa la concurrencia, como se puede observar en las Figuras 5.6, 5.7 y 5.8, donde este efecto de mejoramiento es más evidente en las operaciones de descarga.

En las Figuras 5.6, 5.7 y 5.8 se puede observar cómo las configuraciones con replicación elástica, principalmente la configuración con replicación *pasiva-elástica*, mejora el rendimiento de las operaciones de las descargas, mientras que, como era esperado, la replicación activa mejora las operaciones de carga. Esto es importante porque se espera que el volumen de descargas sea mayor en una carga de trabajo. Se espera que, en la práctica, se observe una relación regida por el principio

de *Pareto*, donde se espera que las cargas representen solo un porcentaje pequeño, típicamente del orden del 20% y las descargas un porcentaje más grande, típicamente del orden de 80%, lo cual producirá un efecto similar al evaluado al incrementar la concurrencia. Esto significa que el mejoramiento producido por la técnica *pasiva-elástica* en las descargas, aunque parece pequeña, es significativa tomando en cuenta el volumen de descargas que se espera recibir en la práctica.

Se puede observar en la Figura 5.6 que el rendimiento cae cuando se usa la configuración de replicación *activa-elástica* y se incrementa el nivel de concurrencia.

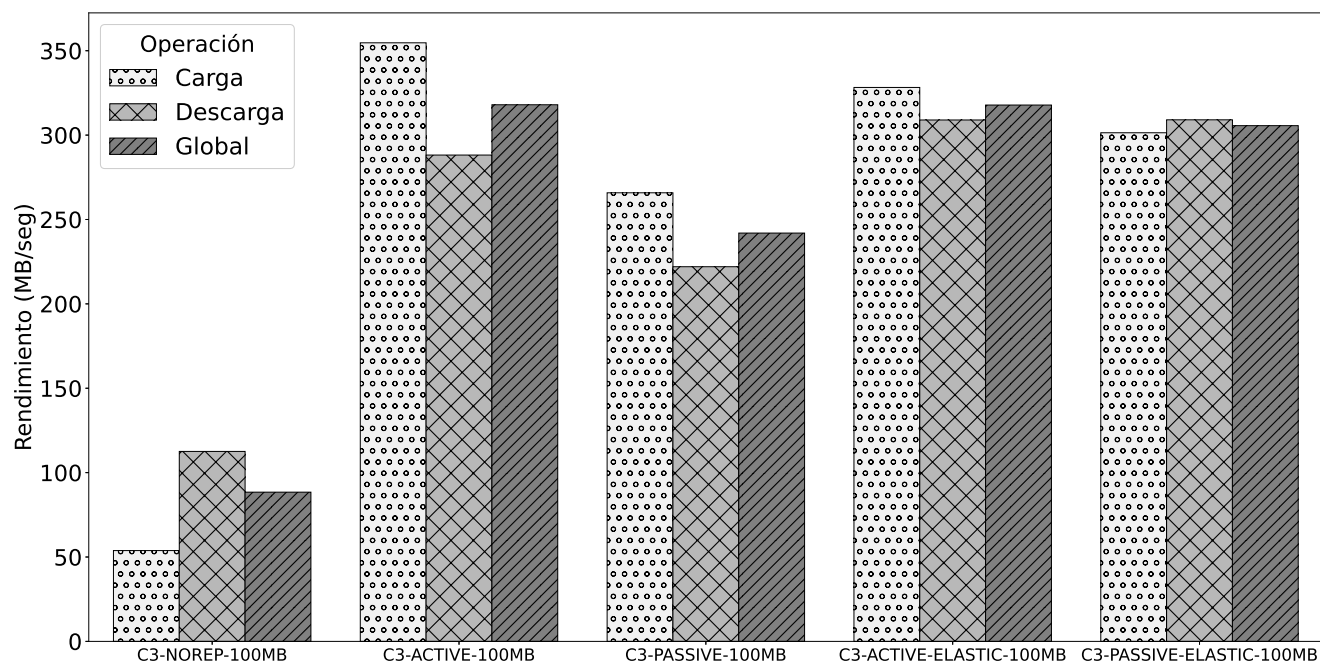


Figura 5.6: Rendimiento de las cinco configuraciones procesando un lote de 1000 archivos con tamaño promedio de 100MB con nivel de concurrencia 3.

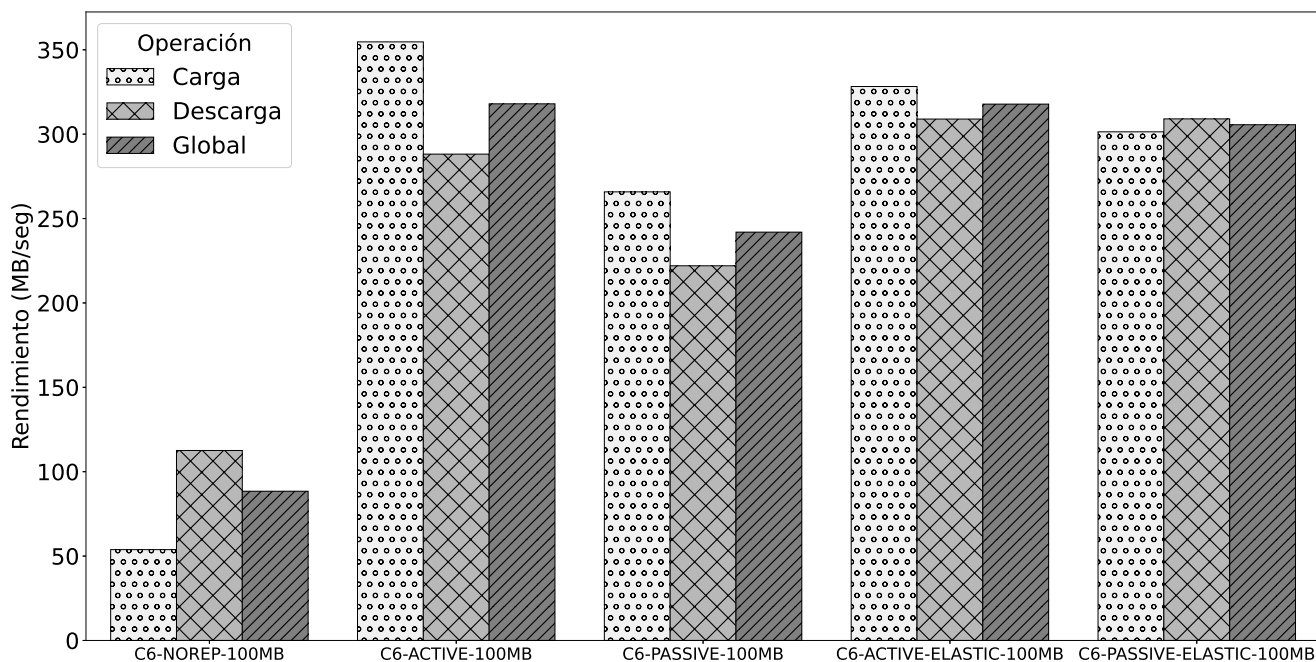


Figura 5.7: Rendimiento de las cinco configuraciones procesando un lote de 1000 archivos con tamaño promedio de 100MB con nivel de concurrencia 6.

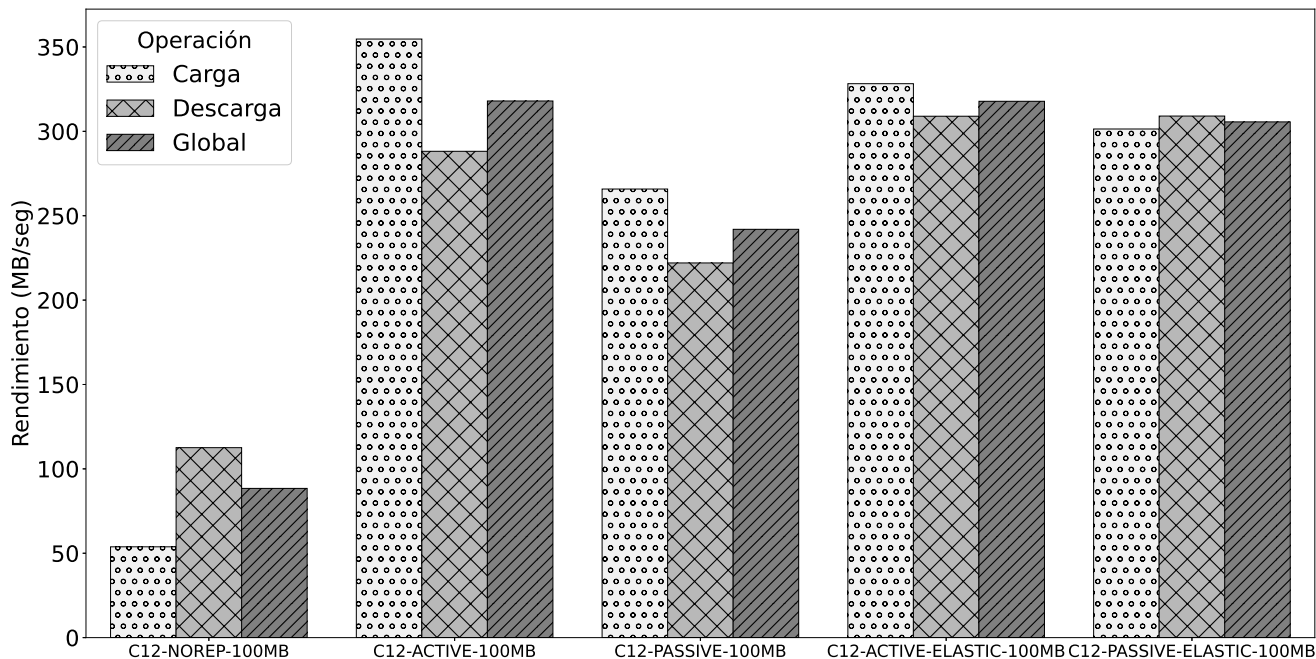


Figura 5.8: Rendimiento de las cinco configuraciones procesando un lote de 1000 archivos con tamaño promedio de 100MB con nivel de concurrencia 12.

Con el fin de mostrar el comportamiento para cada configuración estudiada para cada escenario de concurrencia y tamaño de archivo, en las Figuras 5.9 a 5.17 se muestran las gráficas de tiempos de respuesta y rendimiento de dichas configuraciones. En todas las gráficas la altura izquierda corresponde al tiempo de respuesta (en segundos), la altura derecha corresponde al rendimiento (MB/seg) y la parte baja corresponde a las configuraciones variando el NC -*nivel de concurrencia*-, TR -*técnica de replicación*- y TPA -*tamaño promedio de archivos*-, como se indica en el pie de figura de cada gráfica. En todas las gráficas las etiquetas que corresponden a TR mantienen el siguiente orden: NOREP, ACTIVE, PASSIVE, ACTIVE-ELASTIC Y PASSIVE-ELASTIC.

Como se esperaba y se puede observar, los tiempos de respuesta y el rendimiento expresado en capacidad por unidad de tiempo producen comportamientos inversos.

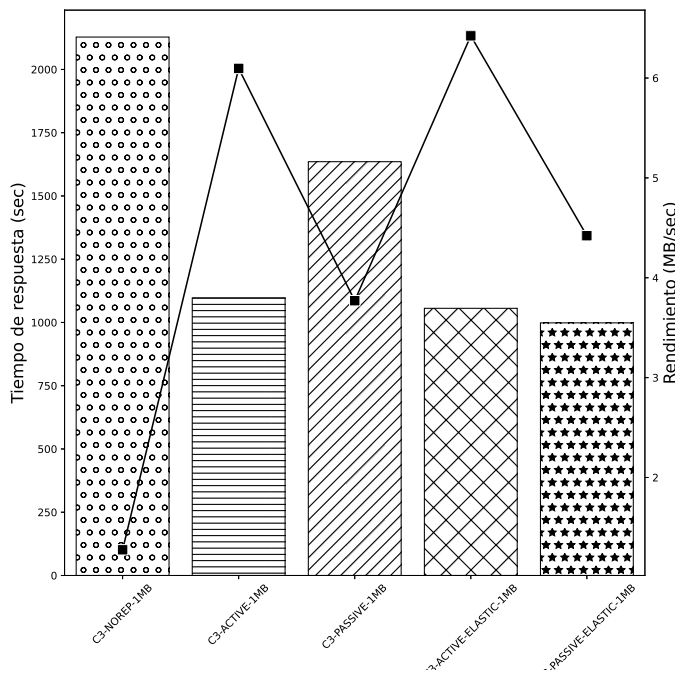


Figura 5.9: Nivel de concurrencia 3 y archivos de tamaño promedio de 1MB

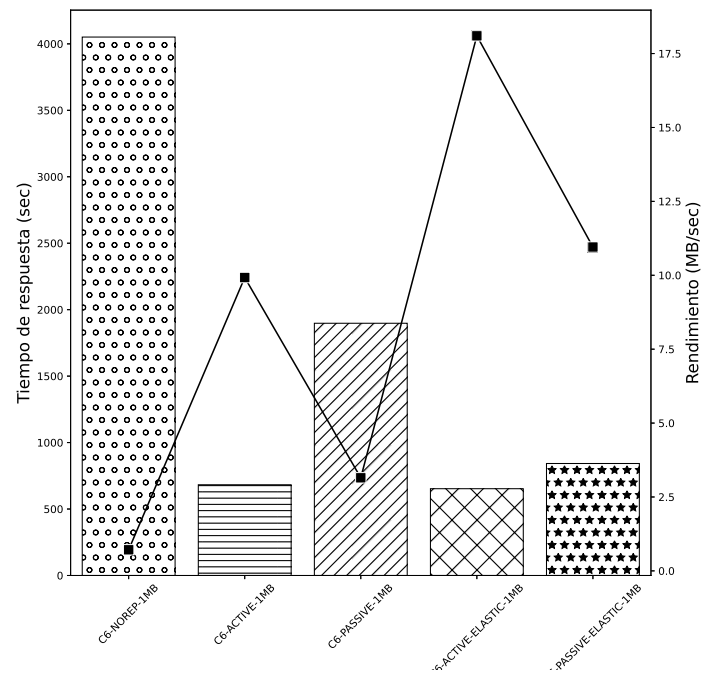


Figura 5.10: Nivel de concurrencia 6 y archivos de tamaño promedio de 1MB

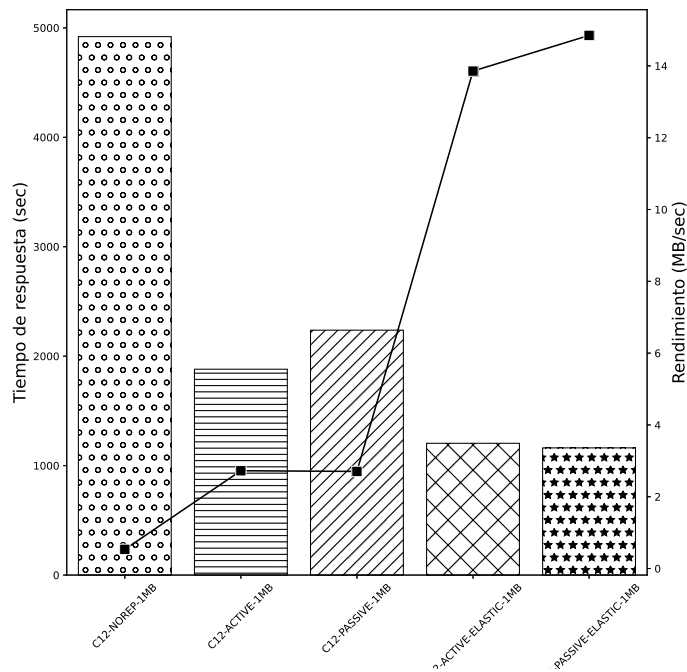


Figura 5.11: Nivel de concurrencia 12 y archivos de tamaño promedio de 1MB

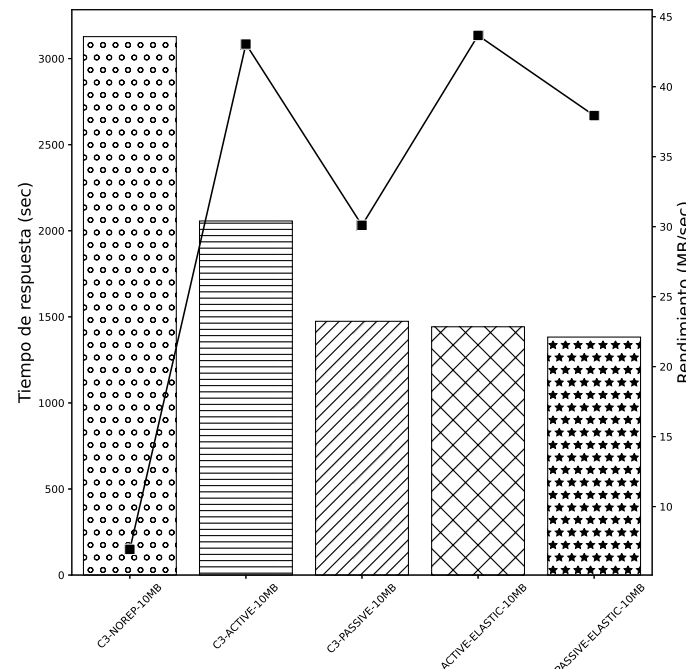


Figura 5.12: Nivel de concurrencia 3 y archivos de tamaño promedio de 10MB

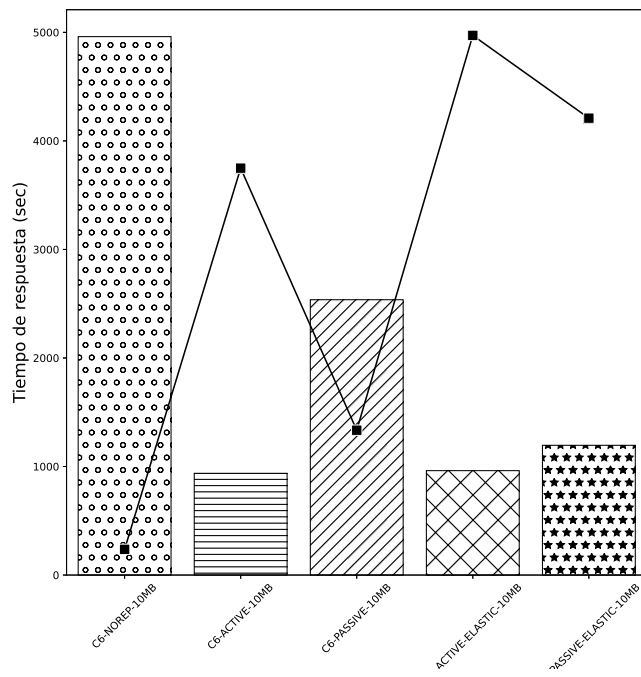


Figura 5.13: Nivel de concurrencia 6 y archivos de tamaño promedio de 10MB

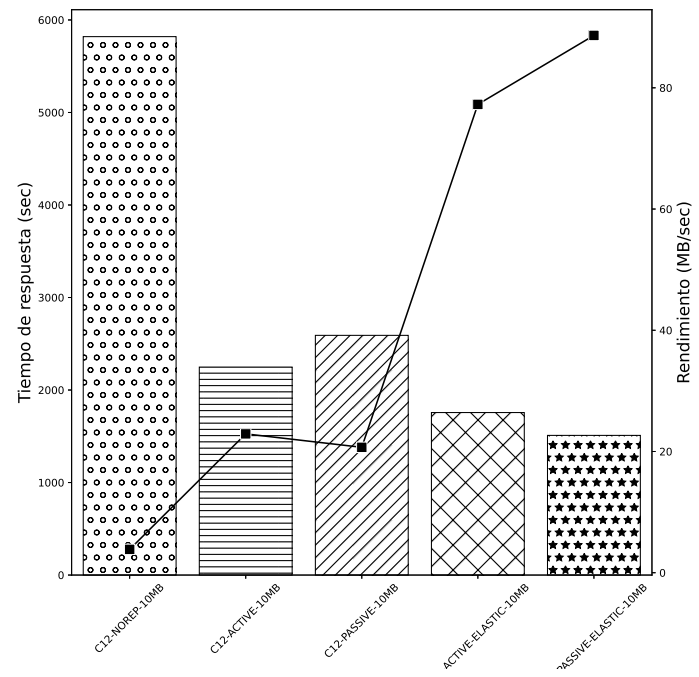


Figura 5.14: Nivel de concurrencia 12 y archivos de tamaño promedio de 10MB

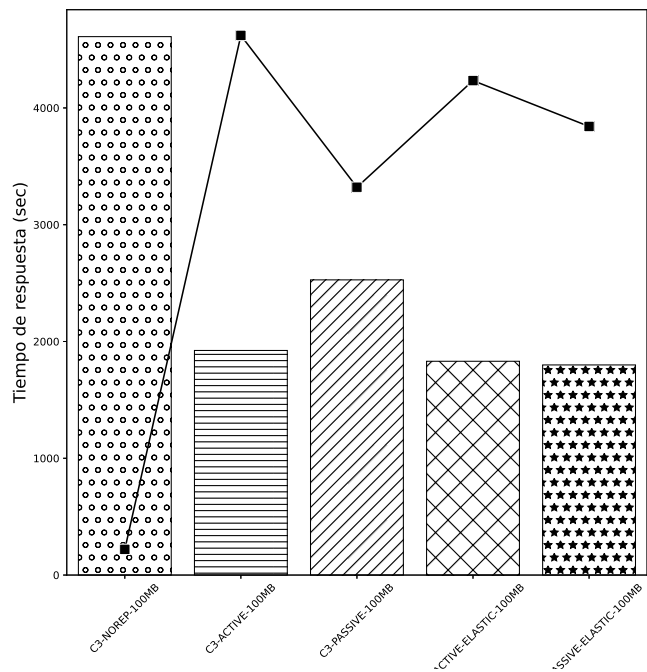


Figura 5.15: Nivel de concurrencia 3 y archivos de tamaño promedio de 100MB

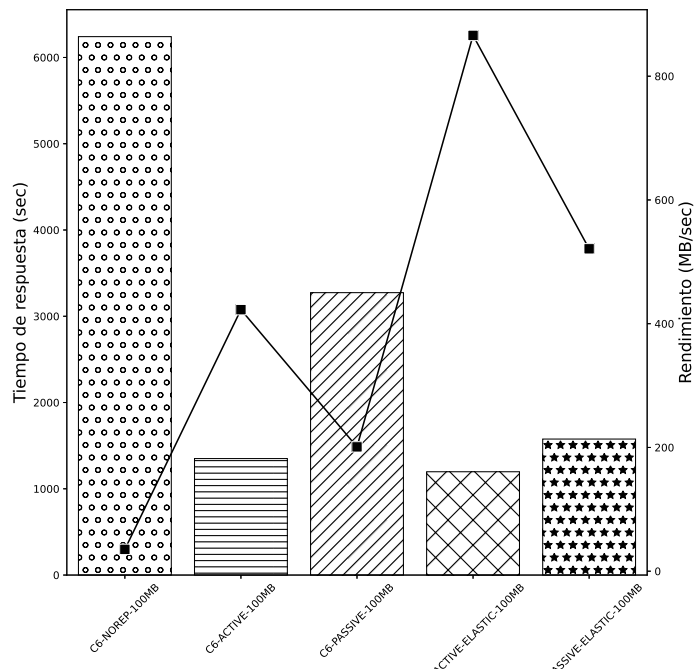


Figura 5.16: Nivel de concurrencia 6 y archivos de tamaño promedio de 100MB

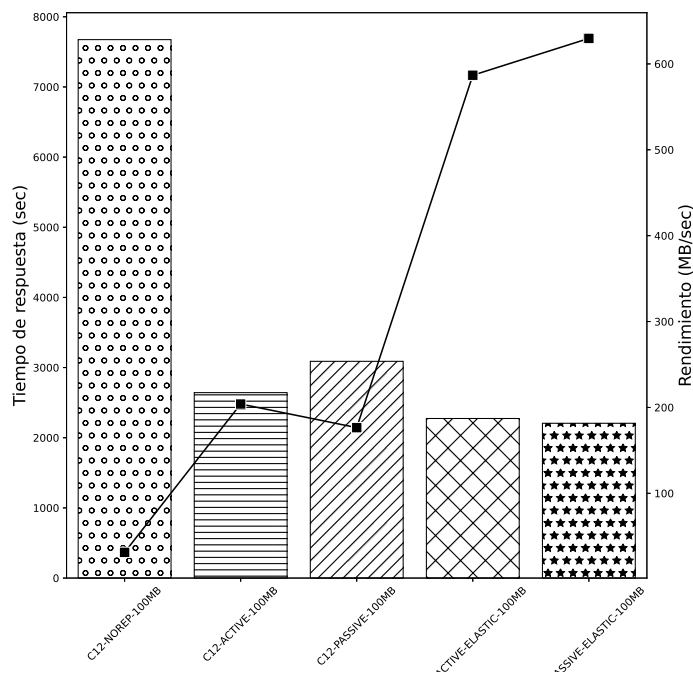


Figura 5.17: Nivel de concurrencia 12 y archivos de tamaño promedio de 100MB

En todos los experimentos, como lo muestran las gráficas, el comportamiento es similar: a mayor nivel de concurrencia más eficaces las técnicas elásticas (activa y pasiva). La carga subyacente aumenta con el tamaño del archivo y las técnicas *pasiva-elástica* mejoran significativamente en escenarios de alta concurrencia, mostrándose como las mejores configuraciones para este tipo de escenarios. Estas técnicas también resultan ser la mejor opción cuando el volumen de descargas es mayor a las cargas. Las técnicas *activa-elástica* resultan ser más eficaces en escenarios de baja concurrencia y para cuando las cargas son críticas y similares en volumen a las descargas.

5.2.3 Análisis de la dinámica de accesos por nodo

También se analizó la cantidad de accesos por nodo de cada configuración de las técnicas de replicación implementadas en el prototipo: *norep*, *activa*, *pasiva*, *activa-elástica* y *pasiva-elástica*.

Los accesos a los datos escritos (réplicas) en un nodo de almacenamiento definen la eficacia de un sistema de replicación. Esta eficacia se puede medir mediante la observación de tres métricas:

- El porcentaje promedio de acceso a réplicas por nodo ($PPRxN$). Esta métrica permite identificar el porcentaje de utilización de las réplicas para cada nodo. Su interpretación se realiza de la siguiente forma:
 - Si el porcentaje de esta métrica es inferior al 100 % significa *sub-utilización* ya que $subutilizacion = 100 - PPRxN$, y se espera que $PPRxN \leq 100$.
 - Si el porcentaje $PPRxN$ es igual a 100 entonces se está en el escenario ideal, en el cual cada réplica ha sido utilizada al menos una vez en un nodo, lo que significa que no se esperan retrasos en la atención y eso garantiza una probabilidad de máxima disponibilidad del sistemas para acceder a cualquier contenido que haya sido cargado al sistema.
 - Si $PPRxN$ es mayor que 100 % entonces se está ante un escenario donde una réplica tentativamente ha sido utilizada de forma concurrente. Esto significa una *sobre-utilización* de las réplicas, lo cual derivará en retraso y posibles encolamientos en el sistema.
- El balanceo de los accesos a las réplicas por cada nodo. Esta métrica se interpreta de la siguiente forma: Un sistema está balanceado cuando los nodos reciben una cantidad similar (igual en el caso ideal) de accesos a las réplicas. El sistema se encuentra desbalanceado cuando, en un tiempo dado, los accesos se concentran en una réplica dada, lo cual producirá un efecto de concurrencia produciendo a su vez un efecto de encolamiento en los accesos.
- Sobrecarga subyacente. Esta métrica mide el efecto de las interrupciones generadas por el sistema de almacenamiento al realizar las tareas de replicación (producción, distribución, manejo de metadatos, sincronización de accesos, retención de peticiones, etc). La magnitud del impacto de esta métrica está determinada por el *tiempo computacional* en el cual se realizan las tareas de replicación. De esta forma se espera que una técnica *activa* produzca una sobrecarga

súbita (todas las réplicas se realizan al mismo tiempo incrementando el nivel de concurrencia del sistema), lo cual producirá un incremento en el ancho de banda inmediato. En cambio, se espera que la técnica *pasiva-reactiva* producirá este efecto de forma gradual (las réplicas creadas se liberan de inmediato). Se espera, por tanto, que el ancho de banda total se libere cuando todas las réplicas sean liberadas. Esta métrica está embebida en la métrica *tiempo de respuesta* y se puede apreciar al restar los tiempos de una técnica tradicional a los tiempos producidos por una técnica de replicación elástica.

La Figura 5.18 muestra, en el eje vertical, el porcentaje promedio de acceso a réplicas por nodo ($PPRxN$) producido por las configuraciones estudiadas para cada nivel de concurrencia evaluado (eje horizontal). Como se puede observar, la técnica *activa* produce el mejor balance de accesos en los nodos (casi uniforme). Sin embargo, al mantener un factor de replicación (FR) y una cantidad de nodos de almacenamiento (RD) estáticos, cuando se incrementa la concurrencia se produce un crecimiento exponencial en la métrica $PPRxN$. Como se ha explicado previamente, es deseable que esta métrica se mantenga fluctuando en 100 %; cualquier valor por encima de 100 % resultaría en encolamientos, incrementando la probabilidad de que las operaciones tengan que esperar en cola, lo cual incrementaría los tiempos de respuesta del sistema, degradando su rendimiento y, por tanto, afectando la experiencia de los usuarios finales. En la misma figura se observa que la técnica de replicación *pasiva* también produce un comportamiento exponencial en los accesos producidos por la técnica *activa* cuando el nivel de concurrencia aumenta. Adicionalmente, y a diferencia de la técnica de replicación *activa*, la *pasiva* no produce un balanceo de carga uniforme, mas bien existe un desequilibrio en el balanceo de accesos, lo cual principalmente ocurre debido a que la técnica de replicación *pasiva* no libera las FR réplicas para que el sistema las pueda utilizar sino que las va liberando gradualmente (una réplica es utilizada inmediatamente una vez que es escrita en un nodo). Esto produce que, en un ambiente concurrente, los nodos que reciban la primera réplica asuman todos los accesos, lo cual produce un efecto de desequilibrio en el balance de accesos por nodo.

La Figura 5.19 muestra una comparación entre la técnica replicación *activa* y una de las técnicas

propuestas, *activa-elástica*, en la cual la cantidad de nodos (cajas) que se expresa con el factor RD es variable en tiempo de ejecución y también la cantidad de réplicas (número de esferas expresado como FR). Como se puede observar, la técnica *activa-elástica* preserva un balanceo de carga uniforme al igual que la técnica *activa*, sin embargo mantiene el valor de PPR_{xN} por debajo, igual o ligeramente por encima del 100 %. Esto básicamente se debe al dinamismo de la técnica elástica, la cual produce un efecto en cual, en promedio cada réplica almacenada por un nodo recibe un acceso por cliente, lo cual disminuye la probabilidad de encolamientos en el sistema.

La Figura 5.20 muestra una comparación entre el PPR_{xN} producido por la técnica de replicación *pasiva* y el PPR_{xN} producido por una de las técnicas propuesta, *pasiva-elástica*. Se puede observar un efecto contrario al observado en la técnica *pasiva* al incrementar el nivel de concurrencia, el cual estabiliza el valor medio de PPR_{xN} manteniéndolo por debajo del valor ideal de 100 % (marcado por la línea punteada roja). Como se puede observar, el acceso promedio de la técnica elástica no solo no crece como lo hace la técnica pasiva original, sino que desciende tendiendo al valor ideal. Lo cual explica por qué la técnica elástica ofrece mejores tiempos de respuesta que la técnica pasiva tradicional.

La Figura 5.21 muestra una comparación del valor medio de PPR_{xN} para las 2 técnicas propuestas en este trabajo de investigación, *activa-elástica* y *pasiva-elástica*. Lo primero que se puede observar es el balanceo casi uniforme de accesos que consigue la *activa-elástica*, algo que no consigue la técnica *pasiva-elástica*, la cual, a pesar de lo mencionado, produce una media de accesos que tiende a estabilizarse en un *cenit* produciendo un efecto similar a la *activa-elástica*, pero sin el balanceo de carga ofrecido por esta última. Este comportamiento es producido debido a que la técnica de replicación *pasiva-elástica* produce un comportamiento conservador, en el cual la cantidad de nodos (número de cajas expresado por la variable RD) se mueve en tiempo de ejecución con el objetivo de reducir el uso de recursos (RD y FR). En cambio el objetivo de la técnica *activa-elástica* es reducir los accesos por debajo del nivel ideal mediante el incremento acelerado de los valores de RD y FR . La técnica *pasiva-elástica* mostró ser conservadora en términos de RD , esto es, creando una

cantidad de nodos menor, como se observa en el nivel de concurrencia igual a 3, donde esta técnica ha determinado crear únicamente 2 nodos (cajas). También se mantuvo conservadora en términos *FR* creando un número menor de réplicas que su contra parte *activa-elástica*. Esto significa que esta técnica produce una *sobrecarga subyacente* menor a la producida por la técnica *activa-elástica*, lo cual impacta en escenarios de concurrencia media alta.

Tomando en cuenta el análisis previo, se puede concluir que las técnicas elásticas son más efectivas que las técnicas tradicionales en escenarios de accesos concurrentes. Entre las técnicas elásticas propuestas en este trabajo de investigación, la técnica *activa* es la mejor posicionada para escenarios de concurrencia baja, donde los beneficios del ancho de banda y el balanceo de carga son mayores a la *sobrecarga subyacente* producida por esta técnica. En cambio la técnica *activa-elástica* ha mostrado un mejor comportamiento cuando se incrementa la concurrencia, lo cual se debe principalmente a que, en escenarios de alta concurrencia, la *sobrecarga subyacente* se vuelve más significativa que los beneficios de la apertura del ancho de banda y el balanceo de carga. Esto hace que esta técnica ofrezca mejor rendimiento en escenarios de alta concurrencia.

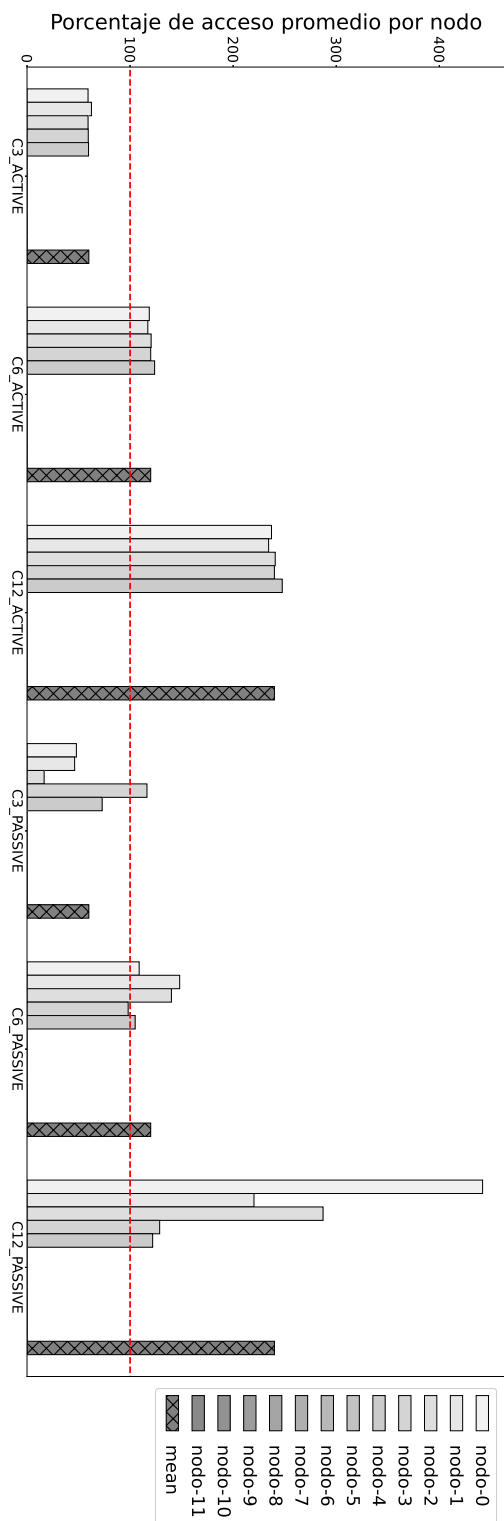


Figura 5.18: Comparación entre activa y pasiva.

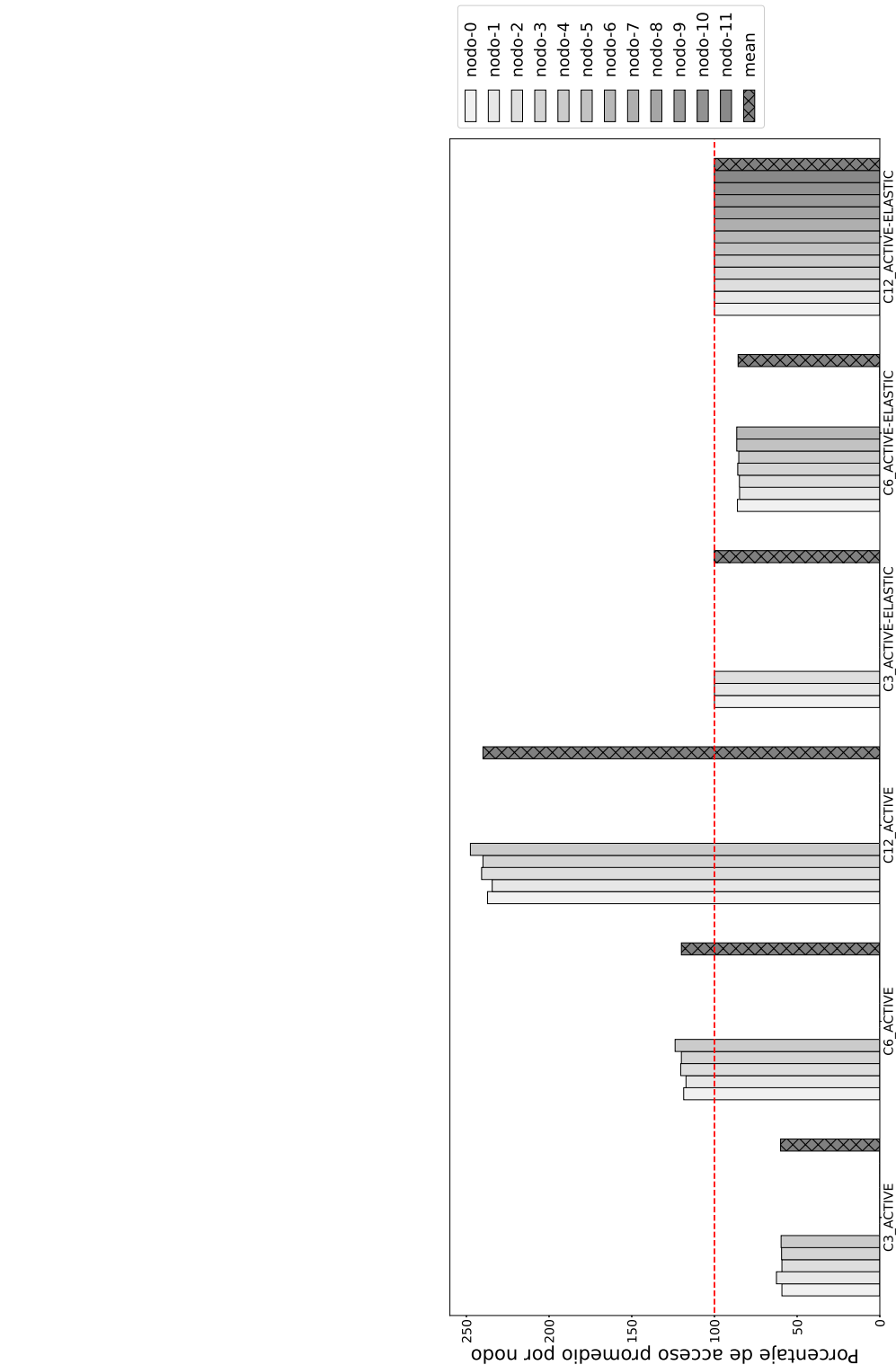


Figura 5.19: Comparación entre técnicas activa y activa elástica.

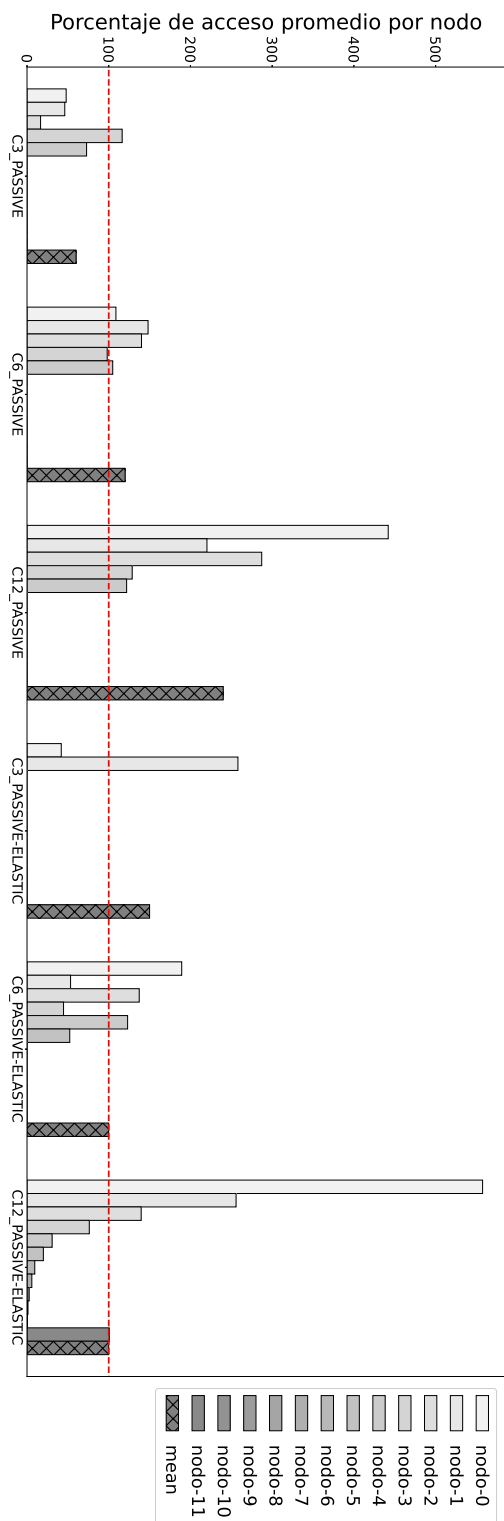


Figura 5.20: Comparación entre pasiva y pasiva elástica.

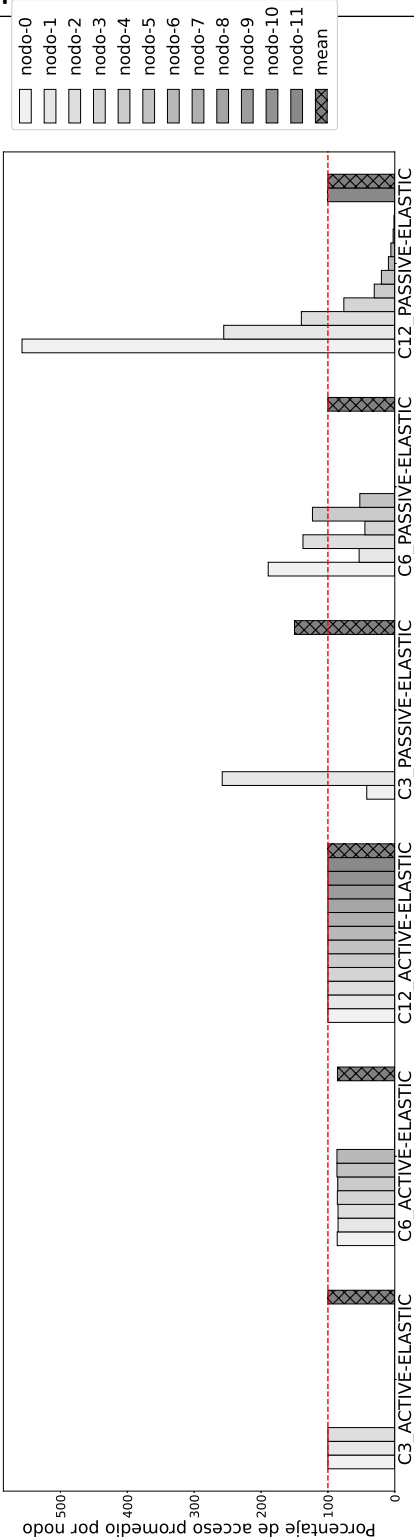


Figura 5.21: Comparación entre activa elástica y pasiva elástica.

6

Conclusiones y trabajo futuro

En este capítulo se describen las conclusiones obtenidas a partir del análisis de la implementación y los resultados de la experimentación realizada, asimismo se describen las limitaciones, la aportación y los aspectos de trabajo futuro a corto plazo que pudieron abordarse a partir de este trabajo de investigación.

6.1 Resumen

En esta tesis se propuso un método adaptativo-reactivo de replicación para sistemas de almacenamiento de alta disponibilidad. Este método tiene la característica que es adaptativo, lo que significa que puede ir cambiando con base a las interacciones realizadas por los usuarios en tiempo de ejecución, las cuales se dan en entornos de alta concurrencia (múltiples usuarios realizando operaciones de carga/descarga al mismo tiempo o en tiempos muy cortos). Esa concurrencia incrementará los tiempos de espera observados por los usuarios finales. El método propuesto toma como criterio los tiempos de espera para ir adaptándose y haciendo frente a la carga de trabajo,

lo cual realiza de forma reactiva, eso significa que en cuanto éste detecte algún cambio reacciona ante el evento acontecido adaptando así la cantidad de réplicas (FR) y la cantidad de recursos disponibles (RD), mejorando los tiempos de respuesta, el rendimiento y manteniendo una carga de trabajo uniforme, como se muestra en los resultados de la experimentación.

El método se compone de cinco etapas, las cuales se describen brevemente a continuación:

1. Etapa 1. En esta etapa se seleccionan los nodos de almacenamiento que presentan problemas en los tiempos de espera, debido a la sobrecarga causada principalmente por los accesos concurrentes.
2. Etapa 2. En esta etapa se filtran los contenidos que han sido más accedidos de los nodos que se han filtrado en la primera etapa. Los contenidos que sobrepasen la media global de accesos serán los contenidos a replicar.
3. Etapa 3. En esta etapa, se verifica si es necesario crear nodos para satisfacer todos los factores de replicación de cada contenido. En caso de que no se pueda satisfacer esto debido a que se cumple que $FR > RD$, entonces se deberán crear $FR - RD$ nodos de almacenamiento para satisfacer al nuevo valor de FR .
4. Etapa 4. En esta etapa se distribuyen las réplicas utilizando un algoritmo de balanceo de carga, y se colocan los nodos de almacenamiento en los servidores físicos, asignando a ellos una cantidad de recursos (memoria RAM, núcleos y espacio en disco).
5. Etapa 5. En esta etapa se describe la replicación deseada mediante un conjunto de cláusulas, estas cláusulas dan respuesta a 5 preguntas:
 - *¿Qué esferas se van a replicar?*
 - *¿Quién inicia la replicación?*
 - *¿Dónde se deben colocar las nuevas réplicas?*

- *¿Cómo debe efectuarse la replicación?*
- *¿Cuándo debe de accionarse la replicación?*

Si bien las preguntas pueden ser respondidas por el administrador del sistema, lo ideal es que éstas sean respondidas por un *bot* para evitar ambigüedades y otorgar una mejor respuesta a cada una de las preguntas y el proceso de replicación se realice de manera adecuada.

El método propuesto puede hacer dos tipos de replicación que no existen en el estado de arte:

1. *Replicación activa-elástica*, la cual se comprobó mediante experimentación controlada y se pudo observar en los resultado finales que permite mantener una distribución uniforme en los accesos a las réplicas y mejorar los tiempos de respuesta, con lo cual se incrementa el rendimiento del sistema. Sin embargo, también se observó que en entornos de alta concurrencia su rendimiento disminuye debido a la carga de trabajo subyacente (*SW*) y la forma en la cual retiene las réplicas, incrementando los tiempos de respuesta. Pero también se observó que mantiene un porcentaje promedio de acceso a las réplicas por nodo ($PPRxN$) igual al 100 %.
2. *Replicación pasiva-elástica*, la cual mediante experimentación controlada se comprobó que mejora el rendimiento y reduce los tiempos de respuesta. Esto ocurre cuando la técnica *pasiva-elástica* se enfrenta a entornos donde los niveles de concurrencia comienzan a incrementarse manteniendo un porcentaje promedio de acceso a las réplicas por nodo ($PPRxN$) igual a 100 %, lo que significa que todas las réplicas han tenido al menos un acceso y han sido utilizadas. Sin embargo, por la naturaleza de la técnica, algunos nodos reciben más accesos que otros. También la técnica ha mostrado ser conservadora manteniendo un número de réplicas menor, lo que mejora el consumo de almacenamiento y de recursos.

Las características generales del método son las siguientes:

- El método propuesto permite crear múltiples políticas de replicación mediante un modelo contextual de definición para diseñar el tipo de replicación que se desea.

- Con este modelo contextual de definición se recrearon las técnicas de replicación tradicionales, tales como *activa* y *pasiva*.
- Se crearon dos nuevas técnicas de replicación reactivas-adaptativas llamadas *activa-elástica* y *pasiva-elástica*. Ambas basadas en el método propuesto en este trabajo de tesis.
- Las técnicas tradicionales, específicamente la activa, son efectivas cuando el nivel de concurrencia es bajo y las ráfagas de descargas son limitadas o contenidas. En cambio las técnicas elásticas (activa y reactiva) son más efectivas que las técnicas tradicionales.
- De las técnicas elásticas, la técnica pasiva es más efectiva en escenarios de alta degradación ya que son menos invasivas que las técnicas activas. En cambio, la técnica activa produce mejores resultados en escenarios de degradación incipiente debido a que el impacto del ancho de banda ampliado que produce la técnica *activa elástica* compensa la sobrecarga subyacente que ésta produce en tiempo de ejecución. Sin embargo esta técnica tiende a dejar de ser efectiva en presencia de cargas de trabajo de alta concurrencia, lo cual es esperado para escenarios de manejo de servicios.

A partir del método propuesto se implementó un prototipo funcional, con el cual se hicieron diversos experimentos en distintos escenarios. Con la experimentación realizada se evaluó el prototipo siguiendo las métricas 1) tiempo de respuesta, 2) rendimiento y 3) porcentaje promedio de acceso a replica por nodo ($PPRxN$). Los resultados de la experimentación demuestran la viabilidad del método propuesto en escenario de alta demanda de almacenamiento, lo cual abre la puerta a explorar otras líneas de trabajo.

A partir del trabajo realizado, se pudo cumplir con los objetivos de investigación planteados al inicio del trabajo:

Objetivo específico 1. *Crear un método de construcción y despliegue de esquemas de almacenamiento basado en un modelo de codificación contextual, con elasticidad del parámetro*

RD para mejorar el ancho de banda del sistema de almacenamiento. Este objetivo se cumplió mediante la definición de cinco etapas que componen al método propuesto.

Objetivo específico 2. *Definir una estrategia que combine la replicación sistémica (elasticidad del parámetro RD) con una replicación de datos adaptable, basada en las interacciones de carga y descarga de los usuarios (definición dinámica de FR) para mejorar la probabilidad de disponibilidad.* Este objetivo se cumplió mediante la implementación de un prototipo que cuenta con un conjunto de componentes que interactúan entre sí y que llevan a cabo las etapas que componen al método para conseguir la replicación adaptable, tomando como criterio los tiempos de espera para accionarla y así hacer que el factor de replicación (FR) y el factor de disponibilidad de recursos (RD) sean dinámicos, adaptándose a la carga de trabajo en entornos concurrentes.

6.2 Limitaciones

El trabajo realizado se llevó a cabo a partir de un plan de actividades que se ajustó a un periodo de un año, por lo que algunos aspectos, inicialmente planteados, tuvieron alcance limitado. Las limitantes más representativas corresponden a dos observaciones que se notaron durante la experimentación. Estas limitantes, más que ser provocadas por el sistema de almacenamiento implementado con el prototipo, corresponden a lo que define el método propuesto. Estas limitantes son las siguientes:

1. Limitante 1. El sistema de almacenamiento no es capaz de detectar la cantidad de recursos disponibles en las máquinas físicas para colocar nodos de almacenamiento. Esto es una limitante debido a que en el método propuesto el balanceo de los nodos de almacenamiento se realiza de manera determinista sin tomar en cuenta los recursos de las máquinas disponibles para colocación de contenidos.
2. Limitante 2. El sistema de almacenamiento no es capaz de decidir el tiempo adecuado para

lanzar las réplicas causando el menor impacto en los tiempos de respuesta. Esto es causante de una carga de trabajo subyacente, la cual, como se observó en la experimentación, a medida que el nivel de concurrencia aumenta cada vez es más nociva para el sistema de almacenamiento.

6.3 Aportaciones

A partir del método propuesto y los resultados obtenidos a partir de la evaluación del prototipo, se puede afirmar que las aportaciones de este trabajo de investigación son las siguientes:

1. Un método de replicación adaptativo-reactivo, el cual utiliza los tiempos de espera como criterio para accionar la replicación de datos y sistémica en un sistema de almacenamiento.
2. Un prototipo funcional que implementa el método propuesto, el cual ha sido probado en la infraestructura de Cinvestav Tamaulipas y probado con ráfagas intensivas de carga y descarga de archivos. simulando un escenario lo más real posible.

6.4 Trabajo futuro

Como trabajo futuro a mediano plazo se buscara resolver algunas limitaciones presentadas en la sección anterior dando prioridad a las siguientes:

- *Predicción de un factor de replicación.* La selección de un factor de replicación es crucial, debido a que una mala selección de este factor podría ocasionar problemas de encolamiento, degradando el sistema o provocando una sub-utilización de la replicación gastando capacidad de almacenamiento. A través de probar diversas configuraciones de replicación y utilizando algoritmos de analítica se podría predecir un FR basado en las interacciones y el contexto del sistema.

- *Mejorar las decisiones.* En ocasiones el sistema podría presentar problemas por una mala configuración o simplemente porque ha llegado a un límite que le impida adaptarse, ocasionando encolamientos, principalmente, en las peticiones de descarga. Este problema se pretende remediar agregando la característica de elasticidad a nivel de sistema. Esto es, conectando diferentes conjuntos de piscinas de almacenamiento, de tal manera que las decisiones tomadas en el pasado puedan mejorar re-balanceando la carga en otro conjunto de piscinas de almacenamiento.
- *Técnicas de replicación híbridas:* Los resultados obtenidos han mostrado las virtudes individuales de las técnicas de replicación elásticas que se han propuesto en este trabajo de investigación. El desempeño de éstas se podría mejorar combinando entre sí estas virtudes. Se podría ver la manera de aprovechar de mejor forma el balanceo de carga uniforme de accesos proporcionado por la técnica de replicación *activa-elástica*, por ejemplo en entornos de alta concurrencia, que es donde la replicación *pasiva-elástica* muestra sus virtudes. Un reto en esto es aplicar ambas técnicas de replicación con niveles más altos de concurrencia.

6.5 Datos de contacto

Todo el trabajo reportado en esta tesis (código de las aplicaciones desarrolladas, compendio de artículos revisados, datos generados y adquiridos por cuenta propia) está disponible a petición de los interesados mediante los siguientes correos.

Jesús Ignacio Castillo Barrios

- Correo: ignacio.bcastillo@gmail.com
- Página web: <https://nachocodexx.github.io/>

José Luis González Compeán

- Correo: joseluis.gonzalez@cinvestav.mx
- Página web: <http://www.adaptivez.org.mx/Adaptivez2015/index.php>

Iván López Arévalo

- Correo: ilopez@cinvestav.mx
- Página web: <https://www.tamps.cinvestav.mx/~ilopez/>

Bibliografía

- [1] , B. C. and , D. C. (2010). Zatara, the Plug-in-able Eventually Consistent Distributed Database. *INTERNATIONAL JOURNAL ON Advances in Information Sciences and Service Sciences*, 2(3).
- [2] Abbes, H., Louati, T., and Cérin, C. (2020). Dynamic replication factor model for Linux containers-based cloud systems. *Journal of Supercomputing*, 76(9).
- [3] Ahamad, M., Neiger, G., Burns, J. E., Kohli, P., and Hutto, P. W. (1995). Causal memory: definitions, implementation, and programming. *Distributed Computing*, 9(1):37–49.
- [4] Ai, L., Deng, Y., Zhou, Y., and Feng, H. (2021). RUE: A caching method for identifying and managing hot data by leveraging resource utilization efficiency. *Software - Practice and Experience*, 51(11):2252–2273.
- [5] Al-Haj Hassan, O., Ramaswamy, L., Miller, J., Rasheed, K., and Canfield, E. R. (2009). Replication in overlay networks: A multi-objective optimization approach. *Lecture Notes of the Institute for Computer Sciences, Social-Informatics and Telecommunications Engineering*, 10 LNICST:512–528.
- [6] Andler, S. F., Hansson, J., Mellin, J., Eriksson, J., and Eftring, B. (2000). An Overview of the DeeDS Real-Time Database. 46(0).
- [7] Arnold, J. (2014). *Openstack swift: Using, administering, and developing for swift object storage*. “ O'Reilly Media, Inc.”.
- [8] Avanzados, E., Laboratorio, N., Gonz, L., and Sosa, S. (2016). Método de emplazamiento de datos adaptativo para aplicaciones en la nube Tesis que presenta : Pablo Morales Ferreira Para obtener el grado de :.

- [9] Bai, X., Jin, H., Liao, X., Shi, X., and Shao, Z. (2013). RTRM: A response time-based replica management strategy for cloud storage system. *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, 7861 LNCS(1).
- [10] Bermbach, D., Kuhlenkamp, J., Derre, B., Klems, M., and Tai, S. (2013). A middleware guaranteeing client-centric consistency on top of eventually consistent datastores. *Proceedings of the IEEE International Conference on Cloud Engineering, IC2E 2013*, pages 114–123.
- [11] Bokhari, S. M. A. and Theel, O. (2020). A Genetic Programming-Based Multi-Objective Optimization Approach to Data Replication Strategies for Distributed Systems. *2020 IEEE Congress on Evolutionary Computation, CEC 2020 - Conference Proceedings*.
- [12] Bramson, M., Lu, Y., and Prabhakar, B. (2010). Randomized load balancing with general service time distributions. *Performance Evaluation Review*, 38(1 SPEC. ISSUE).
- [13] Brazier, F., Cornelissen, F., Gustavsson, R., Jonker, C. M., Lindeberg, O., Polak, B., and Treur, J. (1998). Agents negotiating for load balancing of electricity use. *Proceedings - International Conference on Distributed Computing Systems*, pages 622–629.
- [14] Brikman, Y. (2017). *Terraform: Up and Running: Writing Infrastructure as Code*. “ O'Reilly Media, Inc.”.
- [15] Bsoul, M., Al-Khasawneh, A., Abdallah, E. E., and Kilani, Y. (2011). Enhanced fast spread replication strategy for data grid. *Journal of Network and Computer Applications*, 34(2).
- [16] Burrows, M. (2006). The Chubby lock service for loosely-coupled distributed systems. *OSDI 2006 - 7th USENIX Symposium on Operating Systems Design and Implementation*, pages 335–350.

- [17] Calder, B., Wang, J., Ogus, A., Nilakantan, N., Skjolsvold, A., McKelvie, S., Xu, Y., Srivastav, S., Wu, J., Simitci, H., et al. (2011). Windows azure storage: a highly available cloud storage service with strong consistency. In *Proceedings of the Twenty-Third ACM Symposium on Operating Systems Principles*, pages 143–157. ACM.
- [18] Cao, J., Spooner, D. P., Jarvis, S. A., Saini, S., and Nudd, G. R. (2003). Agent-based grid load balancing using performance-driven task scheduling. *Proceedings - International Parallel and Distributed Processing Symposium, IPDPS 2003*, 00(C).
- [19] Carlson, M., Yoder, A., Schoeb, L., Deel, D., Pratt, C., Lionetti, C., and Voigt, D. (2014). Software defined storage. *Storage Networking Industry Assoc. working draft*, pages 20–24.
- [20] Chen, T., Bahsoon, R., and Tawil, A. R. H. (2014). Scalable service-oriented replication with flexible consistency guarantee in the cloud. *Information Sciences*, 264.
- [21] Cheng, X., Dale, C., and Liu, J. (2008a). Statistics and social network of YouTube videos. *IEEE International Workshop on Quality of Service, IWQoS*.
- [22] Cheng, X., Dale, C., and Liu, J. (2008b). Statistics and social network of YouTube videos. *IEEE International Workshop on Quality of Service, IWQoS*, pages 229–238.
- [23] Cidon, A., Stutsman, R., Rumble, S., Katti, S., Ousterhout, J., and Rosenblum, M. (2013). MinCopssets : Derandomizing Replication In Cloud Storage. *Networked Systems Design and Impementation (NSDI)*.
- [24] Cooper, B. F., Ramakrishnan, R., Srivastava, U., Silberstein, A., Bohannon, P., Jacobsen, H. A., Puz, N., Weaver, D., and Yerneni, R. (2008). PNUTS: Yahoo!’s hosted data serving platform. *Proceedings of the VLDB Endowment*, 1(2):1277–1288.
- [25] Cortes, T. and Labarta, J. (2003). Taking advantage of heterogeneity in disk arrays. *Journal of Parallel and Distributed Computing*, 63(4).

- [26] DeCandia, G., Hastorun, D., Jampani, M., Kakulapati, G., Lakshman, A., Pilchin, A., Sivasubramanian, S., Voshall, P., and Vogels, W. (2007). Dynamo: Amazon's highly available key-value store. *SOSP'07 - Proceedings of 21st ACM SIGOPS Symposium on Operating Systems Principles*.
- [27] Drago, I., Mellia, M., M Munafo, M., Sperotto, A., Sadre, R., and Pras, A. (2012). Inside dropbox: understanding personal cloud storage services. In *Proceedings of the 2012 Internet Measurement Conference*, pages 481–494. ACM.
- [28] Edwin, E. B., Umamaheswari, P., and Thanka, M. R. (2019). An efficient and improved multi-objective optimized replication management with dynamic and cost aware strategies in cloud computing data center. *Cluster Computing*, 22(s5).
- [29] Elbushra, M. M. and Lindström, J. (2014). Eventual Consistent Databases: State of the Art. *Open Journal of Databases (OJDB)*, 1(1):1–16.
- [30] Gantz, J. and Reinsel, D. (2012). The digital universe in 2020: Big data, bigger digital shadows, and biggest growth in the far east. *IDC iView: IDC Analyze the future*, 2007(2012):1–16.
- [31] Ghemawat, S., Gobioff, H., and Leung, S.-T. (2003). The google file system.
- [32] Goda, K. (2016). Network Attached Storage. *Encyclopedia of Database Systems*, pages 1–1.
- [33] Gopinath, S. and Sherly, E. (2018). A Dynamic Replica Factor Calculator for Weighted Dynamic Replication Management in Cloud Storage Systems. *Procedia Computer Science*, 132.
- [34] Hidayat, T., Azzery, Y., and Mahardiko, R. (2020). Load Balancing Network by using Round Robin Algorithm: A Systematic Literature Review. *Jurnal Online Informatika*, 4(2):85.
- [35] Hussein, M. and Mousa, M. (2012). A Light-weight Data Replication for Cloud Data Centers Environment. *International Journal of Engineering and Innovative Technology (IJEIT)*, 1(6):169–175.

- [36] Jayalakshmi, D. S., Rashmi Ranjana, T. P., and Srinivasan, R. (2016). Dynamic Data Replication Strategy in Cloud Environments. *Proceedings - 2015 5th International Conference on Advances in Computing and Communications, ICACC 2015*.
- [37] Kleppmann, M. and Ceri, S. (2017). *Designing data-intensive applications: The big ideas behind reliable, scalable, and maintainable systems*. O'Reilly.
- [38] Kung, H. T. and Robinson, J. T. (1981). On Optimistic Methods for Concurrency Control. *ACM Transactions on Database Systems (TODS)*, 6(2):213–226.
- [39] Li, W., Yang, Y., and Yuan, D. (2011). A novel cost-effective dynamic data replication strategy for reliability in cloud data Centres. *Proceedings - IEEE 9th International Conference on Dependable, Autonomic and Secure Computing, DASC 2011*, pages 496–502.
- [40] Lin, J. W., Chen, C. H., and Chang, J. M. (2013). QoS-aware data replication for data-intensive applications in cloud computing systems. *IEEE Transactions on Cloud Computing*, 1(1).
- [41] Liu, K. and Dong, L. J. (2012). Research on cloud data storage technology and its architecture implementation. *Procedia Engineering*, 29:133–137.
- [42] Long, S. Q., Zhao, Y. L., and Chen, W. (2014). MORM: A Multi-objective Optimized Replication Management strategy for cloud storage cluster. *Journal of Systems Architecture*, 60(2):234–244.
- [43] Macedo, R., Paulo, J., and Pereira, J. (2020). A survey and classification of software-defined storage systems.
- [44] Mansouri, N. (2016). Adaptive data replication strategy in cloud computing for performance improvement. *Frontiers of Computer Science*, 10(5):925–935.
- [45] Mansouri, N., Javidi, M. M., and Zade, B. M. H. (2021). Hierarchical data replication strategy to improve performance in cloud computing. *Frontiers of Computer Science*, 15(2).

- [46] Mazumdar, S., Seybold, D., Kritikos, K., and Verginadis, Y. (2019). *A survey on data storage and placement methodologies for Cloud-Big Data ecosystem*, volume 6. Springer International Publishing.
- [47] Morales-Ferreira, P., Santiago-Duran, M., Gaytan-Diaz, C., Gonzalez-Compean, J., Sosa-Sosa, V. J., and Lopez-Arevalo, I. (2018). A data distribution service for cloud and containerized storage based on information dispersal. In *2018 IEEE Symposium on Service-Oriented System Engineering (SOSE)*, pages 86–95. IEEE.
- [48] Morris, K. (2016). *Infrastructure as code: managing servers in the cloud*. “O’Reilly Media, Inc.”.
- [49] Nair, C., Prabhakar, B., and Shah, D. (2001). The randomness in randomized load balancing. *Proceedings of the 39th Annual Allerton Conference on Communication, Control and Computing*, pages 912—921.
- [50] Nunes, B. A. A., Mendonca, M., Nguyen, X.-N., Obraczka, K., and Turletti, T. (2014). A survey of software-defined networking: Past, present, and future of programmable networks. *IEEE Communications Surveys & Tutorials*, 16(3):1617–1634.
- [51] Odun-Ayo, I., Ajayi, O., Akanle, B., and Ahuja, R. (2018). An overview of data storage in cloud computing. pages 38–42. Institute of Electrical and Electronics Engineers Inc.
- [52] Ranganathan, K. and Foster, I. (2001). Identifying dynamic replication strategies for a high-performance data grid. *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, 2242.
- [53] Reinsel, D., Gantz, J., and Rydning, J. (2018). The Digitization of the World - From Edge to Core. *Framingham: International Data Corporation*, (November).
- [54] Shvachko, K., Kuang, H., Radia, S., and Chansler, R. (2010). The Hadoop distributed file

- system. *2010 IEEE 26th Symposium on Mass Storage Systems and Technologies, MSST2010*, pages 1–10.
- [55] Singh, K. (2016). *Ceph Cookbook*. Packt Publishing Ltd.
- [56] Sosa-Sosa, V. J., Barron, A., Gonzalez, J. L., Carretero, J., and Lopez-Arevalo, I. (2020). Improving Performance and Capacity Utilization in Cloud Storage for Content Delivery and Sharing Services. *IEEE Transactions on Cloud Computing*, 7161(c).
- [57] Stacks, B. and Instances, A. (2015). In this section. *AWS CloudFormation*, page 104.
- [58] Sun, D. W., Chang, G. R., Gao, S., Jin, L. Z., and Wang, X. W. (2012). Modeling a dynamic data replication strategy to increase system availability in cloud computing environments. *Journal of Computer Science and Technology*, 27(2):256–272.
- [59] Unknown (2022). Compose specifications. <https://docs.docker.com/compose/compose-file/>. Accessed: 2022-05-30.
- [60] Wei, Q., Veeravalli, B., Gong, B., Zeng, L., and Feng, D. (2010). CDRM: A cost-effective dynamic replication management scheme for cloud storage cluster. *Proceedings - IEEE International Conference on Cluster Computing, ICC*.
- [61] Westphal, A., Dufrasne, B., Brandenburg, J., Jamsek, J., Jehnen, K., Joseph, S., Olivieri, M., Rendels, U., Rodriguez, M., et al. (2012). *IBM System Storage DS8000: Architecture and Implementation*. IBM Redbooks.
- [62] Yuan, Y., Wu, Y., Yang, G., and Yu, F. (2007). Dynamic data replication based on local optimization principle in data grid. *Proceedings of the 6th International Conference on Grid and Cooperative Computing, GCC 2007*, (Gcc):815–822.

- [63] Zeng, Z. and Veeravalli, B. (2014). Optimal metadata replications and request balancing strategy on cloud data centers. *Journal of Parallel and Distributed Computing*, 74(10):2934–2940.